UNIVERSITÀ DELLA CALABRIA

Dipartimento di Ingegneria Informatica, Modellistica, Elettronica e Sistemistica

Corso di Laurea Magistrale in Ingegneria Elettronica

Design e caratterizzazione di IP Core su FPGA per funzioni di attivazione in modelli di Intelligenza Artificiale

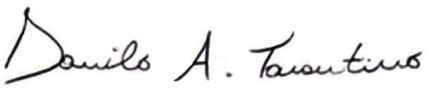
Relatrice:

Prof.ssa Stefania Perri

Candidato:

Danilo Antonio Tarantino

Matr. 247705



ANNO ACCADEMICO

2025/2026

"Studia duramente ciò che ti interessa di più
nel modo più indisciplinato, irriverente
e originale possibile".

RICHARD FEYNMAN

Indice

Elenco degli acronimi III

Elenco delle immagini IV

Elenco delle tabelle VI

Introduzione 1

Capitolo 1 – Transformers: architettura e caratteristiche 3

1.1 L'evoluzione del DL: dai Transformers alla visione artificiale 4

1.2 Struttura Transformer 6

1.3 Il meccanismo di self-attention 8

1.4 Sfide computazionali ed evoluzioni: verso l'efficienza hardware 10

1.5 Ottimizzazione e impatto delle funzioni di attivazione 11

1.6 Funzione sigmoide 14

1.6.1 Descrizione matematica e geometrica 14

1.6.2 Ambiti applicativi 16

1.6.3 Criticità e limiti computazionali 17

Capitolo 2 – Implementazione hardware sigmoide 19

2.1 Progettazione architettura baseline V1 20

2.1.1 Fase di pre-elaborazione dei dati 21

2.1.2 Blocco di pre-processing baseline 22

2.1.3 Blocco di memoria baseline 24

2.1.4 Architettura baseline complessiva 27

2.2 Fase di test e validazione baseline 29

2.2.1 Simulazione temporale post-implementazione 31

2.2.2 Confronto hardware/software 35

2.3 Report post-layout architettura baseline 37

2.3.1 Utilizzo risorse 38

2.3.2 Timing 38

2.3.3 Potenza dissipata 39

Capitolo 3 – Ottimizzazione dell'architettura 43
3.1 Progettazione architettura intermedia V2 44
3.1.1 Affinamento della quantizzazione 45
3.1.2 Blocco di pre-processing intermedio 46
3.1.3 Blocco di memoria intermedio 50
3.1.4 Report post-layout e risultati architettura V2 51
3.2 Progettazione architettura avanzata V3 60
3.2.1 Package Q1.15 61
3.2.2 Blocco di pre-processing avanzato 62
3.2.3 Blocco di memoria avanzato 66
3.2.4 Report post-layout e risultati architettura V3 67
Capitolo 4 – Confronto architetture 73
4.1 Analisi delle diverse implementazioni 74
4.2 Parallelismo 75
4.3 Comportamento su dataset reale 83
4.4 Scalabilità 86
4.5 Confronto FPGA vs CPU 101
Conclusioni 107
Bibliografia 111
Appendice A: Implementazione software e generazione coefficienti (Python) 113
Appendice B: Descrizione hardware, progettazione HDL e test (Vivado) 117
Appendice C: Grafici, risultati e metriche di confronto dell'errore (MATLAB) 141
Ringraziamenti 149

Elenco degli acronimi

AF – *Activation Function*: Funzione di attivazione
AI – *Artificial Intelligence*: Intelligenza artificiale
BERT – *Bidirectional Encoder Representations from Transformers*: Modello di linguaggio basato su transformer
BRAM – *Block Random Access Memory*: Memoria ad accesso casuale a blocchi
CNN – *Convolutional Neural Network*: Rete neurale convoluzionale
CPU – *Central Processing Unit*: Unità di elaborazione centrale
CV – *Computer Vision*: Visione artificiale
DL – *Deep Learning*: Apprendimento profondo
DNN – *Deep Neural Network*: Rete neurale profonda
FFN – *Feed-Forward Network*: Rete neurale con flusso in avanti
FPGA – *Field Programmable Gate Array*: Logica programmabile
GeLU – *Gaussian Error Linear Unit*: Funzione di attivazione
GPT – *Generative Pre-trained Transformer*: Famiglia di modelli generativi basati su Transformer
GPU – *Graphics Processing Unit*: Unità di elaborazione grafica
HDL – *Hardware Description Language*: Linguaggio di descrizione hardware (es. VHDL)
LUT – *Look Up Table*: Tabella di ricerca
MAE – *Mean Absolute Error*: Errore medio assoluto
MLP – *Multi-Layer Perceptron*: Rete neurale a percettrone multistrato
NLP – *Natural Language Processing*: Elaborazione del linguaggio naturale
RAM – *Random Access Memory*: Memoria ad accesso casuale
ReLU – *Rectified Linear Unit*: Funzione di attivazione
RNN – *Recurrent Neural Network*: Rete neurale ricorrente
ROM – *Read-Only Memory*: Memoria a sola lettura
OOC – *Out Of Context*: analisi del modulo isolato nel chip
PE – *Processing Elements*: istanze parallele di un modulo progettuale
SAIF – *Switching Activity Interchange Format*: Formato di interscambio dell'attività di commutazione
TB – *Testbench*: Ambiente di prova per la simulazione
VHDL – *VHSIC Hardware Description Language*: Linguaggio di descrizione hardware utilizzato per modellare, simulare e sintetizzare sistemi digitali
VHSIC – *Very High Speed Integrated Circuit*: Programma di ricerca governativo volto allo sviluppo di circuiti integrati ad alta velocità, da cui ha avuto origine lo standard del linguaggio VHDL
ViT – *Vision Transformer*: Architettura Transformer applicata alla visione artificiale
WHS – *Worst Hold Slack*: Peggior margine temporale di mantenimento
WNS – *Worst Negative Slack*: Peggior margine temporale negativo

Elenco delle immagini

Figura 1 – Architettura Transformer 7
Figura 2 – Self–attention (Sinistra). Multi–head attention (Destra) 9
Figura 3 – Confronto dell'accuratezza finale su dataset CIFAR-100 13
Figura 4 – Confronto dell'accuratezza finale su dataset T-IMNET 13
Figura 5 – Rappresentazione grafica sigmoide (MATLAB) 15
Figura 6 – Logica blocco pre-processing baseline 23
Figura 7 – Schematico post-layout blocco di pre-processing baseline 24
Figura 8 – Schematico post-layout blocco di memoria baseline (Distributed RAM) 26
Figura 9 – Logica blocco di memoria baseline 27
Figura 10 – Logica architettura baseline complessiva 28
Figura 11 – Schematico post-layout architettura baseline complessiva 29
Figura 12 – Logica testbench versione baseline 31
Figura 13 – Simulazione timing post-layout versione baseline 33
Figura 14 – Zoom dettagliato simulazione timing post-layout V1 34
Figura 15 – Forma d'onda analogica in simulazione timing post-layout V1 34
Figura 16 – Confronto MATLAB risultati V1 37
Figura 17 – Report di utilizzo risorse architettura baseline 38
Figura 18 – Report di timing architettura baseline 38
Figura 19 – Report di potenza (vectorless) architettura baseline 40
Figura 20 – Report di potenza (post-SAIF) architettura baseline 41
Figura 21 – Generazione coefficienti LUT da 1024 entry 45
Figura 22 – Logica blocco di pre-processing intermedio 46
Figura 23 – Schematico post-layout blocco di pre-processing intermedio 47
Figura 24 – Analisi temporale dettagliata pre-processing V2 49
Figura 25 – Schematico post-layout blocco di memoria intermedio (BRAM) 51
Figura 26 – Report di utilizzo V2 52
Figura 27 – Report di timing V2 52
Figura 28 – Report di potenza V2 (vectorless) 53
Figura 29 – Confronto MATLAB risultati V2 55
Figura 30 – Report di potenza V2 (SAIF relativo a TB coerenza) 56

Figura 31 – Forma d'onda analogica in simulazione V2 (TB robustezza) 57
Figura 32 – Report di potenza V2 (SAIF relativo a TB robustezza) 58
Figura 33 – Simulazione timing post-layout V2 (TB alta risoluzione) 59
Figura 34 – Forma d'onda analogica in simulazione V2 (TB alta risoluzione) 59
Figura 35 – Report di potenza V2 (SAIF relativo a TB alta risoluzione) 60
Figura 36 – Logica blocco di pre-processing avanzato 64
Figura 37 – Schematico post-layout blocco di pre-processing avanzato 65
Figura 38 – Schematico post-layout blocco di memoria avanzato 67
Figura 39 – Simulazione timing post-layout V3 (TB coerenza) 68
Figura 40 – Forma d'onda analogica simulazione V3 (TB coerenza) 69
Figura 41 – Confronto MATLAB risultati V3 70
Figura 42 – Report di utilizzo V3 71
Figura 43 – Report di timing V3 71
Figura 44 – Report di potenza V3 (vectorless) 72
Figura 45 – Report di potenza V3 (SAIF relativo a TB coerenza) 72
Figura 46 – Schematico post-layout architettura parallela V3.1 (4 moduli) 75
Figura 47 – Simulazione timing post-layout V3.1 (4 moduli) 77
Figura 48 – Forme d'onda analogiche in simulazione V3.1 (4 moduli) 78
Figura 49 – Report di utilizzo V3.1 (4 moduli) 79
Figura 50 – Report di timing V3.1 (4 moduli) 80
Figura 51 – Report di potenza V3.1 (4 moduli, vectorless) 81
Figura 52 – Report di potenza V3.1 (4 moduli, SAIF relativo a TB coerenza) 81
Figura 53 – Conversione dataset reale 83
Figura 54 – Confronto MATLAB risultati V3.1 (4 moduli) con dataset reale 84
Figura 55 – Simulazione timing post-layout V3.1 (4 moduli) con dataset reale 85
Figura 56 – Report di potenza V3.1 (4 moduli, SAIF relativo a TB dataset reale) 85
Figura 57 – Analisi OOC modulo singolo 87
Figura 58 – Analisi OOC 2 moduli 88
Figura 59 – Analisi OOC 4 moduli 89
Figura 60 – Analisi OOC 8 moduli 90
Figura 61 – Analisi OOC 16 moduli 91
Figura 62 – Analisi OOC 32 moduli 92

Figura 63 – Analisi OOC 64 moduli.....93
Figura 64 – Analisi OOC 128 moduli 94
Figura 65 – Analisi OOC 256 moduli 95
Figura 66 – Analisi OOC 512 moduli 96
Figura 67 – Analisi OOC 1024 moduli 98
Figura 68 – Analisi elaborazione CPU Single-Thread 103
Figura 69 – Analisi elaborazione CPU Multi-Thread 104

Elenco delle tabelle

Tabella I – Caratterizzazione dei modelli Transformer per visione artificiale.....5
Tabella II – Confronto architetture.....74
Tabella III – Modulo sigolo vs parallelismo 2 e 4..... 82
Tabella IV – Analisi scalabilità in OOC..... 99
Tabella V – Confronto eterogeneo FPGA vs CPU..... 106

Introduzione

L’evoluzione dell’intelligenza artificiale (AI) ha subito una trasformazione radicale con l’introduzione dell’architettura Transformer [1]. Originariamente concepita per il linguaggio naturale (NLP), essa si distingue per il meccanismo di auto-attenzione (self-attention) [2], [3], capace di modellare le dipendenze globali nei dati più efficacemente delle strutture sequenziali classiche. Tale approccio è stato esteso con successo alla Computer Vision tramite i Vision Transformer (ViT) [4], superando le reti convoluzionali (CNN) per scalabilità e capacità di rappresentazione.

Nonostante i risultati eccellenti, l'adozione massiva dei transformer pone sfide ingegneristiche complesse. L'elevata densità di questi modelli richiede una potenza computazionale che spesso eccede le capacità delle architetture general-purpose. Sebbene l'elaborazione dei flussi tensoriali, tipica dei calcoli lineari, benefici del parallelismo delle GPU, il passaggio ai sistemi embedded e all'edge computing impone un cambio di paradigma. In tali scenari, il contenimento dei consumi e la garanzia di una latenza deterministica diventano vincoli progettuali prioritari, alla pari della precisione numerica.

Il collo di bottiglia risiede nella gestione delle componenti non lineari. Le funzioni di attivazione introducono le non-linearità necessarie affinché il modello mappi relazioni complesse, che altrimenti rimarrebbero confinate a una combinazione lineare. Benché rappresentino una frazione computazionale ridotta, esse richiedono il calcolo di funzioni trascendentali la cui complessità intrinseca ostacola una parallelizzazione efficiente su silicio. In assenza di accelerazione dedicata, la natura sequenziale di tali calcoli degrada il throughput di sistema. La logica programmabile (FPGA) emerge quindi come soluzione ideale, offrendo la versatilità necessaria per implementare pipeline ad hoc capaci di minimizzare la latenza preservando l'integrità del segnale.

Il presente elaborato si focalizza sulla progettazione e implementazione hardware di un acceleratore per la funzione sigmoide. Attraverso un approccio incrementale, l’obiettivo è realizzare un’architettura digitale capace di bilanciare precisione in virgola fissa (fixed-point) ed efficienza nell'uso delle risorse fisiche. La ricerca di un equilibrio tra fedeltà algoritmica e footprint hardware risulta essenziale per garantire la sostenibilità dei modelli di prossima generazione.

La trattazione è organizzata come segue:

- Il **Capitolo 1** delinea il framework teorico dei Transformers e del Deep Learning (DL), approfondendo il ruolo matematico delle funzioni di attivazione e analizzando i limiti computazionali che giustificano la necessità di acceleratori hardware dedicati su logica programmabile (FPGA).
- Il **Capitolo 2** descrive la progettazione e l'implementazione dell'architettura "baseline". In questa fase viene illustrato il passaggio dal modello analitico, derivato dal dataset di specifica, alla sintesi hardware. Vengono analizzati i moduli di pre-processing e l'integrazione della Look-Up Table (LUT) su Distributed RAM con aritmetica fixed-point (Q5.11). Il capitolo include la validazione funzionale e la prima analisi quantitativa dell'errore tramite il confronto tra i risultati hardware e i risultati ottenuti via software, evidenziando inoltre i limiti di timing riscontrati in fase di post-implementazione.
- Il **Capitolo 3** presenta l'evoluzione del design verso un'architettura ottimizzata. Vengono introdotte soluzioni avanzate, tra cui la logica di clipping per dataset dinamici, l'impiego della simmetria della funzione sigmoide, la migrazione verso risorse di memoria Block RAM (BRAM) dedicate e l'utilizzo del formato Q1.15 per l'output. Viene inoltre analizzato l'effetto delle tecniche di pipelining sull'incremento della densità logica e sulla stabilità della frequenza operativa.
- Il **Capitolo 4** analizza i risultati sperimentali e le prestazioni del sistema. Include la validazione dell'architettura su dataset reali e un'analisi sistematica della scalabilità. Vengono inoltre definiti i limiti fisici e architetturali del design, con una discussione sul trade-off tra densità computazionale (parallelismo), consumo energetico e convergenza temporale. Il capitolo si conclude con un confronto prestazionale tra l'acceleratore FPGA proposto e le implementazioni standard su CPU, evidenziando i vantaggi in termini di latenza e determinismo.

Nota sulla riproducibilità: L'intero progetto è consultabile su GitHub, dove sono stati resi disponibili i codici sorgente e gli script di supporto per la replica integrale del flusso di lavoro, al seguente link: *Arsenico22/Sigmoid_Accelerator_IP-CORE*.

Capitolo 1 – Transformers: architettura e caratteristiche

L'evoluzione delle architetture neurali ha portato alla definizione di paradigmi profondamente diversi nel modo in cui i modelli interpretano l'informazione. La distinzione fondamentale tra l'approccio convoluzionale e quello basato sui Transformers risiede nella gestione del cosiddetto bias induttivo. Nelle CNN, la conoscenza della struttura del dato è codificata direttamente nell'architettura tramite i kernel, che impongono proprietà di località spaziale e invarianza alla traslazione. Al contrario, il Transformer adotta una strategia più flessibile, apprendendo direttamente da zero la struttura dei dati e le relazioni tra di essi tramite il meccanismo di self-attention, indipendentemente dalla loro distribuzione o correlazione iniziale.

In un contesto di elaborazione delle immagini, questa differenza si traduce nella capacità del modello di analizzare le relazioni globali tra tutte le parti della figura contemporaneamente, apprendendo le dipendenze strutturali direttamente dai dati anziché assumere a priori che i pixel vicini siano i più rilevanti. Se da un lato questa assenza di vincoli predefiniti consente prestazioni superiori, dall'altro richiede una mole di dati imponente per l'addestramento e introduce una densità di calcolo notevole, specialmente nella gestione delle componenti non lineari. Le diverse applicazioni di queste architetture nella visione artificiale sono riassunte sinteticamente nella tabella I.

In questo capitolo viene delineato il framework teorico necessario a comprendere le scelte progettuali successive, approfondendo in particolare:

- l'architettura Transformer: un'analisi del meccanismo di self-attention e della struttura dei blocchi neurali;
- le funzioni di attivazione: una rassegna del ruolo delle non-linearità, con un focus specifico sulla funzione sigmoide, analizzandone le proprietà matematiche e la criticità del calcolo in fixed-point;
- i limiti computazionali: un'analisi di come le funzioni trascendentali ostacolino l'efficienza sui sistemi hardware tradizionali, definendo i vincoli di precisione e risorse che giustificano lo sviluppo di un acceleratore dedicato su FPGA.

1.1 L'evoluzione del DL: dai Transformers alla visione artificiale

Il panorama dell'Intelligenza Artificiale ha subito una trasformazione radicale nell'ultimo decennio, passando da architetture specializzate per singoli domini a modelli universali basati sul meccanismo di attenzione. Inizialmente, lo sviluppo delle Deep Neural Networks (DNN) era dominato dalle CNN per l'elaborazione delle immagini e dalle RNN per le sequenze temporali e il linguaggio.

Tuttavia, la nascita dell'architettura Transformer ha segnato un punto di svolta. Originariamente concepiti per il NLP, modelli come BERT [5] e GPT-3 [6] hanno dimostrato una capacità senza precedenti nell'apprendere relazioni a lungo raggio tra i dati, superando i limiti intrinseci delle architetture precedenti. Questo successo non è rimasto confinato al testo: con l'introduzione dei ViT, il paradigma della self-attention è stato applicato con successo anche alla visione artificiale [7], [8], trattando le porzioni di un'immagine come se fossero parole di una frase.

Al giorno d'oggi, i modelli basati su Transformer rappresentano lo stato dell'arte in una vasta gamma di applicazioni. Per fornire una visione d'insieme dell'eterogeneità di questo ecosistema, è possibile consultare la tabella I. Tale classificazione è riportata a scopi puramente descrittivi, con l'obiettivo di evidenziare come la stessa struttura di base (il blocco Transformer) venga oggi declinata in task estremamente diversi: dalla classificazione di immagini (backbone), alla ricostruzione ad alta risoluzione (high-level vision), fino all'analisi temporale nei flussi video.

La tabella I evidenzia l'impressionante versatilità dei Transformers. Tuttavia, l'implementazione pratica di questi modelli solleva sfide ingegneristiche non trascurabili. La complessità computazionale richiesta per gestire milioni (o miliardi) di parametri rende questi sistemi estremamente energivori e difficili da integrare in dispositivi con risorse limitate. Il presente lavoro di tesi non mira all'analisi dei singoli task applicativi elencati in tabella, bensì si focalizza sull'ottimizzazione dei componenti hardware fondamentali che ne permettono il funzionamento. Nello specifico, il focus si sposta sulle funzioni di attivazione non lineari, come la sigmoide, che pur essendo essenziali per le prestazioni del modello, rappresentano un critico collo di bottiglia in termini di efficienza su circuiti digitali.

Tabella I – Categorizzazione dei modelli Transformer per visione artificiale

Categoria	**Sotto-categoria**	**Metodo**	**Punti chiave**
Backbone	Pre-addestramento supervisionato	ViT	Patch di immagine, Transformer standard
		TNT	Attenzione locale
		Swin	Auto-attenzione basata su finestre
	Pre-addestramento auto-supervisionato	iGPT	Apprendimento auto-supervisionato
		MoCo v3	Apprendimento contrastivo auto-supervisionato
Visione ad alto/medio livello	Rilevamento oggetti	DETR	Predizione basata su set, matching bipartito
		Deformable DETR	DETR con modulo di attenzione deformabile
		UP-DETR	Pre-addestramento non supervisionato, rilevamento con patch query casuali
	Segmentazione	Max-DeepLab	Matching bipartito stile PQ, Transformer a doppio percorso
		VisTR	Matching di sequenze di istanze e segmentazione
		SETR	Predizione sequence-to-sequence, Transformer standard
	Stima della posa	Hand-Transformer	Transformer non autoregressivo, set di punti 3D
		HOT-Net	Estrattore di riferimenti strutturati
		METRO	Riduzione progressiva della dimensionalità
Visione a basso livello	Generazione di immagini	Image Transformer	Generazione di pixel tramite Transformer
		Taming Transformer	VQ-GAN, Transformer autoregressivo
		TransGAN	GAN basata esclusivamente su architettura Transformer
	Miglioramento immagini	IPT	Multi-task, pre-addestramento ImageNet, modello Transformer
		TTSR	Transformer per texture, RefSR
Elaborazione video	Inpainting video	STTN	Perdita avversaria spazio-temporale
	Descrizione video	Masked Transformer	Rete di mascheramento, proposta di eventi
Multimodalità	Classificazione	CLIP	Supervisione NLP per immagini, trasferimento zero-shot
	Generazione di immagini	DALL·E	Generazione testo e immagine zero-shot
	Multi-task	CogView	VQ-VAE, input in cinese
		UniT	Compiti NLP e CV diversi, parametri condivisi
Transformer efficienti	Decomposizione	ASH	Numero di teste, stima dell'importanza
	Distillazione	TinyBERT	Varie perdite per moduli differenti
	Quantizzazione	FullyQT	Transformer completamente quantizzato
	Progettazione architetturale	ConvBert	Dipendenza locale, convoluzione dinamica

1.2 Struttura Transformer

Il Transformer, la cui struttura è mostrata in Fig. 1, si distacca radicalmente dalle precedenti architetture CNN grazie alla sua capacità di elaborare sequenze di dati in parallelo. Esso è basato su una struttura encoder-decoder [1], dove ogni componente è formata da una pila di blocchi identici che operano trasformazioni successive sui dati.

Mentre l'encoder ha il compito di mappare una sequenza di input in una rappresentazione continua che racchiuda il significato contestuale di ogni elemento, il decoder utilizza tali rappresentazioni per generare una sequenza di output, un elemento alla volta. Negli anni, questa struttura è stata declinata con successo anche nel dominio della visione artificiale (ViT), dove le immagini vengono suddivise in patch ed elaborate come sequenze di token [4], [9].

Ogni blocco fondamentale del Transformer è costituito da quattro sottoblocchi principali che lavorano in sinergia:

- Multi-Head Attention (MHA): Rappresenta il cuore pulsante dell'architettura. Attraverso il calcolo di matrici di *Query*, *Key* e *Value*, il meccanismo di attenzione permette al modello di pesare dinamicamente l'importanza di ogni parte della sequenza rispetto alle altre, catturando dipendenze a lungo raggio che risultano difficili da modellare con le reti convoluzionali o ricorrenti [1].
- Feed-Forward Network (FFN): Questo sottolivello è composto da due trasformazioni lineari intervallate da una funzione di attivazione non lineare. È proprio in questo modulo che risiede la capacità di apprendimento di pattern complessi: come analizzato in letteratura, la scelta della funzione di attivazione all'interno della FFN (come la GeLU o la sigmoide) è determinante per la convergenza e l'accuratezza finale del modello [10], [11].
- Layer Normalization: Applicata prima o dopo ogni sottolivello, la normalizzazione di strato è cruciale per stabilizzare il processo di addestramento, riducendo le fluttuazioni nei valori dei gradienti e permettendo l'uso di tassi di apprendimento più elevati.

- Residual Connections (Shortcuts): Queste connessioni di salto permettono al gradiente di fluire più facilmente attraverso i numerosi layer della rete, mitigando il problema del *vanishing gradient* e facilitando l'ottimizzazione di architetture estremamente profonde [12].

Questa combinazione di moduli permette al Transformer di scalare con estrema efficacia sia in termini di dimensioni del dataset che di parametri del modello, rendendolo la base per i moderni sistemi di intelligenza artificiale. Tuttavia, la gestione hardware di questi blocchi, specialmente per quanto riguarda il calcolo delle funzioni non lineari nella FFN, rappresenta una delle sfide ingegneristiche più rilevanti per l'inferenza in tempo reale su dispositivi embedded.

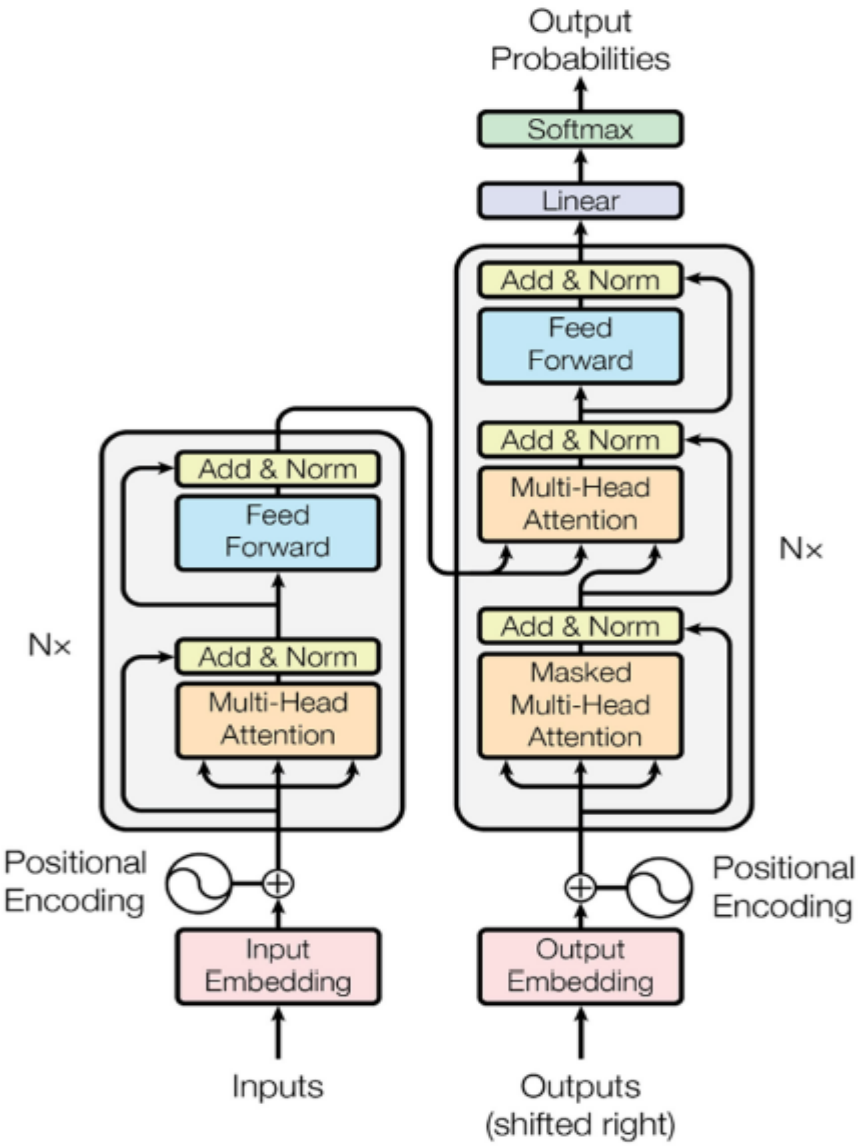


Figura 1 – Architettura Transformer

1.3 Il meccanismo di self-attention

Il meccanismo di self-attention (Scaled Dot-Product Attention), schematizzato in Fig.2, permette al modello di pesare dinamicamente l'importanza di diverse parti della sequenza di input. A differenza delle operazioni di convoluzione, vincolate da un campo ricettivo locale e fisso, o delle RNN, che soffrono di una memoria a breve termine dovuta alla natura sequenziale del calcolo, la self-attention consente a ogni elemento della sequenza di interagire con tutti gli altri simultaneamente. Questo permette di catturare dipendenze globali e relazioni semantiche indipendentemente dalla loro distanza spaziale o temporale.

Dal punto di vista matematico, dato un vettore di input (o un embedding di patch nel caso dei ViT), esso viene proiettato in uno spazio latente per generare tre rappresentazioni distinte mediante matrici di peso addestrabili W^Q, W^K, W^V, dove:

- **Query (Q):** rappresenta l'elemento attivo che interroga gli altri componenti della sequenza per estrarre informazioni rilevanti.
- **Key (K):** funge da indice descrittivo per ogni elemento della sequenza, permettendo di stabilire la "chiave" di accesso all'informazione.
- **Value (V):** racchiude il contenuto informativo effettivo che verrà propagato ai livelli successivi della rete.

Il calcolo dell'attenzione non è un singolo passaggio, ma un processo di filtraggio dinamico che segue quattro fasi logiche fondamentali:

1. **Calcolo dello score di somiglianza:** Si determina il grado di affinità tra la Query corrente e tutte le Key disponibili attraverso un prodotto scalare ($Q \cdot K^T$). Questo valore rappresenta quanto un elemento debba essere "ascoltato" rispetto agli altri.
2. **Scaling:** I punteggi ottenuti vengono divisi per un fattore di scala $\sqrt{(d_k)}$ (dove d_k è la dimensione del vettore delle chiavi). Questo passaggio è vitale per prevenire la saturazione del gradiente durante l'addestramento: senza di esso, il prodotto scalare potrebbe assumere magnitudo elevate, spingendo la successiva funzione softmax in regioni dove il gradiente è quasi nullo.

3. **Normalizzazione (softmax):** Si applica la funzione softmax per convertire gli score in una distribuzione di probabilità (pesi di attenzione) compresa nell'intervallo [0, 1]. È essenziale sottolineare che la softmax, essendo basata su funzioni esponenziali, condivide con la sigmoide le medesime criticità di implementazione hardware. Entrambe sono funzioni trascendentali che richiedono, in fase di sintesi su FPGA, l'utilizzo di approssimazioni o LUT per gestire l'elevata complessità computazionale.
4. **Aggregazione dei valori:** L'output finale Z è ottenuto come somma pesata dei vettori Value (V), moltiplicati per i coefficienti calcolati dalla softmax.

L'intero processo sopra descritto può essere riassunto matematicamente dalla seguente espressione, che definisce il meccanismo di Scaled Dot-Product Attention:

$$Attention(Q, K, V) = softmax\left(\frac{Q\,K^T}{\sqrt{d_k}}\right)V$$

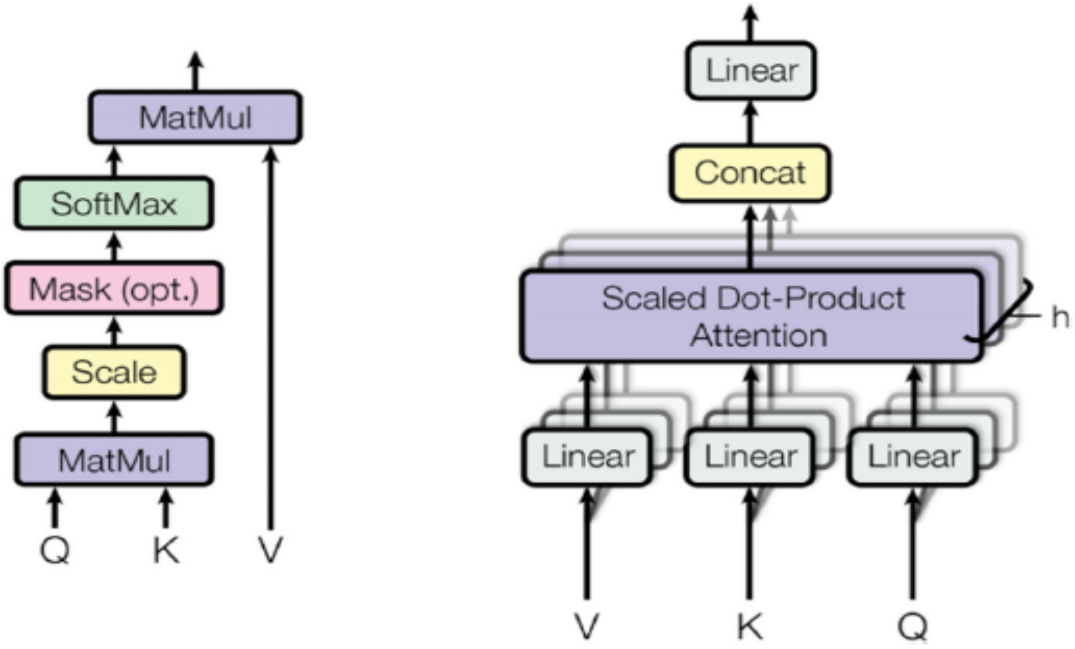


Figura 2 – Self–attention (Sinistra). Multi–head attention (Destra)

Immagine tratta da [1].

1.4 Sfide computazionali ed evoluzioni: verso l'efficienza hardware

Nonostante l'efficacia del modello ViT, la sua architettura originale e le sue varianti presentano limitazioni strutturali che hanno guidato l'evoluzione della ricerca verso l'efficienza. Il limite principale risiede nella complessità computazionale della self-attention, che cresce in modo quadratico rispetto al numero di patch. Questo rende i Transformers difficili da applicare a immagini ad altissima risoluzione o a contesti con risorse limitate senza un'adeguata ottimizzazione hardware e algoritmica.

Per ovviare a ciò, la ricerca si è mossa verso due direzioni: architetture gerarchiche e modelli specializzati:

- Swin Transformer: Introduce una struttura gerarchica basata su Finestre Locali e Shifted Windows [13], [14]. Questo riduce la complessità da quadratica a lineare, permettendo al modello di operare su scale locali pur mantenendo i vantaggi dell'attenzione globale.
- Modelli Specializzati (DETR e SETR): L'integrazione dei Transformers ha trasformato radicalmente settori come l'object detection e la segmentazione semantica. Tuttavia, l'uso di "object queries" in DETR [15], [16] o di encoder puri in SETR [17] aumenta drasticamente il numero di operazioni non lineari necessarie per processare i dati.

Questa massiccia diffusione dei Transformers in ogni ambito della visione artificiale ha reso critica la gestione del consumo di memoria e della potenza di calcolo, specialmente per applicazioni su dispositivi edge. Tecniche come la quantizzazione, il pruning dei pesi e l'uso di matrici di attenzione sparse sono fondamentali, ma non sufficienti.

Il vero collo di bottiglia risiede spesso nell'implementazione fisica delle funzioni di attivazione e normalizzazione (come la softmax e la sigmoide) che, a causa della loro natura trascendentale, non possono essere calcolate efficientemente con le unità logiche standard di una FPGA. È in questo scenario che si inserisce il presente lavoro: l'ottimizzazione del calcolo della funzione sigmoide diventa un tassello fondamentale per rendere sostenibili tutte le architetture sopra citate in contesti di elaborazione real-time e a basso consumo.

1.5 Ottimizzazione e impatto delle funzioni di attivazione

Un aspetto cruciale nella progettazione e nell'ottimizzazione dei Transformers riguarda la scelta delle componenti interne al blocco Multi-Layer Perceptron (MLP) o Feed-Forward Network (FFN), con particolare riferimento alle funzioni di attivazione (AF). Mentre nelle architetture CNN classiche la funzione ReLU (*Rectified Linear Unit*) è stata per lungo tempo lo standard utilizzato, l'avvento dei Transformers ha spostato l'attenzione verso operatori capaci di introdurre una non-linearità più complessa e continua, necessaria per mappare input non linearmente separabili in rappresentazioni astratte ad alta dimensionalità [18], [19].

Le funzioni di attivazione influenzano direttamente la dinamica dell'apprendimento, la stabilità del gradiente e i tempi di convergenza [10]. La ricerca recente ha evidenziato come l'uso di funzioni alternative alla ReLU possa mitigare fenomeni critici come il gradiente evanescente o la perdita di espressività in reti estremamente profonde [12], [20]. Nell'attuale panorama dei Vision Transformers (ViT), si osservano tre approcci dominanti:

- GeLU (Gaussian Error Linear Unit): Standard nei modelli ViT originali [4], questa funzione introduce una componente stocastica pesando l'input tramite una distribuzione gaussiana, garantendo un flusso del gradiente più morbido rispetto alla brusca discontinuità della ReLU.
- Famiglia delle Sigmoidi: Nonostante le criticità storiche legate alla saturazione, analisi empiriche condotte su dataset complessi come CIFAR-100 [21] e Tiny ImageNet [22], come mostrato in Fig. 3 e Fig. 4, hanno rivelato che le funzioni basate sulla sigmoide (incluse le varianti LogSigmoid e Hard Sigmoid) offrono performance superiori in termini di accuratezza finale proprio nei modelli ViT [4], [23].
- Varianti basate su Tanh: Spesso evitate per l'alto costo degli operatori esponenziali, studi su modelli *Tokens-to-Token* (T2T) [24] dimostrano che tali funzioni possono essere estremamente competitive se opportunamente ottimizzate a livello hardware.

La selezione di queste funzioni impone tuttavia un severo trade-off ingegneristico, di fatto esiste una correlazione diretta tra la complessità matematica della funzione e il costo computazionale associato. Ad esempio, architetture gerarchiche come lo Swin Transformer [14] mostrano una enorme sensibilità alla scelta della AF: un'impostazione errata può far crollare l'accuratezza del modello a valori marginali, mentre l'adozione di una funzione "smooth" coerente con l'architettura ne sblocca il pieno potenziale [25].

In un'ottica di implementazione su hardware dedicato (FPGA), questo scenario delinea la motivazione centrale del presente elaborato. Poiché i Transformers richiedono la valutazione di queste funzioni per milioni di parametri in tempo reale, l'efficienza nel calcolo di operatori non-lineari determina la fattibilità stessa del modello in contesti embedded.

L’analisi comparativa riportata in Fig. 3 evidenzia come l’accuratezza finale dei modelli Transformer sia fortemente influenzata dalla funzione di attivazione scelta. Si nota con particolare interesse che, nel modello ViT, le funzioni appartenenti alla famiglia delle sigmoidi (come la sigmoide standard e la LogSigmoid) superano o eguagliano le prestazioni della ReLU, raggiungendo i vertici della classifica di precisione sul dataset CIFAR-100. Questo dato smentisce il pregiudizio comune che vede la sigmoide come una funzione obsoleta per i layer interni, confermandone invece l'efficacia nelle architetture basate su attenzione.

In Fig. 4 viene illustrata la robustezza delle funzioni di attivazione su un dataset più complesso come Tiny ImageNet. Anche in questo scenario, si osserva che per architetture come ViT e PiT, la sigmoide garantisce risultati stabili e competitivi rispetto alla GeLU. Un dato critico emerge dall'osservazione dello Swin Transformer [14]: l'estrema variabilità dei risultati in questo modello dimostra che la scelta della AF non è intercambiabile, ma richiede una valutazione specifica. La stabilità dimostrata dalla sigmoide in diverse architetture ne giustifica l’analisi approfondita e l’ottimizzazione hardware proposta in questo studio.

Sebbene la funzione sigmoide sia stata talvolta messa in ombra da varianti più recenti per gli strati interni, essa rimane l'elemento cardine per i meccanismi di gating, normalizzazione e per le architetture moderne che ne sfruttano la derivata continua. La sfida di tradurre la precisione della sigmoide in un'architettura hardware ottimizzata rappresenta, dunque, il punto di giunzione tra l'eccellenza algoritmica e la sostenibilità computazionale.

Entrambe le figure mostrano il progresso del training su 100 epoche (asse X) e la corrispettiva precisione per ogni modello usato (asse Y).

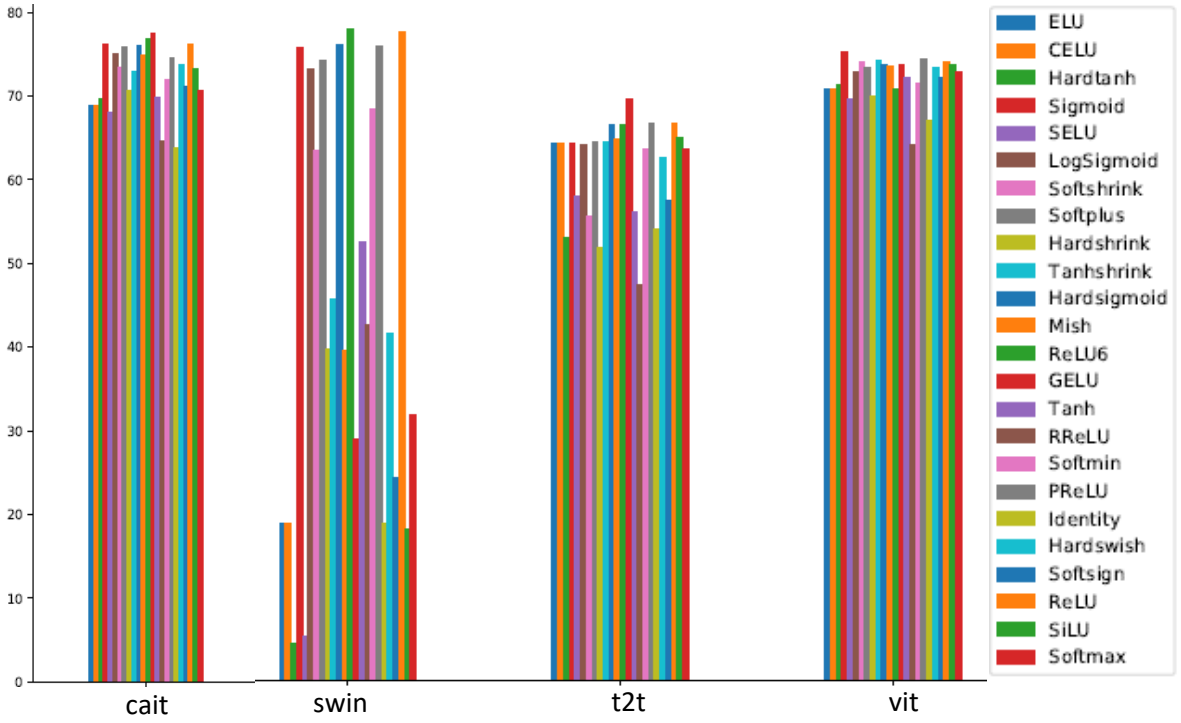


Figura 3 – Confronto dell'accuratezza finale su dataset CIFAR-100

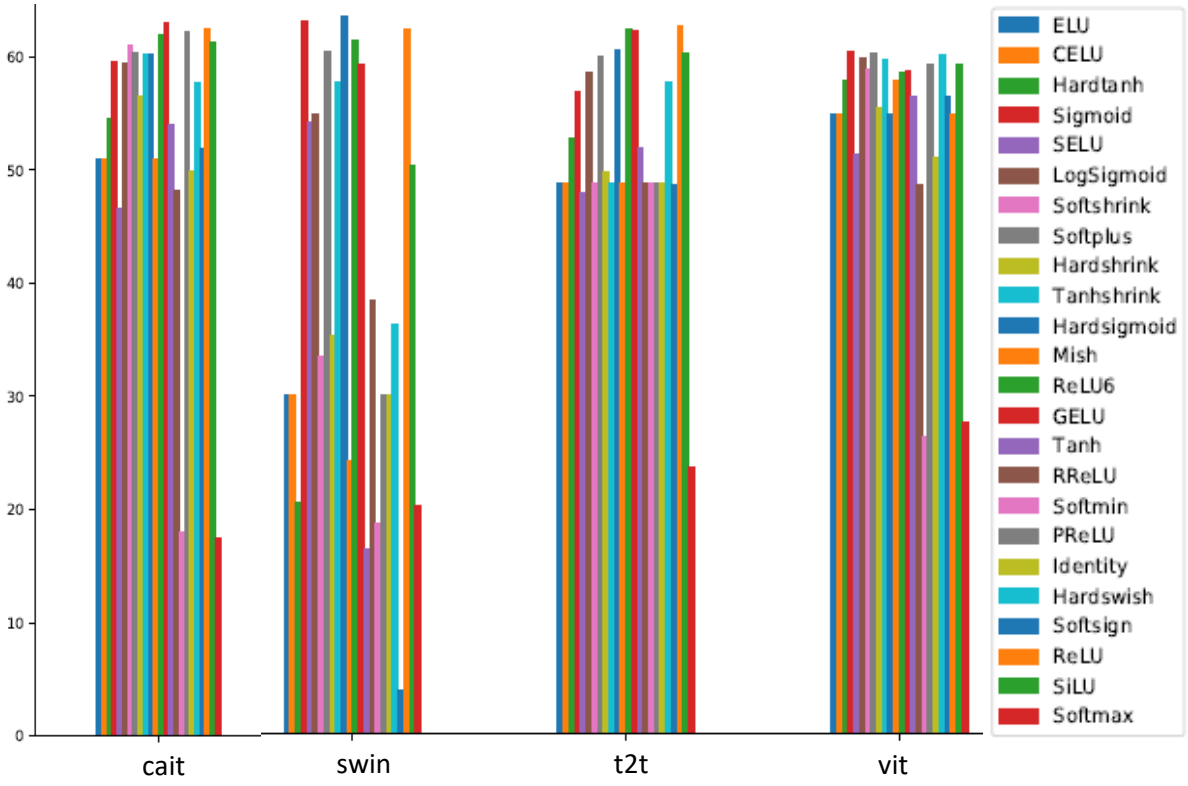


Figura 4 – Confronto dell'accuratezza finale su dataset T-IMNET

Proprio in virtù della sua versatilità e della capacità di mappare un intervallo reale arbitrario nell'intervallo limitato $[0\,;\,1]$, la funzione sigmoide si conferma un elemento insostituibile non solo nei moduli di gating, ma come componente cardine per l'intero processo di inferenza in architetture basate su attenzione. Il presente lavoro si focalizza dunque sulla sfida di tradurre tale precisione matematica in un'architettura hardware ottimizzata, capace di bilanciare le esigenze di accuratezza dei modelli moderni con i vincoli di banda e risorse tipici degli IP Core su FPGA.

1.6 Funzione sigmoide

La funzione matematica sigmoide, anche nota come funzione logistica, ricopre un ruolo fondamentale nelle reti neurali agendo come operatore non lineare. Il suo compito principale è trasformare lo spazio dei segnali di input, evitando che la rete si comporti come un unico operatore lineare. Senza l'introduzione di tali non-linearità, una rete neurale non sarebbe in grado di svolgere l'operazione di back-propagation attraverso la quale il modello apprende.

1.6.1 Descrizione matematica e geometrica

La funzione sigmoide ha la seguente espressione analitica:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

In Fig. 5 è presente il grafico della sigmoide realizzato in MATLAB, con codice sorgente riportato in Appendice C.1.

Le proprietà geometriche e analitiche principali includono:

- Dominio e Codominio: La funzione mappa l'intero asse reale nell'intervallo aperto (0, 1), presentando la caratteristica curva a "S".
- Continuità e Monotonia: Si tratta di una funzione limitata, continua e strettamente crescente.

- Simmetria: Possiede una simmetria centrale rispetto al punto (0, 0.5), che rappresenta il centro della transizione tra i due asintoti orizzontali.
- Proprietà della Derivata: La derivata prima può essere espressa in funzione della sigmoide stessa come:

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$$

- Il grafico della derivata ha una forma a campana, raggiungendo il valore massimo in corrispondenza dello zero.
- Concavità: La derivata seconda descrive la variazione di concavità, indicando il passaggio da un'accelerazione della crescita per valori negativi a una decelerazione per valori positivi, annullandosi nel punto di flesso (0, 0.5).

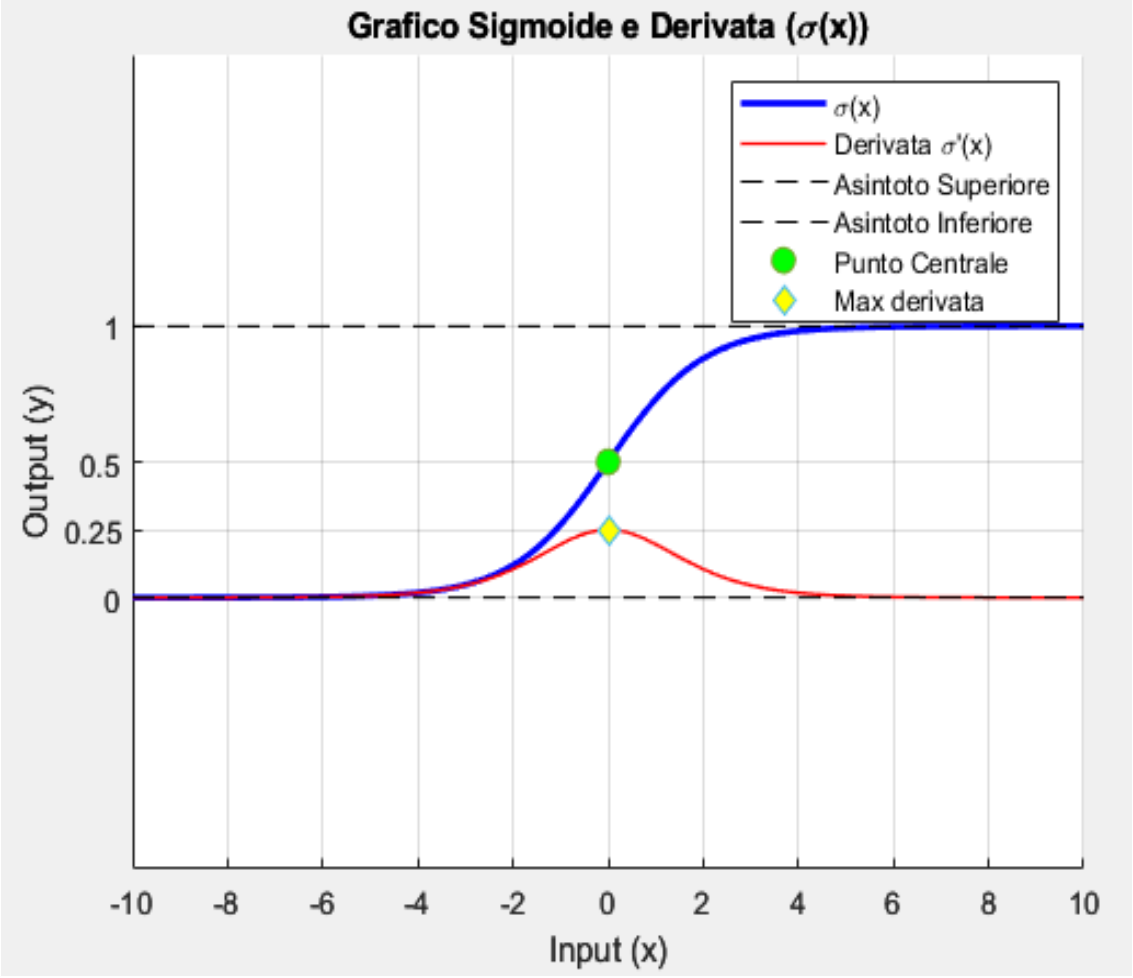


Figura 5 – Rappresentazione grafica sigmoide (MATLAB)

1.6.2 Ambiti applicativi

La funzione sigmoide svolge un ruolo poliedrico all'interno delle architetture neurali, grazie alla sua capacità intrinseca di operare uno "schiacciamento" del segnale di input. Le sue applicazioni principali possono essere riassunte in categorie fondamentali:

- Interpretazione probabilistica: Poiché la funzione mappa l'intero dominio reale nell'intervallo (0, 1), il suo output è matematicamente coerente con i postulati della probabilità. Questa caratteristica la rende lo standard per lo strato di uscita di modelli destinati alla classificazione binaria. In tale contesto, il punto di flesso situato a 0.5 funge da soglia decisionale naturale: valori superiori indicano l'appartenenza alla classe target, mentre valori inferiori ne sanciscono l'esclusione.
- Meccanismi di controllo e Gating: Nelle architetture più sofisticate, come le reti Long Short-Term Memory (LSTM), la sigmoide non è utilizzata per produrre un'inferenza finale, ma agisce come un regolatore del flusso informativo. Sfruttando la saturazione agli asintoti (dove l'output è prossimo a 0 o a 1), la funzione opera come una porta logica differenziabile. Essa determina quale frazione dell'informazione debba essere rimossa dalla memoria interna e quale debba essere trasmessa agli strati successivi, garantendo la gestione efficace delle dipendenze temporali a lungo termine.
- Stabilizzazione dei Transformers: Sebbene meno comune della funzione GeLU negli strati interni, varianti della sigmoide vengono impiegate nei meccanismi di attenzione per normalizzare i pesi o per stabilizzare la dinamica delle attivazioni. In particolare, si utilizzano soluzioni come la funzione SiLU, che integra la sigmoide per ottenere una risposta più fluida, o la Hard-Sigmoid, ottimizzata per l'efficienza computazionale. Tali varianti prevengono divergenze numeriche durante il calcolo dei prodotti scalari, migliorando la robustezza complessiva del modello.

Questa versatilità applicativa sottolinea l'importanza di disporre un'implementazione hardware precisa in quanto un errore nell'approssimazione della curva sigmoide non comporterebbe solo una perdita di precisione numerica, ma potrebbe alterare drasticamente le decisioni di gating o le probabilità di classificazione del modello.

1.6.3 Criticità e limiti computazionali

Nonostante la sua utilità, la sigmoide presenta sfide note sia dal punto di vista algoritmico che hardware:

- Saturazione del gradiente: Quando l'input assume valori estremi, la derivata tende a zero. Durante l'addestramento, questo causa la scomparsa del segnale di errore, impedendo l'aggiornamento dei pesi dei neuroni.
- Costo Computazionale: Dal punto di vista della sintesi digitale su FPGA, la sigmoide rappresenta un elemento critico. La presenza dell'esponenziale naturale richiede l'uso di ingenti risorse hardware se calcolata direttamente.

Capitolo 2 – Implementazione hardware sigmoide

L'obiettivo centrale della fase sperimentale di questo capitolo consiste nel realizzare un'efficiente approssimazione hardware della funzione sigmoide trovando il miglior trade-off tra la precisione del risultato e il risparmio di risorse hardware.

L'implementazione diretta di questa equazione su logica programmabile presenta sfide significative che rendono inefficiente l'approccio puramente analitico:

- **Complessità Computazionale:** Il calcolo dell'esponenziale richiederebbe lo sviluppo di serie di Taylor molto lunghe o algoritmi iterativi, che aumentano la latenza del segnale.
- **Costo dell'Operatore di Divisione:** La divisione è una delle operazioni più costose in termini di risorse hardware (LUT e DSP). Implementare un divisore generico occupa una porzione significativa dell'area del chip.
- **Consumo Energetico:** Nelle tipiche specifiche di progetto, la minimizzazione della potenza dissipata è un vincolo primario. Un calcolo matematico complesso richiede un numero elevato di commutazioni logiche, aumentando drasticamente il consumo dinamico.

Per ovviare a tali problemi, la progettazione hardware moderna si sposta verso tecniche di approssimazione numerica.

Mentre approcci come *CORDIC* offrono una precisione superiore a scapito di una latenza elevata, e le approssimazioni *Piecewise* richiedono l'uso di risorse DSP, l'approccio basato su LUT qui proposto è stato scelto per massimizzare il throughput e garantire una latenza deterministica minima [26]. Questo lo rende ideale per l'integrazione in acceleratori hardware per Transformer, dove la velocità di calcolo delle funzioni di attivazione è un collo di bottiglia critico [27].

Le fasi di sintesi e implementazione descritte in questo capitolo hanno come target il dispositivo *Xilinx Artix-7 XC7A200TSBV484-1*. La scelta di questo specifico chip è stata dettata dalla necessità di operare su una piattaforma che rappresenti lo stato dell'arte della famiglia Artix-7, capace di coniugare un consumo energetico contenuto con elevate capacità di calcolo parallelo.

Grazie alla sua generosa densità logica e alla vasta disponibilità di blocchi di memoria integrati, il dispositivo costituisce un ecosistema ideale per la prototipazione di acceleratori hardware dedicati al DL. L'adozione di questa FPGA consente di validare l'architettura proposta in uno scenario d'uso reale, garantendo la scalabilità necessaria per ospitare, nei successivi sviluppi della ricerca, le strutture ben più articolate dei Transformers senza incorrere in colli di bottiglia fisici o limitazioni nelle risorse di routing.

Un aspetto chiave nella progettazione dell'acceleratore hardware risiede nell'ottimizzazione del parallelismo dei dati e nella prevenzione dei fenomeni di overflow o di clipping distorsivo. Tutte le varianti architetturalmente sviluppate (dalla baseline fino alle ottimizzazioni avanzate basate sulla simmetria) sono state ingegnerizzate per operare nativamente su un bus dati d'ingresso standardizzato a 16 bit in fixed-point.

Tale scelta risponde a un preciso vincolo di co-design con l'architettura del modello Transformer a monte. Essendo i blocchi del sistema strutturati per scalare e limitare i dati a 16 bit, l'interfaccia d'ingresso dell'acceleratore riceve un flusso informativo intrinsecamente confinato nel suo range dinamico.

Questa strutturazione del data-path elimina alla radice la necessità di implementare circuiti di protezione hardware ridondanti o logiche complesse di saturazione e controllo dell'overflow all'interno dello stadio di pre-processing della sigmoide. Di conseguenza, il modulo hardware può dedicare l'intero budget di area logica e di routing alla massimizzazione del throughput e all'efficienza dei blocchi di calcolo e delle LUT, preservando la stabilità del cammino critico alla frequenza operativa di target senza alcuna penalizzazione in termini di latenza o di accuratezza numerica.

2.1 Progettazione architettura baseline V1

L'architettura baseline rappresenta il nucleo primario del sistema e la prima trasposizione fisica dei parametri analitici all'interno della logica programmabile. La progettazione di questo stadio iniziale è stata guidata dalla necessità di stabilire un riferimento funzionale e prestazionale (benchmark) che permettesse di validare l'intera catena di elaborazione del dato prima di procedere con ottimizzazioni strutturali più performanti.

Il design della baseline è stato sviluppato adottando come specifica vincolante i dati contenuti nel file di riferimento *sigmoid.xlsx.* In particolare, sono stati utilizzati gli input campionati e ordinati nell'intervallo $[-4\,;\,4]$ per definire la risoluzione della tabella di ricerca e la precisione del formato numerico. Questa architettura è stata concepita come un "prototipo di validazione" atto a garantire che il passaggio dal dominio software a quello hardware avvenisse senza perdite di coerenza logica, agendo da termine di paragone per valutare l'efficacia delle soluzioni introdotte nel capitolo successivo, assicurando che ogni incremento prestazionale mantenga la fedeltà ai dati originali forniti in fase di specifica.

Il flusso logico si apre con la gestione di un ingresso a 16 bit in formato Q5.11. Poiché la funzione sigmoide presenta la sua massima variazione nell'intorno dello zero, ma il file sigmoid.xlsx definisce un intervallo che parte da -4, la prima sfida progettuale è stata la linearizzazione dell'indice di memoria. Non potendo indirizzare una RAM con numeri negativi, l'architettura implementa una traslazione hardware fissa. In pratica, il circuito somma il valore binario corrispondente a +4 al dato in ingresso, spostando l'intero spettro dei valori in un range puramente positivo, compreso tra 0 e 8.

Questa operazione serve a preparare il dato per la fase di decimazione degli indirizzi. Dei 16 bit risultanti dalla somma, il sistema ne seleziona una finestra di 8 bit. Questa scelta non è arbitraria, ma deriva direttamente dalla profondità della LUT impostata nel file di specifica. Troncando i bit meno significativi, l'hardware effettua una discretizzazione istantanea, individuando lo "slot" di memoria che contiene il valore della sigmoide più vicino all'input ricevuto [28].

2.1.1 Fase di pre-elaborazione dei dati

Per garantire la massima fedeltà tra il modello analitico e l'implementazione hardware, è stato sviluppato in ambiente Python lo script *Baseline_samples_gen.py* (riportato in Appendice A.1) finalizzato alla generazione automatica dei coefficienti della funzione di attivazione. Lo script funge da interfaccia tra i dati calcolati nel file excel di riferimento e il codice sorgente VHDL, automatizzando la fase di quantizzazione e formattazione dei dati.

L'operazione principale dello script consiste nella trasformazione dei 256 campioni della funzione sigmoide, estratti dal file Excel, nel formato fixed-point Q5.11. Nello specifico, l'algoritmo scala ogni valore di un fattore 2^{11} e lo arrotonda all'intero più vicino (quantizzazione). Viene inoltre applicata una logica di saturazione per mantenere il dato entro il range dinamico $[-32768\ ;\ 32767]$ consentito dai 16 bit signed. Per i valori negativi, lo script calcola automaticamente la rappresentazione in complemento a due, assicurando che l'hardware li interpreti correttamente. Lo script genera il file di testo *LUT_Values.txt* contenente stringhe nel formato specifico x"valore_hex". Ciò permette di copiare l'output direttamente all'interno di un package VHDL, eliminando la necessità di trascrizione manuale delle tabelle. I campioni così generati sono organizzati in blocchi da 8 valori per riga, pronti per popolare la costante di tipo array definita nel package *Sigmoid_Values.vhd* (riportato in Appendice B.1).

2.1.2 Blocco di pre-processing baseline

Il modulo di pre-processing agisce come stadio di condizionamento del segnale, interfacciando il data-path principale con la LUT della funzione di attivazione. Poiché la sigmoide è stata mappata nell'intervallo operativo $[-4\ ;\ +4]$ su una memoria da 256 locazioni, il modulo ha il compito critico di risolvere la traduzione tra il dominio dei numeri reali (in fixed-point) e il sistema di indirizzamento fisico della ROM. Il modulo opera in modo sincrono per garantire la stabilità del segnale. La prima fase dell'elaborazione prevede la traslazione del range (offsetting), in cui il dato in ingresso, codificato nel formato Q5.11 a 16 bit, viene sommato a una costante di offset pari a 8192. Questo valore rappresenta il numero reale +4.0 nel sistema fixed-point adottato. Tale operazione è fondamentale poiché sposta l'intervallo operativo dall'originale al dominio strettamente positivo $[0\ ;\ 8]$, rendendo il valore numerico direttamente compatibile con una logica di indirizzamento a indici.

Successivamente, il sistema esegue l'indirizzamento tramite bit-slicing. Per ottimizzare l'occupazione di area hardware ed evitare l'utilizzo di complessi divisori o moltiplicatori, l'indice a 8 bit necessario per la LUT viene estratto selezionando una specifica finestra di bit dal risultato della somma. Estrarre i bit nel range $[13\ ;\ 6]$ equivale a effettuare una divisione implicita per il passo di campionamento (0.03125).

Questa tecnica di manipolazione dei bus permette di ottenere l'indice corretto per puntare ai 256 campioni tabellati con una latenza computazionale nulla e un costo in termini di risorse logiche estremamente ridotto, garantendo al contempo che ogni valore di input sia correttamente mappato sulla corrispondente "entry" della memoria.

Il funzionamento del modulo di pre-processing è riassunto nello schema a blocchi in Fig. 6, lo schematico post-implementazione è mostrato in Fig. 7 e il codice sorgente è riportato integralmente in Appendice B.2.

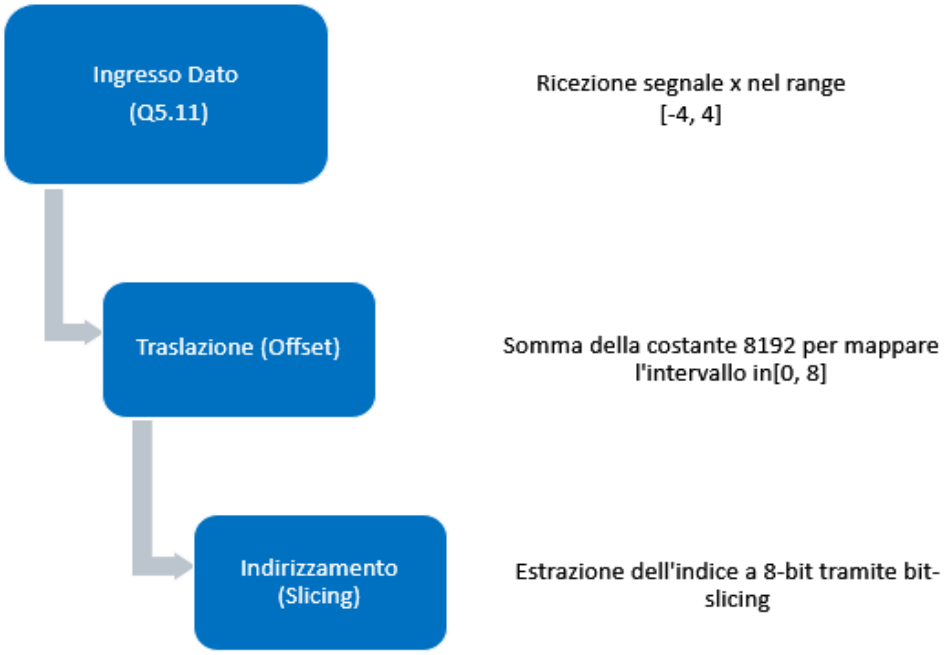


Figura 6 – Logica blocco pre-processing baseline

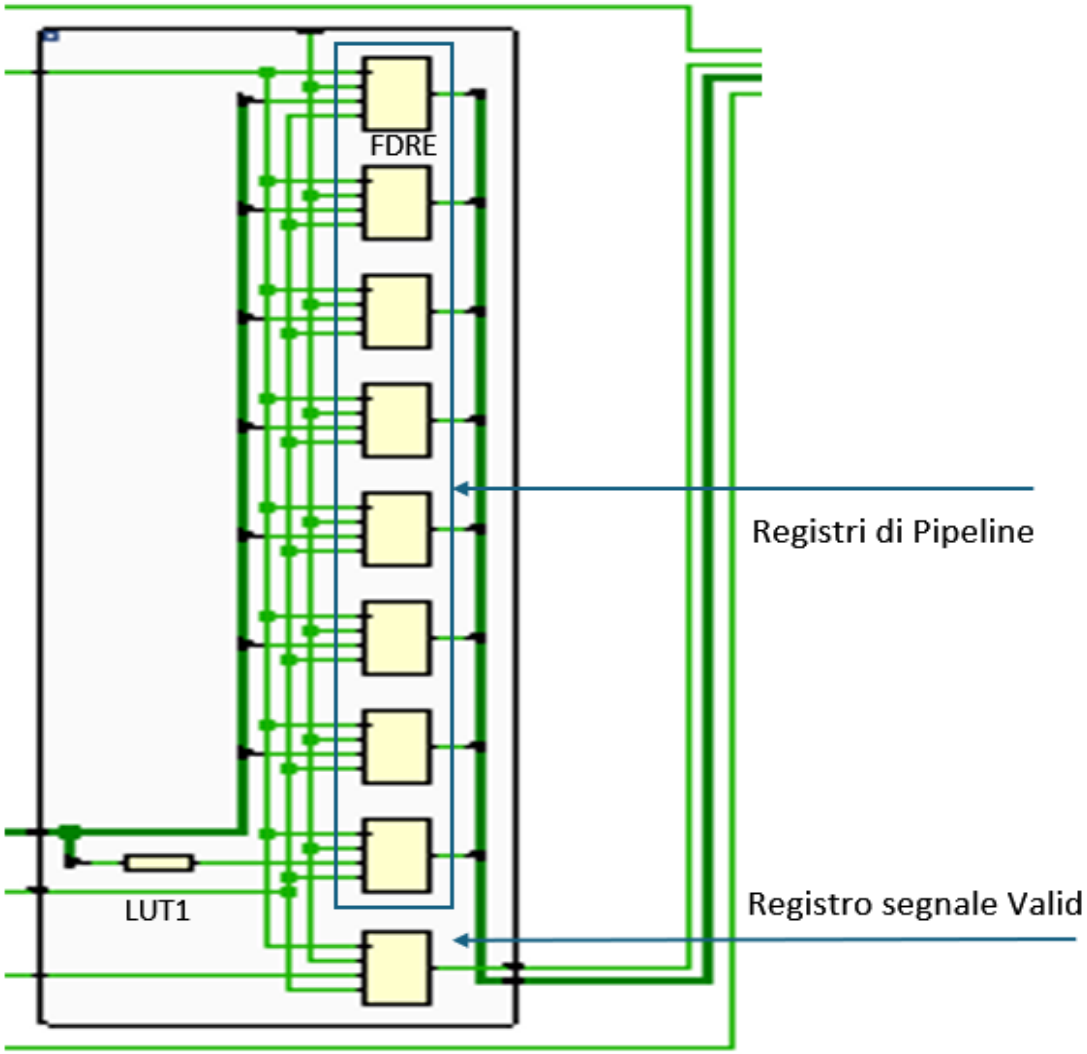


Figura 7 – Schematico post-layout blocco di pre-processing baseline

2.1.3 Blocco di memoria baseline

Per l'implementazione del modulo di memoria dedicato alla LUT, si è scelto di adottare un approccio basato su Distributed RAM. Sebbene l'architettura sia intrinsecamente flessibile e permetta di selezionare diversi paradigmi di memorizzazione tramite direttive di sintesi (come l'attributo rom_style), per questa versione baseline si è deciso di forzare l'uso della logica distribuita.

Questa scelta progettuale è motivata dalla dimensione contenuta della LUT da 256 ingressi, che permette di mappare i dati direttamente all'interno delle Slice dell'FPGA senza impegnare BRAM. Tale strategia consente di ottimizzare l'uso delle risorse hardware, preservando le BRAM (risorsa limitata) per le future evoluzioni del sistema o per la gestione di buffer dati più voluminosi. Inoltre, l'impiego della Distributed RAM garantisce una gestione estremamente efficiente del datapath a livello di routing, mantenendo l'intera logica della funzione di attivazione vicina alle risorse di calcolo e riducendo la latenza complessiva del modulo singolo.

L'architettura del modulo *Sigmoid_LUT.vhd* è stata progettata seguendo una struttura pipeline a 2 stadi, per massimizzare la frequenza operativa e garantire la stabilità dei segnali ad alta velocità. L'inserimento di registri di sincronizzazione permette di frammentare il percorso critico, facilitando il lavoro dei tool di sintesi e implementazione di Vivado. Il flusso di elaborazione si articola nei seguenti stadi:

1. L'indirizzo proveniente dal pre-processing viene campionato nel registro *addr_reg*. Questo isola la logica combinatoria a monte e permette un'ottimale inferenza della memoria.
2. Il valore della sigmoide viene estratto dalla *SIGMOID_TABLE* e memorizzato nel registro di uscita *data_out*, garantendo un segnale stabile e pronto per essere processato dai blocchi successivi del sistema.

Per mantenere la perfetta coerenza temporale tra il flusso dati e i segnali di controllo, il bit di validità (*valid_in*) attraversa un cammino di ritardo parallelo implementato tramite uno Shift Register a 2 stadi. Questa sincronizzazione assicura che il segnale *valid_out* venga asserito esattamente dopo 2 cicli di clock, in perfetta corrispondenza con la disponibilità del dato calcolato in uscita. Tale configurazione abilita il sistema a operare a frequenze di clock elevate, minimizzando il rischio di violazioni temporali durante la fase di Place & Route.

Lo schematico post-implementazione del blocco di memoria è visibile in Fig. 8, il funzionamento è riassunto nello schema a blocchi in Fig. 9 e il codice sorgente è riportato integralmente in Appendice B.3.

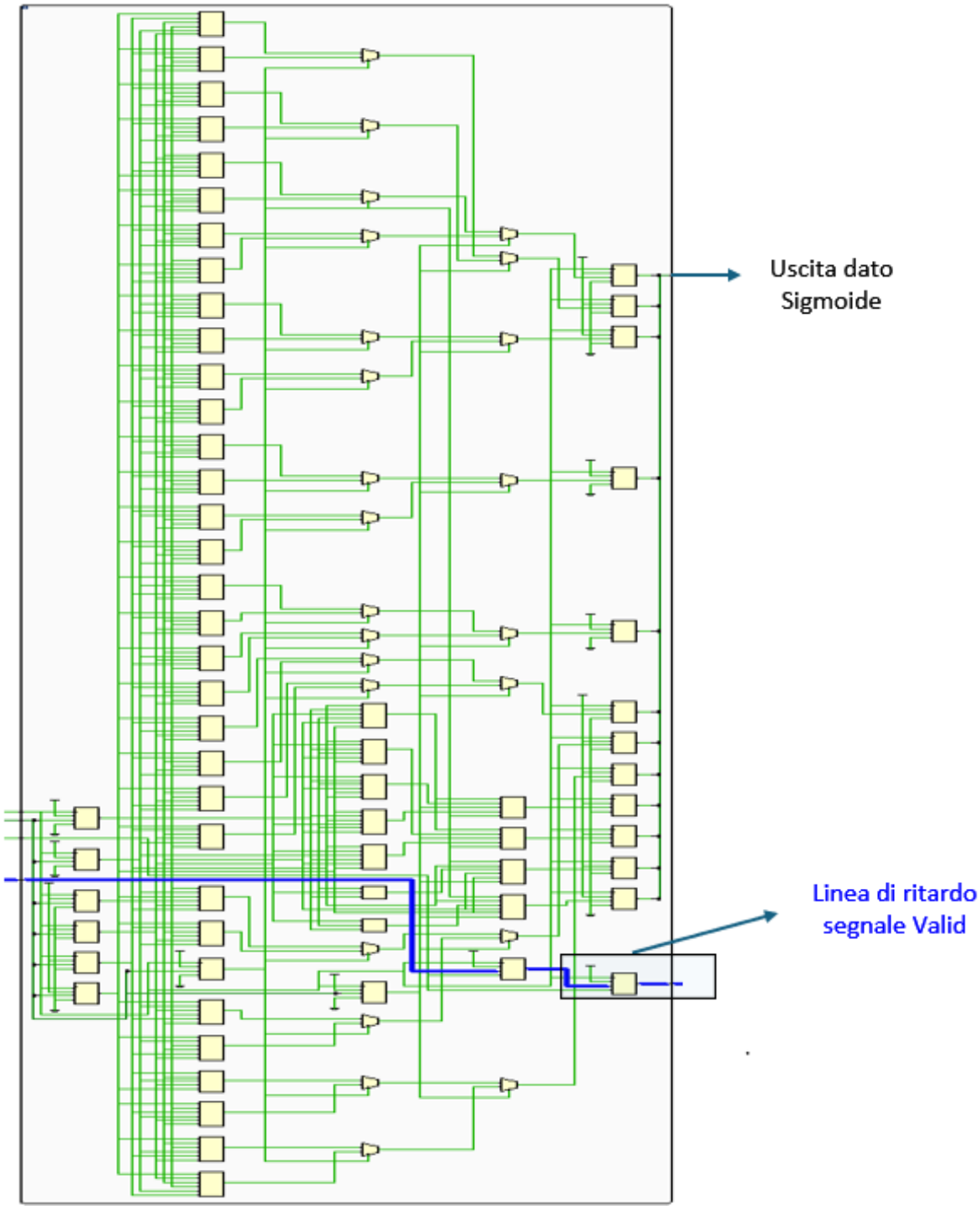


Figura 8 – Schematico post-layout blocco di memoria baseline (Distributed RAM)

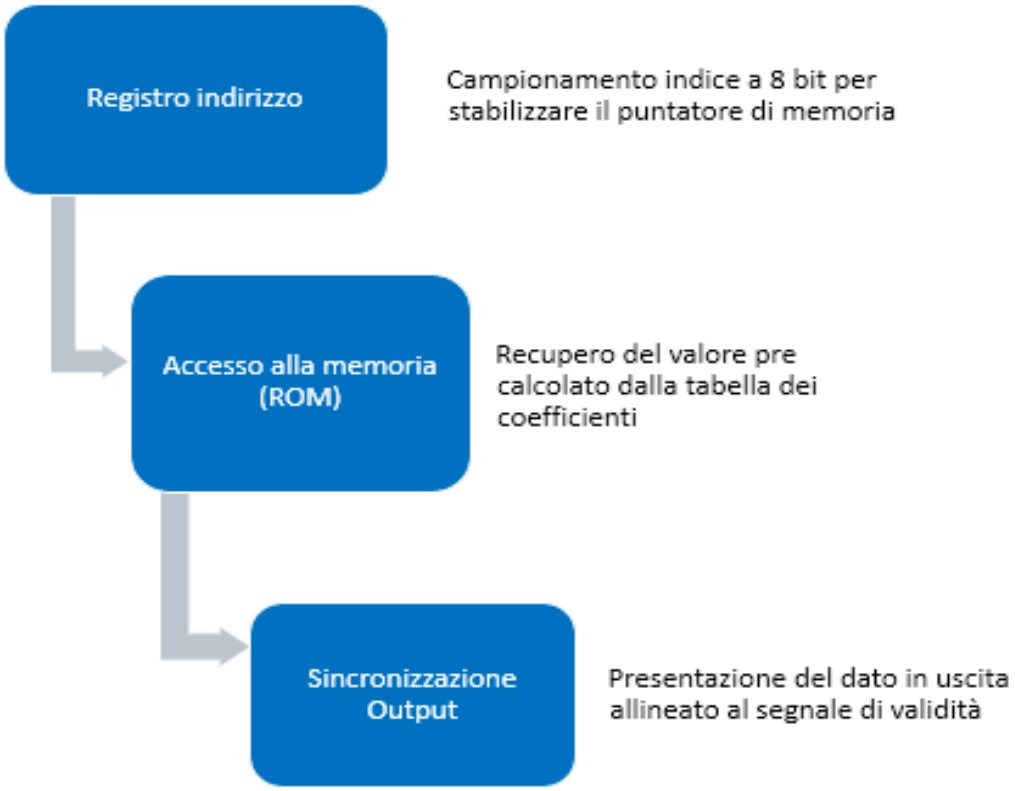


Figura 9 – Logica blocco di memoria baseline

2.1.4 Architettura baseline complessiva

L'architettura complessiva del sistema è consolidata nel modulo *Top_Design.vhd.* Questo blocco implementa una gerarchia di tipo strutturale, interconnettendo il modulo di condizionamento (*Pre_Processing.vhd*) con l'unità di memorizzazione dei coefficienti (*Sigmoid_LUT.vhd*). Il funzionamento del top module (architettura complessiva) è riassunto nello schema a blocchi in Fig. 10, lo schematico post-implementazione in Fig. 11 e il codice sorgente riportato integralmente in Appendice B.4.

Caratteristica fondamentale di questo design è l'introduzione di uno stadio di input buffering (registri *reg_input_data* e *reg_valid_in*). Questo accorgimento tecnico permette di isolare i pin fisici dell'FPGA dalla logica interna, migliorando i tempi di setup e proteggendo il sistema da eventuali glitch sui segnali d'ingresso.

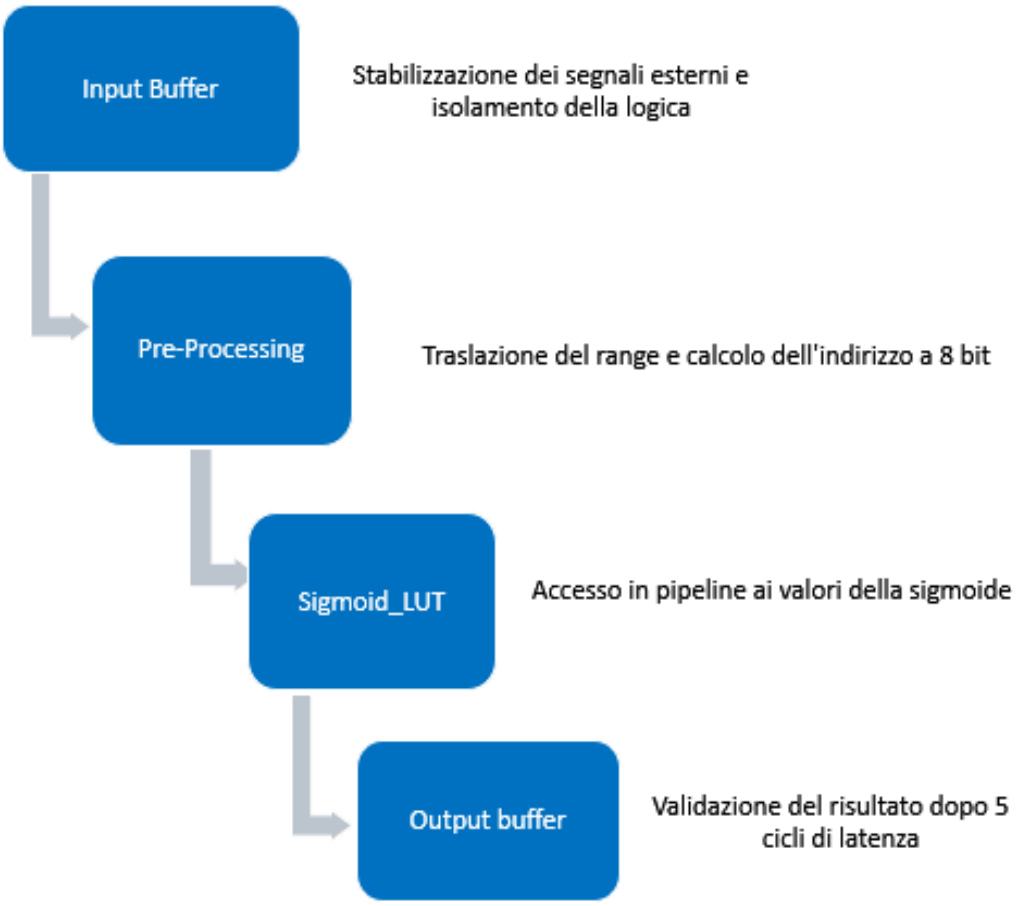


Figura 10 – Logica architettura baseline complessiva

L'intero sistema opera come una pipeline deterministica. Il flusso di dati attraversa i vari stadi introducendo una latenza totale calcolata di 5 cicli di clock

- Un ciclo per il buffering d'ingresso nel *Top_Design*;
- Un ciclo per la traslazione nel pre-processing.
- Due cicli per la pipeline interna della ROM (indirizzamento e output);
- Un ciclo per il buffering d'uscita nel *Top_Design*.

Tale profondità di pipeline è stata scelta per massimizzare il throughput del sistema [29], [30], permettendo di processare un nuovo dato ad ogni colpo di clock nonostante il ritardo complessivo di attraversamento.

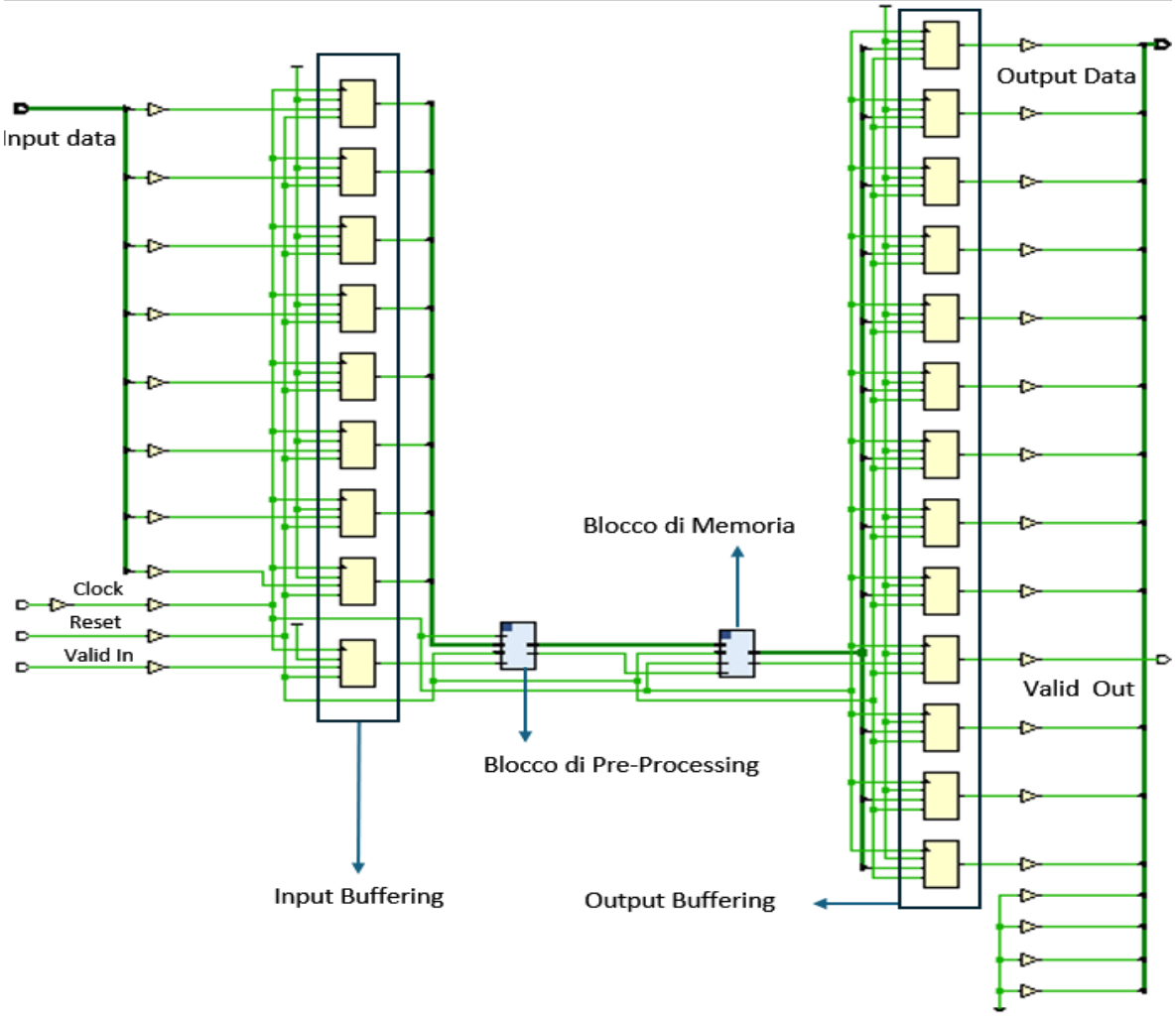


Figura 11 – Schematico post-layout architettura baseline complessiva

2.2 Fase di test e validazione baseline

Per convalidare il corretto funzionamento dell'architettura hardware in fase di simulazione, è stato realizzato uno script Python (*Baseline_inputs.py*), riportato integralmente in Appendice A.2, finalizzato alla generazione del file di test *stimoli_input_baseline.mem*. Lo script estrae dal dataset di riferimento una rampa di 256 valori ordinati nell'intervallo $[-4.0\,;\,+4.0]$, permettendo di verificare la risposta della funzione sigmoide su tutti i punti critici.

I dati così estratti vengono convertiti nel formato fixed-point Q5.11 e formattati come stringhe esadecimali pure a 16 bit, seguendo lo standard richiesto dai simulatori hardware per l'inizializzazione di array e memorie tramite funzioni di TextIO.

L'adozione di questa procedura automatizzata garantisce una perfetta coerenza tra l'input fornito al simulatore e quello utilizzato nel modello analitico in software.

In seguito è stato predisposto un ambiente di simulazione basato su un testbench strutturato per validare il sistema sia in termini funzionali che in condizioni di stress temporale. Il flusso dei dati è gestito in modo deterministico da tre processi concorrenti, ottimizzati per garantire l'integrità dei segnali anche nelle simulazioni post-layout:

1. Generazione del Clock e Temporizzazione: Il processo *clk_process* genera il segnale di sincronismo globale con un periodo impostato a 4 ns, corrispondente a una frequenza operativa di target pari a 250 MHz. Questa scelta consente di verificare la robustezza del sistema a frequenze elevate, stressando i cammini critici emersi dal report di timing.
2. Iniezione degli Stimoli e Sincronizzazione: Il processo *stim_read* gestisce il caricamento sequenziale dei dati dal file *stimoli_input_baseline.mem*. La fase di sblocco dal reset e l'invio dei vettori d'ingresso sono stati sincronizzati sul fronte di discesa (*falling_edge*) del clock. Questa tecnica consente di eliminare le incertezze di setup, fornendo alla logica interna della FPGA un margine temporale pari a mezzo ciclo di clock (2 ns) affinché i segnali sul bus *input_data* e il flag *valid_in* si stabilizzino prima del sopraggiungere del fronte di salita attivo. Al termine del file, il processo provvede tempestivamente ad abbassare il segnale di validità e attende lo svuotamento completo dei 5 stadi della pipeline prima di arrestare la simulazione.
3. Monitoraggio e Validazione dei Risultati: Il processo *results_writer* è deputato alla cattura delle uscite e alla generazione del file di log *risultati_HW_256.hex*. Previo rilascio del segnale di reset, il processo monitora il sistema a ogni fronte di salita (*rising_edge*) del clock, campionando i dati in uscita in modo strettamente sincrono con l'architettura. La cattura e la successiva scrittura su file avvengono esclusivamente quando il modulo asserisce il flag *valid_out*. Questo approccio assicura il perfetto tracciamento dei dati stabili all'uscita dei registri di output buffer, garantendo l'immunità da glitch o stati transitori della Distributed RAM, indipendentemente dai ritardi fisici di propagazione del silicio.

L'ambiente così configurato permette di testare agevolmente e con la medesima metodologia sia le varianti basate su Distributed RAM che quelle su BRAM, offrendo un riscontro immediato sulla stabilità dei dati e sulla latenza deterministica della pipeline.

Questo sistema di test permette di generare un output hardware speculare ai dati di input. Il file dei risultati così ottenuto costituisce la base per la successiva fase di post-elaborazione in ambiente MATLAB, dove verrà calcolato l'errore medio assoluto (MAE) e verificata la rispondenza alle specifiche di progetto.

Il funzionamento del testbench è riassunto nello schema a blocchi in Fig. 12, mentre il codice sorgente è riportato integralmente in Appendice B.5.

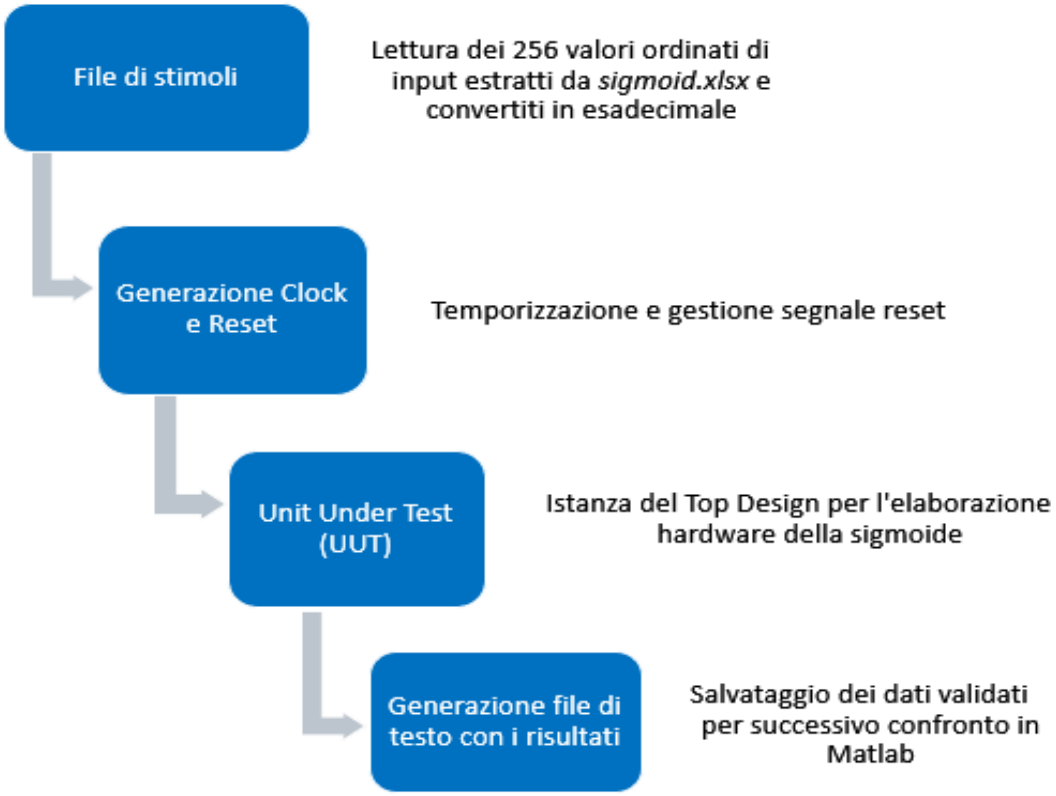


Figura 12 – Logica testbench versione baseline

2.2.1 Simulazione temporale post-implementazione

La validazione dell'architettura procede con la fase di simulazione, un passaggio cruciale per verificare la fedeltà del design rispetto alle specifiche funzionali in un contesto operativo reale.

Questa fase integra le informazioni estratte dal piazzamento e cablaggio (Place & Route) definitivo sulla FPGA.

Questo processo permette di modellare con precisione i ritardi di propagazione dei segnali lungo i cammini critici e le interconnessioni, assicurando che la risposta del sistema sia priva di criticità temporali non rilevabili ai livelli di astrazione superiori.

Tale fase serve a verificare che l'approssimazione della funzione sigmoide mantenga l'integrità numerica necessaria per non compromettere la convergenza dei pesi all'interno delle architetture Vision Transformer.

L’approccio Distributed RAM, sfruttando la prossimità fisica delle LUTRAM al datapath, garantisce una transizione dei bit più sincrona, eliminando i glitch e permettendo una campionatura del dato più affidabile, pertanto tale approccio è stato selezionato come miglior trade-off per la versione baseline.

La simulazione funzionale post-implementazione, mostrata in Fig. 13, mette in luce il corretto funzionamento dell'acceleratore e il rispetto della latenza di pipeline impostata in fase di progetto.

Osservando il diagramma è possibile tracciare l'intero ciclo di vita di un dato all'interno del modulo:

L'elaborazione ha inizio nel momento in cui il segnale *valid_in* commuta allo stato logico alto. Contestualmente, viene campionato il primo valore d'ingresso valido, pari a -4. Come evidenziato dalla numerazione presente nella forma d’onda (waveform), il dato attraversa i 4 stadi di pipeline previsti per il calcolo dell'indirizzo e l'accesso alla memoria.

Il primo risultato valido in uscita appare dopo esattamente 5 colpi di clock, istante in cui il segnale *valid_out* si alza correttamente. In questa finestra temporale, il valore d'uscita visualizzato (0) rappresenta il valore della funzione sigmoide calcolato per l'ingresso corrispondente. Il perfetto allineamento tra l'attivazione di *valid_out* e la disponibilità del dato finale conferma la corretta sincronizzazione della logica di controllo con il data-path.

La simulazione post-layout dell'architettura baseline conferma la corretta implementazione della logica di indirizzamento e della memoria. Un aspetto fondamentale emerso dalla validazione è l'andamento non uniforme della quantizzazione della sigmoide.

Come previsto dalle specifiche originali, la funzione presenta zone di saturazione agli estremi, infatti per ampi intervalli di valori di input (vicini a -4 e +4) l'output rimane costante in quanto a diversi incrementi del segnale *input_data* corrisponde il medesimo valore di *output_data*. Nella zona di massima pendenza al centro, In prossimità dello zero, i valori cambiano repentinamente, rispecchiando la natura asintotica della sigmoide.

Questo comportamento "a gradini" conferma che il sistema di bit-slicing e l'integrità della LUT funzionano come previsto. Il fatto che più input producano lo stesso output non è la conferma che l'hardware sta operando nel pieno rispetto del modello di riferimento fornito.

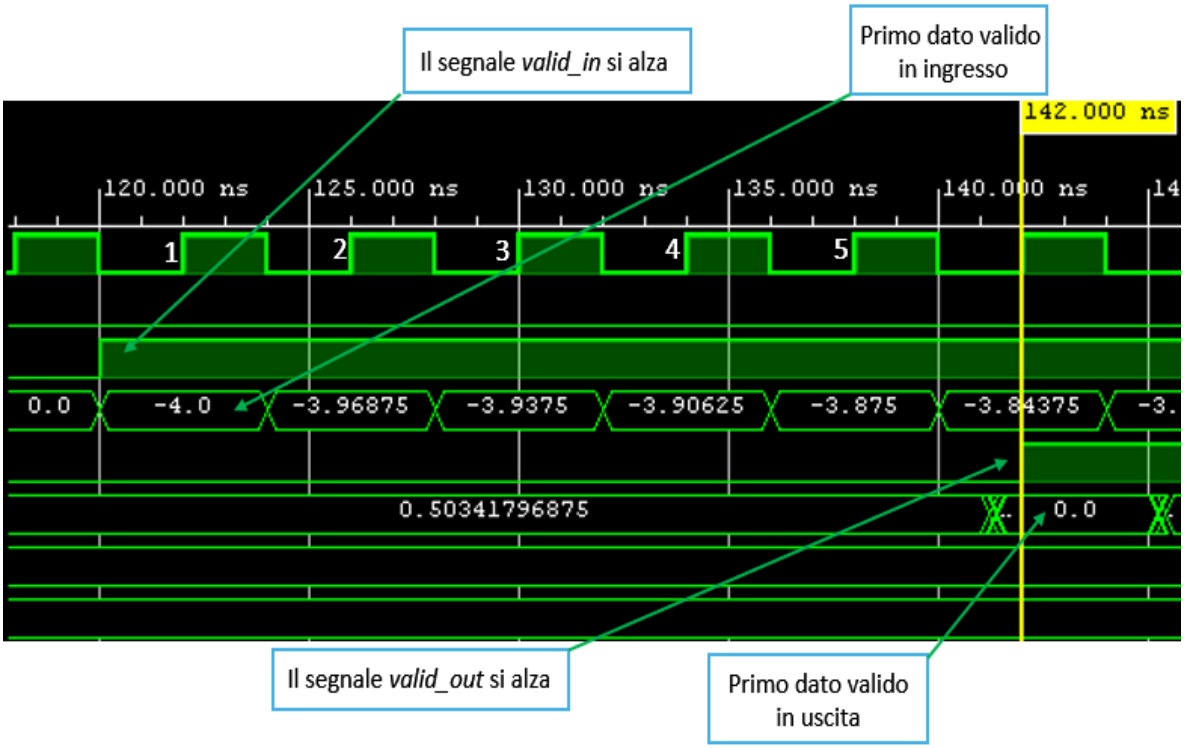


Figura 13 – Simulazione timing post-layout versione baseline

Osservando nel dettaglio lo zoom proposto in Fig. 14 si nota come al fronte di salita successivo il dato venga stabilmente acquisito sul bus di uscita, dal momento che il segnale di validità è passato allo stato logico alto, pochi picosecondi prima.

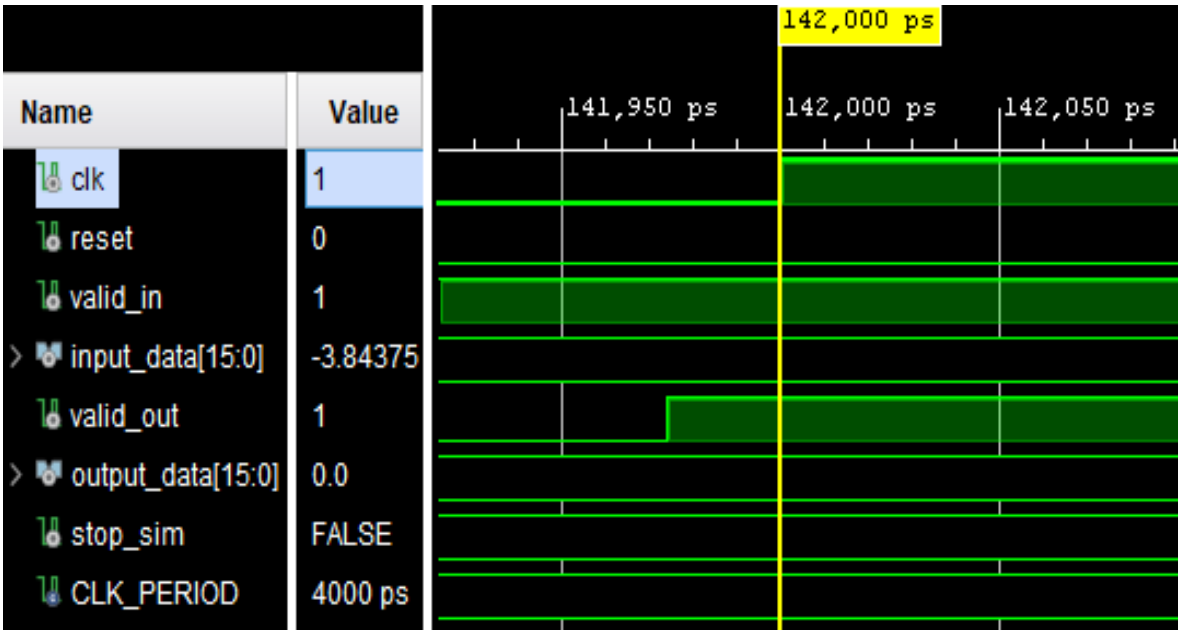


Figura 14 – Zoom dettagliato simulazione timing post-layout V1

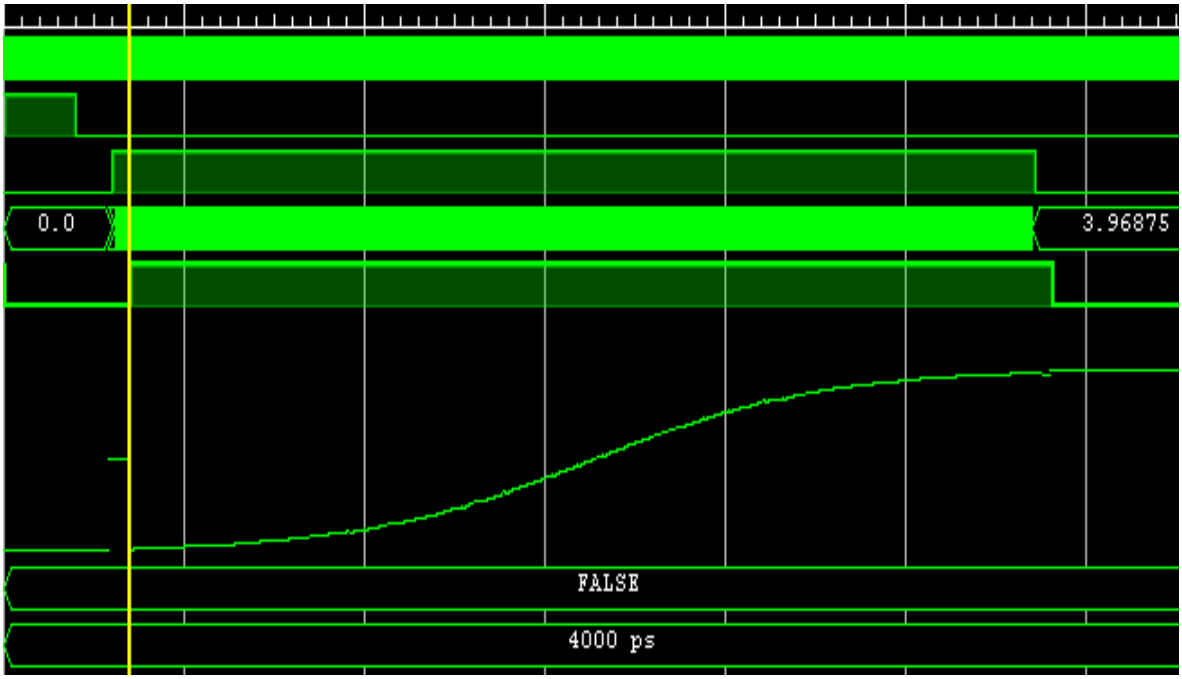


Figura 15 – Forma d'onda analogica in simulazione timing post-layout V1

Una volta riempita la pipeline, il sistema è in grado di fornire un nuovo risultato a ogni fronte di salita del clock, garantendo un throughput di un'operazione per ciclo.

Questa regolarità nel flusso dei dati, unita alla latenza deterministica, dimostra la solidità dell'implementazione hardware e la sua idoneità a operare all'interno di architetture neurali complesse, dove la sincronizzazione tra i diversi layer è un requisito critico.

Un'ulteriore conferma della validità dell'architettura baseline emerge dalla visualizzazione della forma d'onda in modalità analogica (Fig.15). Convertendo l'andamento temporale del bus *output_data* in un segnale analogico equivalente all'interno del simulatore, si ottiene la caratteristica curva a "S" tipica della funzione sigmoide.

Questa rappresentazione grafica permette di apprezzare visivamente due aspetti:

1. Linearità del Mapping: La regolarità della curva dimostra che la logica di pre-processing (offset e bit-slicing) scansiona gli indirizzi della LUT in modo perfettamente lineare. Non sono presenti discontinuità o "salti" nei dati, il che prova che la sequenza degli stimoli di input viene processata correttamente nel tempo.
2. Effetto della Discretizzazione: Sebbene a prima vista la curva appaia fluida, un'osservazione più ravvicinata della visualizzazione analogica rivela la natura discreta del segnale. La "scalinata" visibile lungo il profilo della curva è la diretta conseguenza dell'approssimazione a 256 livelli.

2.2.2 Confronto hardware/software

Per la definizione del modello di riferimento software è stato adoperato uno script Python, *golden_model.py,* riportato in Appendice A.3, che calcola i valori teorici della funzione sigmoide con precisione decimale a 64 bit. Questo riferimento ideale permette di isolare l'errore di quantizzazione dell'hardware confrontandolo con una funzione matematica priva di approssimazioni iniziali.

Il modello calcola esattamente 256 campioni distribuiti in un intervallo che va da -4.0 a +4.0, utilizzando un incremento costante tra i punti di 0.03125. I risultati vengono salvati in un file di testo, *sigmoid_outputs_sw.txt,* mantenendo 15 cifre decimali di precisione.

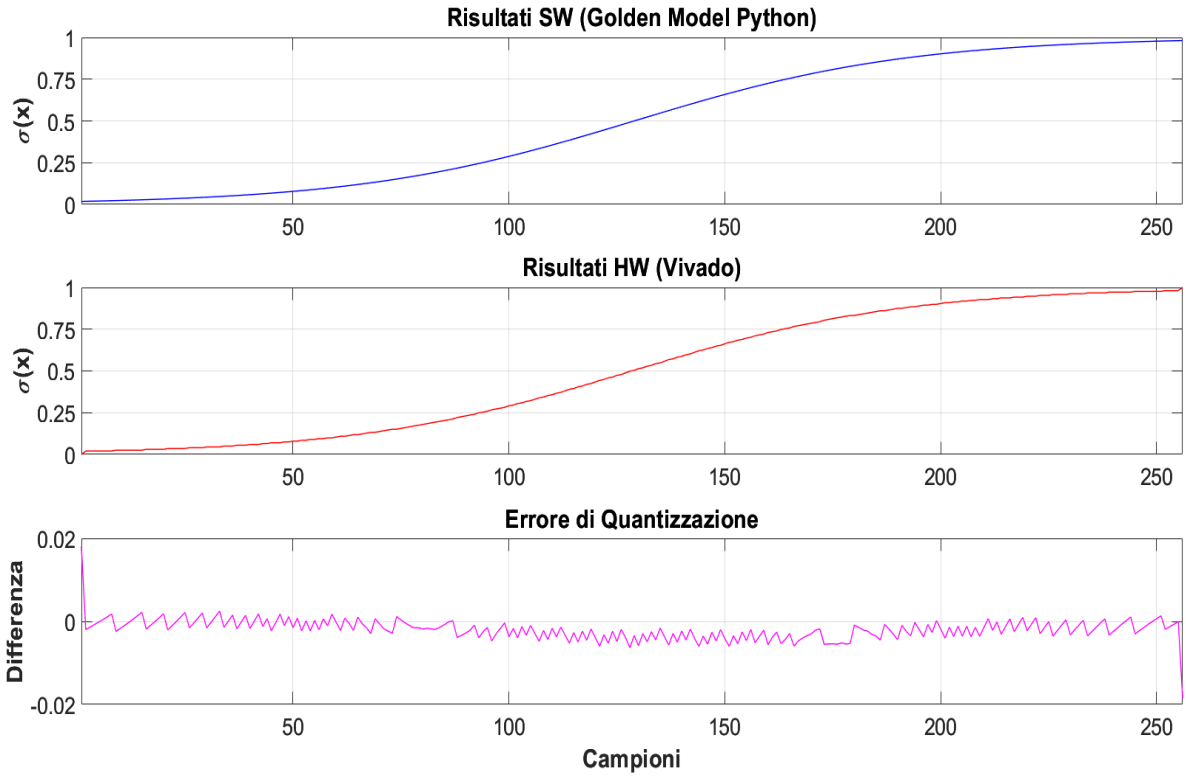
Questo set di dati costituisce la base di confronto per l'analisi statistica in MATLAB, dove i valori ideali vengono comparati con quelli estratti dalla simulazione hardware per determinare l'accuratezza finale del sistema. Il confronto tra l'approssimazione hardware in fixed-point e il modello software permette di misurare l'impatto della quantizzazione sulla precisione della funzione sigmoide.

L'elaborazione dei dati in ambiente MATLAB, mediante lo script *Confronto_HW_SW.m,* riportato in Appendice C.2*,* ha permesso di quantificare l'accuratezza del modulo baseline rispetto al modello ideale.

Analizzando il grafico dell'Errore di Quantizzazione (Fig. 16), si nota un andamento oscillatorio "a dente di sega". Questo pattern è la diretta conseguenza della natura discreta della LUT: l'errore cresce man mano che l'input si allontana dal punto esatto campionato nella tabella, per poi crollare quando l'input raggiunge lo step successivo.

Un dettaglio cruciale riguarda la gestione degli estremi dell'intervallo operativo, derivata direttamente dalle specifiche del file Excel di riferimento. Il valore di input -4.0 è stato mappato esattamente al valore di uscita 0, mentre il valore di input massimo (3.96875) è stato mappato al valore di uscita 1. Questa scelta di saturazione completa agli estremi spiega i picchi di errore visibili ai margini del grafico dei residui. In particolare, l'errore massimo (0.018) si registra proprio in corrispondenza degli estremi e nelle zone di massima pendenza della sigmoide (attorno allo zero), dove la variazione della funzione tra due campioni adiacenti è più marcata.

Nonostante l'errore massimo possa sembrare significativo, il MAE di $2.39 \cdot 10^{-3}$ indica che, mediamente, l'approssimazione fornita dalla versione baseline è sufficientemente accurata per applicazioni di inferenza che non richiedano una precisione estrema.




```
MAE: 0.0023995273
Errore Massimo: 0.0185465645
```

Figura 16 – Confronto MATLAB risultati V1

2.3 Report post-layout architettura baseline

In questo paragrafo viene esaminato il profilo prestazionale dell'architettura baseline, attraverso un'analisi comparativa dei risultati ottenuti nella fase di implementazione, con l'obiettivo di caratterizzare quantitativamente l'efficacia del design in termini di costi hardware e fedeltà algoritmica.

Si esaminano i report di occupazione delle risorse e i parametri di timing, verificando il rispetto dei vincoli temporali necessari.

Inoltre, attraverso le simulazioni post-implementazione, si valuta l'efficienza energetica del modulo, parametro critico per l'integrazione dei Vision Transformers in dispositivi edge ad autonomia limitata.

2.3.1 Utilizzo risorse

Il report di occupazione delle risorse, mostrato in Fig. 17, ottenuto dopo la fase di *Place & Route* fornisce una misura precisa dell'efficienza dell'architettura baseline. I dati confermano un design estremamente ottimizzato, capace di operare con un impatto irrilevante sulle risorse del dispositivo.

Name	Slice LUTs (134600)	Slice Registers (269200)	F7 Muxes (67300)	F8 Muxes (33650)	Slice (33650)	LUT as Logic (134600)	Bonded IOB (285)	BUFGCTRL (32)
N TOP_Design	43	53	16	8	21	43	28	1
	0.03%	0.02%	0.02%	0.02%	0.06%	0.03%	9.82%	3.13%

Figura 17 – Report di utilizzo risorse architettura baseline

L'implementazione ha richiesto l'impiego di 16 Mux F7 e 8 Mux F8, che testimonia un'efficace mappatura della memoria distribuita all'interno delle Slice. Grazie a questa configurazione, è possibile realizzare la funzione di attivazione senza ricorrere a blocchi BRAM dedicati. Il risultato complessivo è un'occupazione di area inferiore allo 0,1% della FPGA, un margine che garantisce ampia flessibilità per l'integrazione di moduli paralleli o per l'implementazione di logiche computazionali più complesse.

2.3.2 Timing

L'analisi temporale post-implementazione, mostrata in Fig. 18, è stata condotta impostando un vincolo di clock con un periodo di 4 ns, corrispondente a una frequenza operativa di 250 MHz.

Setup

Worst Negative Slack (WNS):	1,394 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	44

Hold

Worst Hold Slack (WHS):	0,157 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	44

All user specified timing constraints are met.

Figura 18 – Report di timing architettura baseline

I risultati ottenuti confermano la solidità del design:

- Worst Negative Slack (WNS): Il valore di 1,394 ns indica che il cammino critico più lento completa la sua propagazione con un margine di sicurezza significativo rispetto al periodo di clock. Questo risultato è frutto della pipeline a più stadi, che segmenta efficacemente il percorso logico tra la fase di pre-processing e l'accesso alla memoria distribuita.
- Worst Hold Slack (WHS): Il valore di 0,157 ns attesta il pieno rispetto dei vincoli di mantenimento, garantendo la stabilità dei segnali tra i registri adiacenti ed escludendo il rischio di corse critiche (*race conditions*).

Il rispetto di questi vincoli permette all'architettura baseline di operare a 250 MHz garantendo un throughput di un dato per ciclo di clock con una latenza deterministica. Sebbene l'ampio margine di slack positivo suggerisca margini per un incremento della frequenza, i test condotti indicano che superare la soglia dei 300 MHz porterebbe a fenomeni di corruzione dei dati, rendendo instabile il campionamento nei registri di uscita.

Tuttavia, è opportuno notare che tale stabilità è favorita dalla semplicità della struttura a LUT ridotta. Nelle versioni successive, l'introduzione di logiche di interpolazione più onerose o l'espansione della memoria andranno a ridurre questi margini, rendendo la gestione della pipeline un fattore ancora più critico per mantenere le prestazioni richieste dai modelli di Transformer.

2.3.3 Potenza dissipata

Per caratterizzare in modo accurato il profilo energetico dell'acceleratore hardware alla frequenza operativa di 250 MHz, si è scelto di condurre un'analisi comparativa in due fasi: una stima probabilistica iniziale (vectorless analysis) e una successiva caratterizzazione deterministica accurata tramite l'annotazione delle attività di commutazione reali con file SAIF (Switching Activity Interchange Format). Inizialmente, il tool Vivado Power Analysis è stato eseguito in modalità vectorless. In questa configurazione, lo strumento non dispone di informazioni reali sugli stimoli d'ingresso e applica modelli statistici di default (tra cui un toggle rate standard del 12.5% sulle porte di I/O).

I risultati estratti dalla netlist post-implementazione in questa prima fase indicano una potenza totale dissipata di 0.149 W, con una componente dinamica stimata in 0.018 W (pari al 12% del consumo complessivo). Dall'analisi dettagliata, mostrata in Fig. 19, si nota come la potenza dinamica sia dominata in modo quasi esclusivo dai buffer di I/O (0.015 W, pari all'81% della frazione dinamica). Questo comportamento evidenzia l'approccio fortemente conservativo (pessimistico) dell'algoritmo statistico del tool, il quale tende a sovrastimare l'attività dei pin di comunicazione quando non guidato da una simulazione reale. In questa fase, il livello di confidenza associato al report si attesta su un valore basso. Per rimuovere l'incertezza statistica e ottenere una stima scientificamente accurata, si è proceduto all'estrazione delle reali attività di commutazione. L'attività di toggle delle net strutturali, relativa al testbench che prevede come ingresso il vettore di stimoli composto da una rampa di 256 valori sequenziali, è stata catturata e salvata all'interno del file *baseline_rampa.saif*, successivamente riannotato nel tool di Power Analysis.

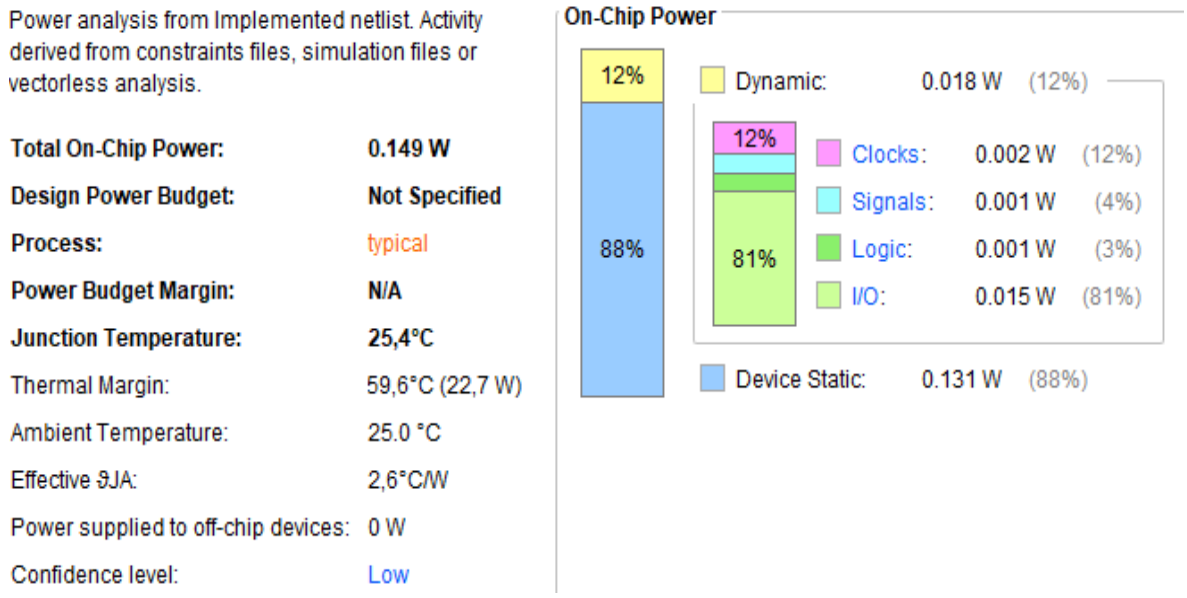


Figura 19 – Report di potenza (vectorless) architettura baseline

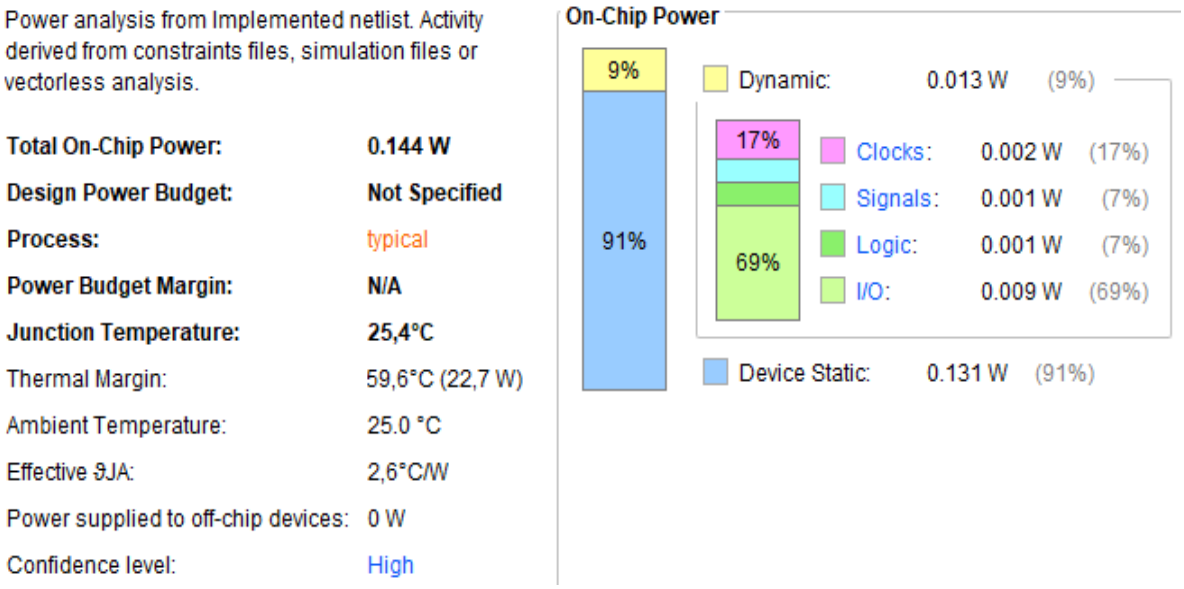


Figura 20 – Report di potenza (post-SAIF) architettura baseline

L'introduzione del file SAIF ha mostrato il reale comportamento hardware dell'acceleratore, portando il livello di confidenza del report al grado High (Fig. 20). Con i dati reali di simulazione, la potenza totale calcolata scende a 0.144 W e la potenza dinamica si attesta sul valore reale di 0.013 W (9% del totale).

Di seguito si riporta la scomposizione dei contributi post-annotazione:

- Potenza Statica (0.131 W): Rappresenta il 91% del consumo totale nel report definitivo. Questo contributo è imputabile principalmente alle correnti di leakage dei transistor e alle risorse statiche del chip. Essendo un valore legato alla tecnologia costruttiva e alle dimensioni fisiche del dispositivo, rimane pressoché costante. Esso risulta preponderante a causa del basso tasso di occupazione complessivo delle risorse della FPGA da parte del singolo modulo acceleratore.
- Potenza Dinamica Reale (0.013 W): Il costo energetico effettivo dell'elaborazione scende a soli 13 mW. Il file SAIF ha dimostrato che la rampa di stimoli reali fa commutare le porte di I/O in modo nettamente più efficiente rispetto alla stima probabilistica, riducendo il consumo dei buffer di ingresso/uscita a 0.009 W. I consumi interni legati alle reti di clock (0.002 W), ai segnali di routing (0.001 W) e alla logica interna (0.001 W) rimangono estremamente contenuti.

Un consumo dinamico reale così ridotto, a una frequenza di 250 MHz, conferma l'efficacia delle scelte orientate al low-power. L'impiego del bit-slicing e della Distributed RAM ha permesso di evitare l'attivazione di risorse più energivore, come i blocchi BRAM, minimizzando le transizioni logiche necessarie per il calcolo della funzione sigmoide.

In un contesto di calcolo parallelo massivo, tipico delle architetture Transformer, la capacità di contenere la componente dinamica reale della singola unità a soli 13 mW è un requisito fondamentale. Tale efficienza garantisce la sostenibilità termica dell'intero acceleratore hardware, permettendo l'integrazione di moduli simultanei senza compromettere l'integrità del dispositivo o richiedere sistemi di dissipazione complessi.

Capitolo 3 – Ottimizzazione dell'architettura

L'architettura baseline descritta nel capitolo precedente ha svolto con successo il compito di validare la catena di elaborazione hardware, dimostrando la correttezza del flusso dati e della logica di indirizzamento della memoria. Sulla base di questi risultati, il presente capitolo si pone l'obiettivo di elevare le prestazioni del sistema, introducendo soluzioni avanzate che ne aumentino la precisione analitica e la versatilità operativa in scenari di calcolo intensivo.

Al fine di superare i limiti di accuratezza e di occupazione di area della soluzione baseline, il lavoro di tesi propone lo sviluppo di due ulteriori architetture hardware ottimizzate (denominate rispettivamente "**V2**" e "**V3**"), ciascuna mirata a spingere al limite un differente trade-off prestazionale:

V2: questa variante punta a massimizzare l'accuratezza analitica della funzione sfruttando le risorse di memoria dedicate della FPGA, ed è basata su:

- Aumento della risoluzione della LUT: Il numero di campioni della sigmoide passa da 256 a 1024 locazioni di memoria. I valori, calcolati mediante uno script Python dedicato e tabellati all'interno del modulo, riducono drasticamente l'errore di quantizzazione residuo del formato fixed-point.
- Gestione dinamica degli input (Clipping): Viene integrato un modulo di pre-elaborazione hardware avanzato per interfacciare l'acceleratore con i dati reali provenienti da framework esterni (come PyTorch). Questo blocco implementa una logica di saturazione (clipping) che intercetta i valori eccedenti il dominio di campionamento standard, forzando l'indirizzo della memoria all'asintoto corretto ed evitando fenomeni distruttivi di rollover del dato.
- Migrazione su risorse BRAM: Per non gravare sulla logica combinatoria delle Slice LUT (come avveniva nella Distributed RAM), la LUT viene mappata su un blocco di memoria RAM dedicato (BRAM). Sfruttando i registri d'uscita nativi integrati nel silicio della BRAM, si ottiene un de-coupling del cammino critico, ottimizzando il timing globale del sistema.

V3: questa variante rappresenta l'ottimizzazione più avanzata del design, focalizzata sull'efficienza dell'area e sulla precisione del dato d'uscita ed è basata su:

- Ottimizzazione della densità tramite simmetria matematica: sfruttando la proprietà di simmetria della sigmoide, ossia $\sigma(-x) = 1 - \sigma(x)$, viene implementata una logica hardware di specchiatura dell'indirizzo. Questa tecnica consente di raddoppiare la densità dei campioni memorizzati, mantenendo inalterate le risorse di memoria richieste sulla FPGA.
- Risoluzione d'uscita in formato Q1.15: poiché l'output della funzione sigmoide è strettamente confinato nell'intervallo $[0\,;\,1]$, il formato fixed-point d'uscita viene ottimizzato eliminando i bit ridondanti della parte intera. L'adozione del formato full-fractional Q1.15 consente di allocare tutti i 15 bit disponibili alla parte frazionaria, garantendo una precisione maggiore sui pesi del Transformer senza incrementare il parallelismo del bus dati a 16 bit.

3.1 Progettazione architettura intermedia V2

L'architettura V2 introduce l'espansione della risoluzione della LUT, sfruttando la capacità della BRAM. Questo incremento della profondità permette di mappare il dominio della funzione con una fittezza di campionamento quattro volte superiore, abbattendo drasticamente il MAE e restituendo una curva estremamente fedele al modello software originario. L'introduzione della Logica di clipping deterministica serve per interfacciare in totale sicurezza l'acceleratore con flussi di dati reali. Nel modulo di pre-processing di questa architettura viene pertanto integrato un circuito combinatorio di saturazione. Prima di eseguire l'operazione di bit-slicing per ricavare l'indirizzo, l'hardware confronta il dato con una soglia critica; se l'input eccede i limiti del dominio memorizzato, la logica intercetta l'overflow e ancora l'indirizzo d'uscita al valore di saturazione massimo (locazione 1023), blindando la stabilità del sistema contro dati aberranti.

La tabella dei valori della sigmoide viene migrata dalla logica distribuita (LUT dell'FPGA usate come ROM) a una risorsa di memoria *hard* nativa sul chip (BRAM da 18 Kb). Tale scelta permette di liberare una quantità massiccia di celle logiche riconfigurabili sul silicio, delegando la memorizzazione a un blocco dedicato ad alta velocità.

3.1.1 Affinamento della quantizzazione

Il package VHDL *Sigmoid_Values_1024.vhd* (Appendice B.6) è stato aggiornato con 1024 coefficienti esadecimali, generati tramite script Python *(1024_values_lut.py)* riportato in Appendice A.4. La logica è illustrata in Fig.21.

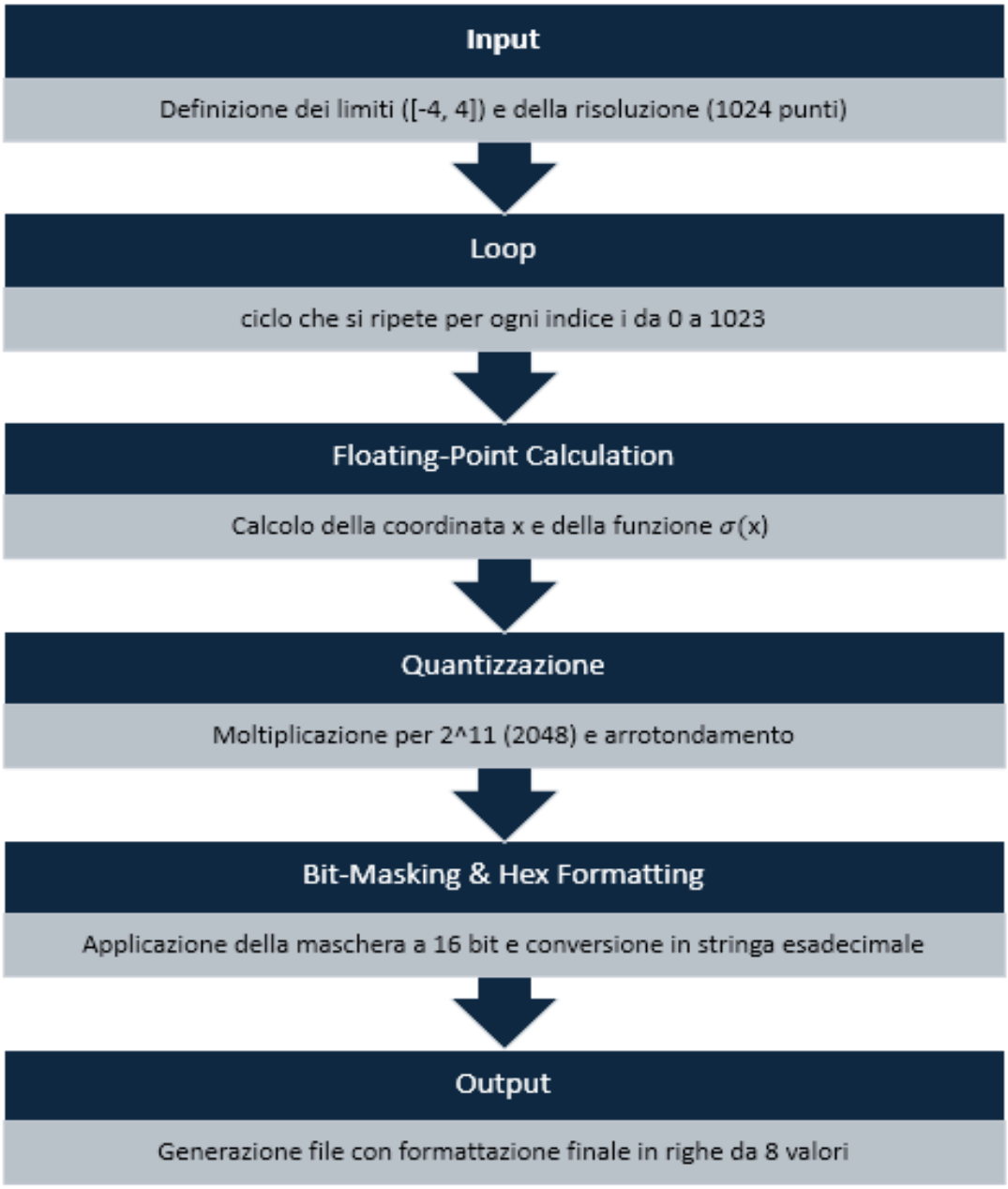


Figura 21 – Generazione coefficienti LUT da 1024 entry

3.1.2 Blocco di pre-processing intermedio

La logica interna del modulo di pre-elaborazione è stata strutturata secondo una pipeline a due stadi bilanciati, articolata in quattro fasi ottimizzate:

1- Nel primo stadio della pipeline, il dato in ingresso data_in viene convertito in formato segnato e sommato alla costante di offset 8192 (ossia +4.0 tradotto in formato Q5.11). Per prevenire fenomeni di overflow durante l'operazione algebrica, la parola viene preventivamente ridimensionata a 17 bit. Questa operazione trasla l'intero intervallo operativo nel dominio puramente positivo $[0\,;+8.0]$, memorizzando il risultato nel registro intermedio *sum_reg*. L'uso di catene di riporto veloci native della FPGA garantisce che questa addizione non costituisca un limite per il clock di sistema.

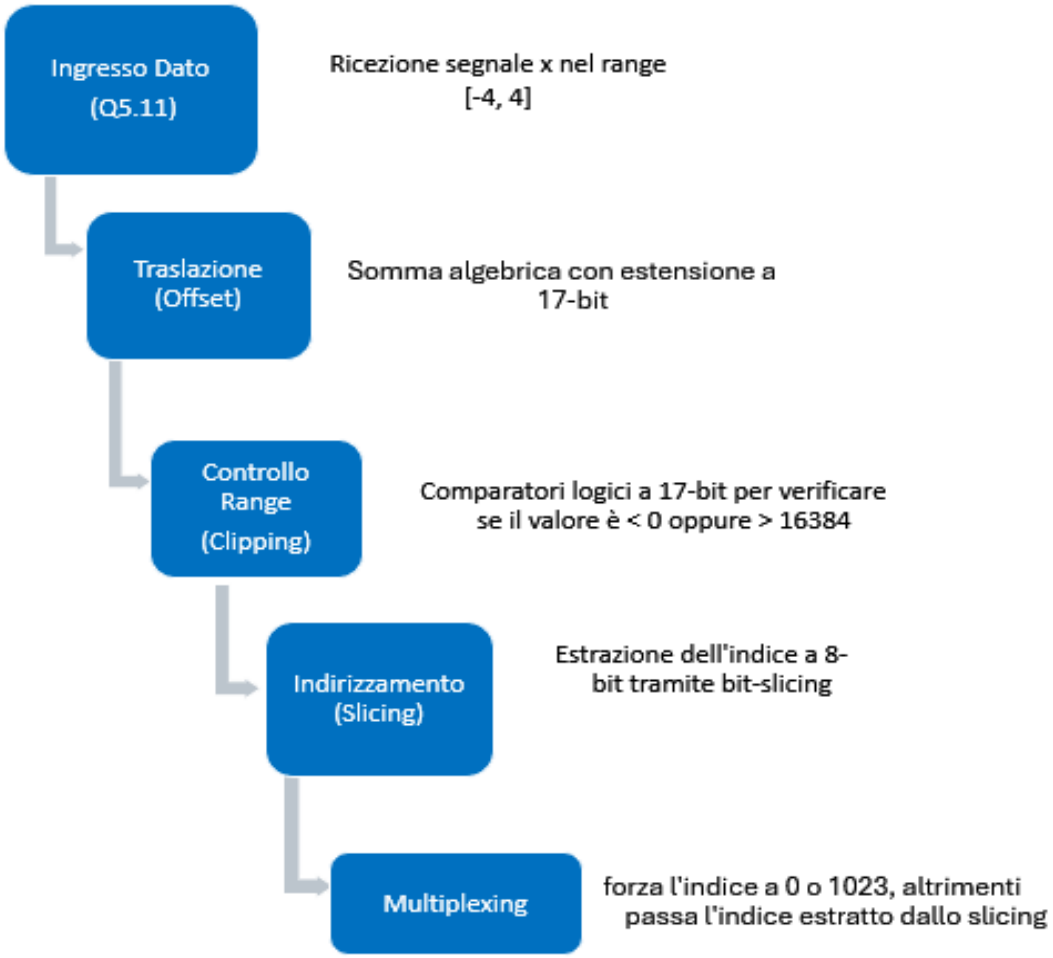


Figura 22 – Logica blocco di pre-processing intermedio

2- Nel secondo stadio della pipeline, per gestire in sicurezza dataset reali contenenti valori imprevedibili, viene eseguita una logica di clipping hardware a due soglie sul valore precedentemente registrato. Se la somma risulta minore di 0, l'indirizzo d'uscita viene forzato al limite inferiore 0. Se il valore incontra la condizione di saturazione superiore, verificata quando *sum_reg* ≥ 16384 (pari a +8.0 tradotto in Q5.11), oppure se si attiva il bit di overflow positivo (*sum_reg(16)* = '0' *e sum_reg(15)* = '1'), l'indice viene blindato al limite massimo di indirizzo 1023. Questa protezione impedisce errori di wrap-around nell'indirizzamento della BRAM, modellando in modo rigoroso l'asintoto orizzontale della funzione.

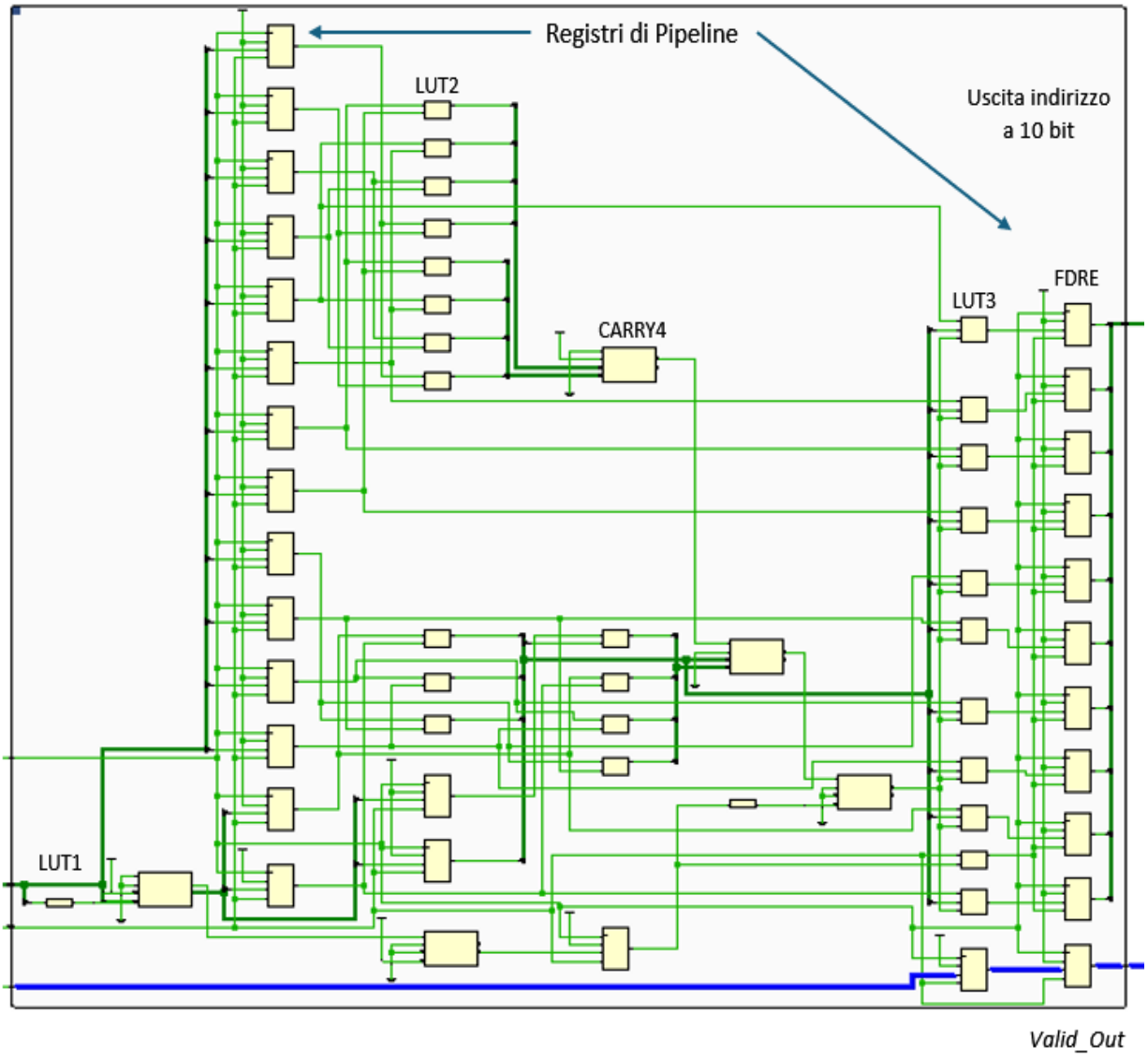


Figura 23 – Schematico post-layout blocco di pre-processing intermedio

Qualora il dato si trovi all'interno del dominio lineare di calcolo, l'estrazione dell'indice a 10 bit avviene tramite un'operazione di *bit-slicing* sincrona sul registro della somma, intercettando il bus nel range $(13 ; 4)$. Rispetto alla soluzione baseline, l'inclusione di due bit meno significativi aggiuntivi consente di quadruplicare la risoluzione della funzione, riducendo drasticamente l'errore di quantizzazione a costo hardware nullo, sfruttando appieno la granularità nativa della BRAM da 18 Kb.

La strutturazione del modulo in due stadi sequenziali precisi (calcolo dell'offset nel primo ciclo, clipping e slicing combinatorio nel secondo) consente di spezzare efficacemente il cammino critico. L'isolamento dei ritardi introdotti dai comparatori a 17 bit rispetto alla logica d'ingresso della memoria evita l'accumulo dei tempi di propagazione, stabilizzando il sistema e permettendo all'hardware di operare in sicurezza.

La logica di questo blocco è illustrata in Fig. 22, lo schematico post-implementazione in Fig. 23 mentre il codice è riportato integralmente in Appendice B.7.

A valle della fase di implementazione, è stata condotta un'analisi temporale statica dettagliata sul percorso critico emerso all'interno del modulo di pre-processing. Il report di timing, i cui dettagli sono illustrati in Fig. 24, fornisce una validazione quantitativa fondamentale dell'efficacia delle scelte architetturali e della strutturazione della pipeline.

Il sistema è stato implementato impostando un vincolo di clock pari a 4 ns (f = 250 MHz). In queste condizioni di lavoro, il report evidenzia un WNS positivo e pari a 0.773 ns. La presenza di questo margine conferma che tutti i segnali completano la loro transizione e si stabilizzano prima del sopraggiungere del fronte di salita attivo successivo, garantendo una protezione totale d fenomeni di metastabilità. Tale valore di slack, pari a circa il 19,3% dell'intero periodo di clock, attesta l'elevata robustezza e stabilita operativa del blocco.

Il ritardo totale del cammino critico esaminato (*Data Path Delay*) è pari a 3.195 ns e si sviluppa interamente all'interno del sottomodulo di pre-processing, propagandosi dal flip-flop sorgente *sum_reg_reg[8]* (uscita del blocco di calcolo dell'offset) fino al flip-flop di destinazione *addr_out_reg[0]* (ingresso del blocco di clipping e bit-slicing). Un dato di cruciale interesse per la validazione del layout riguarda la scomposizione analitica di questo ritardo:

- Logic Delay (1.850 ns - 57.911%): Rappresenta il tempo intrinseco impiegato dai transistor per effettuare la commutazione logica all'interno delle celle della FPGA. Lungo il percorso si riscontrano 4 livelli di logica complessivi, costituiti da elementi combinatori locali (LUT2, LUT3) e, in modo preminente, dalle catene di riporto hardware dedicate (CARRY4) impiegate per ottimizzare l'addizione a 17 bit.
- Route Delay (1.345 ns - 42.089%): Rappresenta il puro tempo di volo dei segnali elettrici durante la propagazione lungo le linee di interconnessione metalliche e le matrici di routing del chip.

Name	Path 1
Slack	0.773ns
Source	PP/sum_reg_reg[8]/C (rising edge-triggered cell FDRE clocked by sys_clk_pin {rise@0.000ns fall@2.000ns period=4.000ns})
Destination	PP/addr_out_reg[0]/D (rising edge-triggered cell FDRE clocked by sys_clk_pin {rise@0.000ns fall@2.000ns period=4.000ns})
Path Group	sys_clk_pin
Path Type	Setup (Max at Slow Process Corner)
Requirement	4.000ns (sys_clk_pin rise@4.000ns - sys_clk_pin rise@0.000ns)
Data Path Delay	3.195ns (logic 1.850ns (57.911%) route 1.345ns (42.089%))
Logic Levels	4 (CARRY4=2 LUT2=1 LUT3=1)
Clock ... Skew	-0.026ns
Clock U...tainty	0.035ns

Source Clock Path

Data Path

Delay Type	Incr (ns)	Path (ns)	Location	Cell Pin	Cell	Netlist Resources
FDRE (Prop_fdre_C_Q)	(f) 0.419	5.492	Site: SLICE_X7Y113	Q	sum_reg_reg[8] (FDRE	PP/sum_reg_reg[8]/Q
net (fo=3, routed)	0.676	6.168				PP/p_2_in[4]
			Site: SLICE_X6Y114	I0	addr_out1_carry__0_i_	PP/addr_out1_carry__
LUT2 (Prop_lut2_I0_O)	(r) 0.297	6.465	Site: SLICE_X6Y114	O	addr_out1_carry__0_i_	PP/addr_out1_carry__
net (fo=1, routed)	0.000	6.465				PP/addr_out1_carry__
			Site: SLICE_X6Y114	S[0]	addr_out1_carry__0 (CA	PP/addr_out1_carry__
CARRY4 (Prop_c...4_S[0]_CO[3])	(r) 0.513	6.978	Site: SLICE_X6Y114	CO[3]	addr_out1_carry__0 (CA	PP/addr_out1_carry__
net (fo=1, routed)	0.000	6.978				PP/addr_out1_carry__
			Site: SLICE_X6Y115	CI	addr_out1_carry__1 (CA	PP/addr_out1_carry__
CARRY4 (Prop_carry4_CI_CO[0])	(r) 0.254	7.232	Site: SLICE_X6Y115	CO[0]	addr_out1_carry__1 (CA	PP/addr_out1_carry__
net (fo=10, routed)	0.668	7.900				PP/addr_out1_carry__
			Site: SLICE_X7Y115	I2	addr_out[0]_i_1 (LUT3)	PP/addr_out[0]_i_1/I2
LUT3 (Prop_lut3_I2_O)	(r) 0.367	8.267	Site: SLICE_X7Y115	O	addr_out[0]_i_1 (LUT3)	PP/addr_out[0]_i_1/O
net (fo=1, routed)	0.000	8.267				PP/addr_out[0]_i_1_n_
FDRE			Site: SLICE_X7Y115	D	addr_out_reg[0] (FDRE)	PP/addr_out_reg[0]/D
Arrival Time		8.267				

Figura 24 – Analisi temporale dettagliata pre-processing V2

Il rapporto risultante certifica la correttezza del *Place & Route*. Tale bilanciamento indica che l'architettura è fisicamente efficiente: la frazione di routing contenuta (42%) esclude la presenza di congestioni locali nel posizionamento dei componenti, mentre il moderato ritardo logico (58%) architetturale conferma che la pipeline a due stadi introdotta nel pre-processing memorizza e spezza i cammini nel punto ideale della rete.

3.1.3 Blocco di memoria intermedio

Il cuore della funzione di attivazione risiede nella memorizzazione dei campioni della sigmoide. Nella versione V2, l'architettura della ROM è stata riprogettata per massimizzare l'efficienza nel contesto di un'architettura FPGA, forzando l'uso di risorse di memoria dedicate. A differenza della versione baseline, dove il tool di sintesi veniva istruito a mappare la ROM sulle LUT (Distributed RAM), nella versione V2 è stata imposta l'inferenza di BRAM (BRAM). Questo è stato ottenuto attraverso l'uso dell'attributo VHDL *ram_style = "block"*. Tale scelta è fondamentale per non saturare la logica combinatoria della FPGA, lasciando le LUT libere per le operazioni aritmetiche del resto del sistema. Utilizzando una primitiva RAMB18E1, è stato possibile mappare le 1024 entry (16-bit ciascuna) utilizzando un unico blocco di memoria dedicato. Questo ha permesso di quadruplicare la risoluzione della funzione rispetto alla baseline senza aumentare l'area occupata in termini di Slice.

Uno dei vantaggi cruciali delle BRAM Xilinx è la presenza di registri opzionali integrati sia all'ingresso (indirizzo) che all'uscita (dato). L'uso forzato delle BRAM con pipeline integrata offre vantaggi riguardanti il ritardo di setup della memoria, che viene drasticamente ridotto. Come evidenziato dal Timing Report, questo permette di mantenere uno slack positivo anche ad alte frequenze, poiché il percorso critico non deve più attraversare lunghe catene di interconnessione tra Flip-Flop generici e LUT. Inoltre la BRAM è intrinsecamente più efficiente della Distributed RAM per densità superiori alle 128 entry. L'attivazione dei registri interni riduce l'attività di commutazione sulle matrici di routing globali, limitando la potenza dinamica dissipata sotto forma di calore. Poiché la BRAM è un blocco hard-wired, i suoi tempi di accesso sono costanti e indipendenti dal piazzamento del resto della logica. Questo garantisce che la latenza del sistema rimanga deterministica (pari ai cicli di pipeline impostati).

Lo schematico post-implementazione è illustrato in Fig.25, mentre il codice integrale del blocco è riportato in Appendice B.8. Il top module dell'architettura V2 è riportato in Appendice B.9.

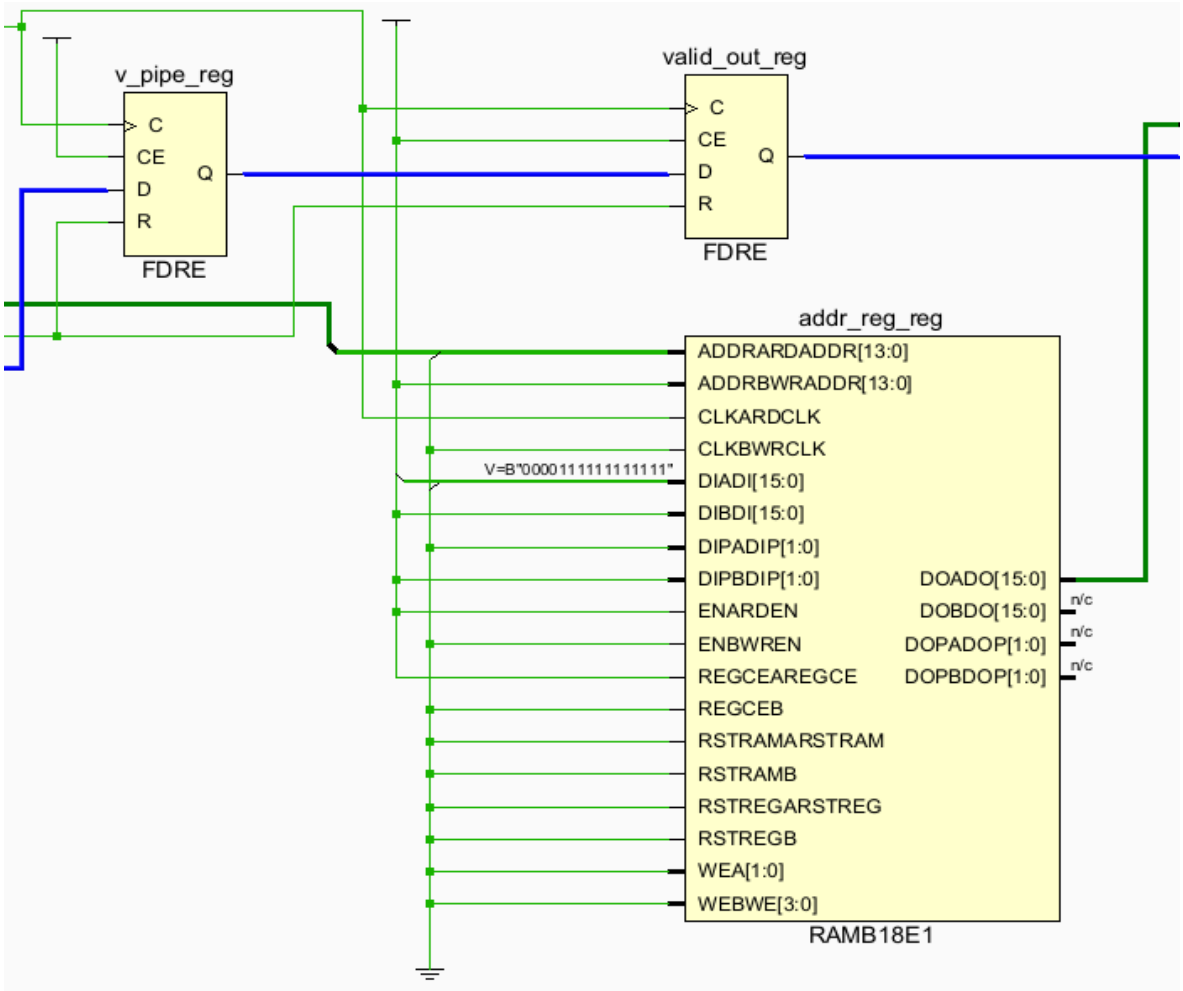


Figura 25 – Schematico post-layout blocco di memoria intermedio (BRAM)

3.1.4 Report post-layout e risultati architettura V2

Nonostante l'incremento di quattro volte della risoluzione e l'aggiunta di comparatori e multiplexer per la gestione della logica di clipping, il costo hardware è rimasto pressoché invariato rispetto alla versione baseline, come mostrato in Fig. 26: sfruttando la capacità residua della BRAM da 18 Kb (0.5 BRAM 36 Kb), si liberano le LUT per altri compiti computazionali, migliorando la densità logica del design. La frequenza di lavoro è stata preservata grazie alla strategia di pipelining analizzata precedentemente, offrendo una base solida per future implementazioni di parallelismo massivo.

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)	Bonded IOB (285)	BUFGCTRL (32)
N Sigmoid_Accelerator_TOP_V2	16	65	22	16	0.5	36	1
	0.01%	0.02%	0.07%	0.01%	0.14%	12.63%	3.13%

Figura 26 – Report di utilizzo V2

Nella configurazione baseline, il sistema registrava un WNS pari a 1.394 ns; nella variante V2, tale valore si attesta a 0.773 ns, pur mantenendo il medesimo vincolo di clock a 4 ns (250 MHz). Questo decremento dello slack era ampiamente previsto in fase di progettazione ed è imputabile a due fattori principali: in primo luogo, l'espansione della LUT a 1024 locazioni ha esteso il bus di indirizzamento a 10 bit, incrementando la complessità delle reti di routing locali; in secondo luogo, l'integrazione dei comparatori a 17 bit dedicati alla logica di clipping e al controllo dell'overflow ha introdotto un cammino combinatorio intrinsecamente più profondo rispetto alla baseline.

Tuttavia, il mantenimento di un WNS positivo di 0.773 ns, unitamente a un Total Negative Slack (TNS) nullo, dimostra la perfetta efficacia dello stadio di pipeline inserito nel modulo di pre-processing. Quest'ultimo è riuscito a isolare i ritardi dei comparatori impedendo il superamento del periodo di clock e assicurando un'architettura ad alte prestazioni, stabile e totalmente immune da violazioni di hold, come testimoniato dal report temporale in Fig. 27.

Setup

Worst Negative Slack (WNS):	0,773 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	68

Hold

Worst Hold Slack (WHS):	0,166 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	68

All user specified timing constraints are met.

Figura 27 – Report di timing V2

L'analisi dei consumi energetici, (Fig. 28), evidenzia una potenza totale assorbita dall'hardware pari a 168 mW. Il confronto con l'architettura baseline mostra che la potenza statica del dispositivo è rimasta costante a 131 mW (legata strettamente al leakage intrinseco del silicio a una temperatura di giunzione di 25.4 °C), mentre la potenza dinamica ha subito un incremento, passando da 0.018 W a 0.037 W, imputabile alle ottimizzazioni introdotte nella versione V2. In particolare, la migrazione della LUT da logica distribuita a un blocco di memoria dedicato BRAM (responsabile del 7% della quota dinamica, pari a 0.003 W) e l'introduzione dei comparatori a 17 bit per la logica di clipping hanno incrementato la capacità di commutazione complessiva del circuito.

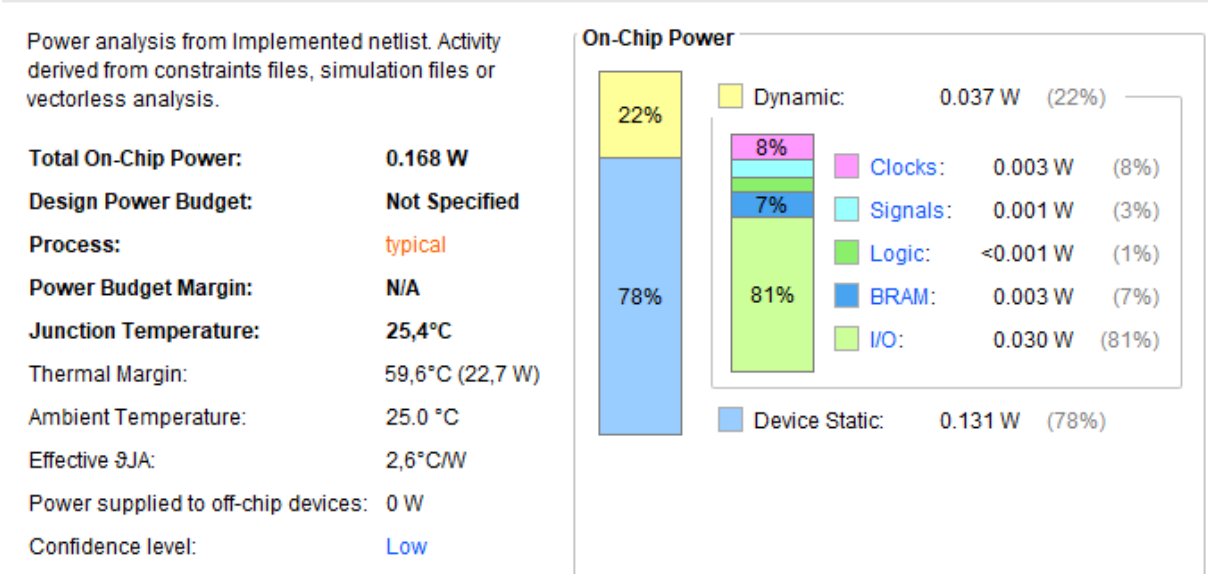


Figura 28 – Report di potenza V2 (vectorless)

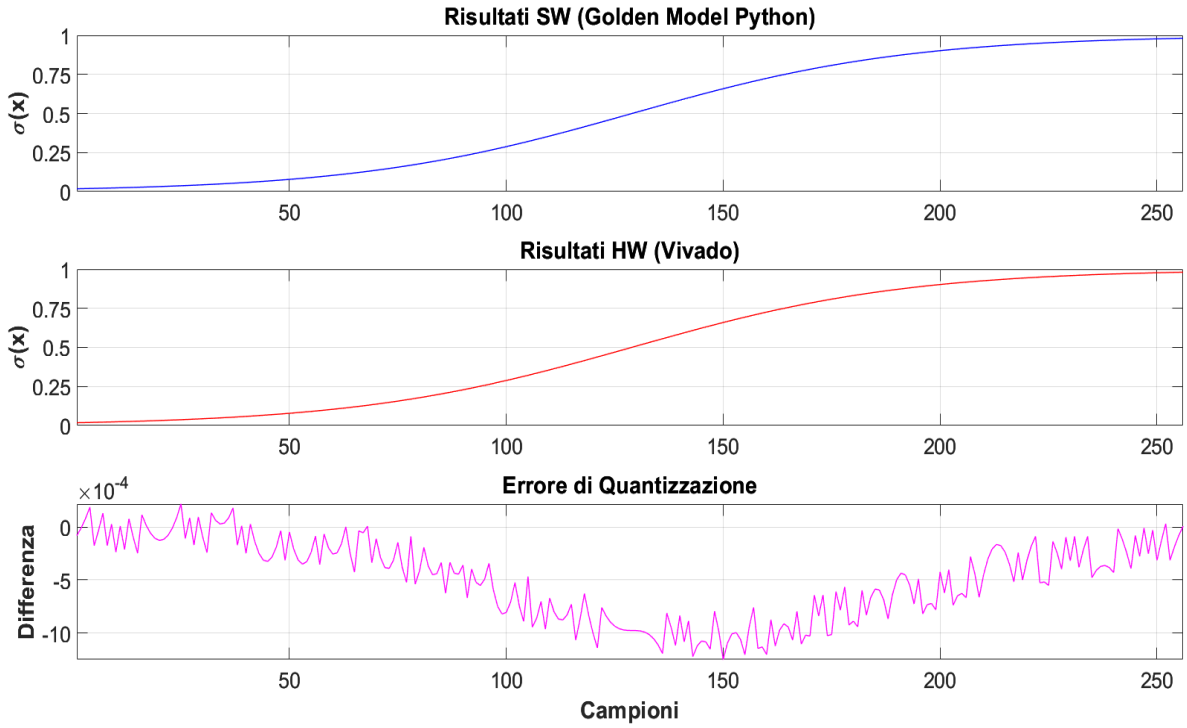
Per convalidare rigorosamente le nuove funzionalità introdotte nell'architettura ottimizzata V2, la strategia di verifica è stata estesa mediante la predisposizione di tre ambienti di simulazione che si avvalgono dello stesso testbench (TB_Sigmoid_VX) della versione baseline, ma con 3 vettori di ingressi differenti, nello specifico si distinguono:

- **Testbench di coerenza** (*TB_Sigmoid_V2.vhd*): Identico al testbench della versione baseline, utilizza il medesimo file di stimoli (*stimoli_input_baseline.mem*), composto dalla rampa standard di 256 valori confinati nel dominio $[-4.0\,;\,+4.0]$, per vedere gli effetti delle modifiche architetturali rispetto al modello di partenza e restituisce il file di testo *risultati_HW_256.hex*;

- **Testbench di robustezza** (*TB_Stability.vhd*): Il secondo ambiente, riportato in Appendice B.10, è mirato esclusivamente a verificare la tenuta della logica di saturazione deterministica a fronte di input fuori scala provenienti da framework esterni. A tale scopo, tramite uno script Python dedicato, riportato in Appendice A.5 è stato generato il file di *stimoli_input_12.mem,* contenente 256 campioni estesi sul dominio $[-12.0\ ;\ 12.0]$. Questa simulazione ha permesso di certificare la capacità dell'hardware di intercettare gli overflow, bloccando l'indirizzamento della BRAM esattamente alle soglie limite (0 e 1023) ed evitando fenomeni di *wrap-around* del dato. Restituisce il file di testo *risultati_HW_12.hex*;
- **Testbench ad alta risoluzione** (*TB_Full_ROM.vhd*): Il terzo ambiente, riportato in Appendice B.15, sfrutta appieno il potenziale della nuova BRAM da 18 Kb mappata a 1024 locazioni. In questo caso, l'input è costituito dal file di testo (*stimoli_input_1024.mem*) generato da uno script Python, riportato in Appendice A.6, contenente 1024 ingressi distinti. Questo test ha consentito di verificare il corretto funzionamento del meccanismo di *bit-slicing* sui bit meno significativi del bus e di validare l'incremento di precisione analitica della funzione su una griglia di campionamento quattro volte più densa rispetto alla baseline. Restituisce il file di testo *risultati_HW_1024.hex*

L'approccio metodologico basato su questa suite di test ha permesso di isolare e validare singolarmente ogni componente dell'architettura V2, garantendo la totale stabilità del sistema sia in condizioni operative standard che in scenari di input estremi.

Dopo aver eseguito il primo testbench di coerenza, e aver esaminato il file di testo generato da Vivado in seguito alla simulazone temporale post-implementazione, si nota come il passaggio all'architettura V2 a 1024 campioni abbia prodotto un salto qualitativo nel profilo di approssimazione della funzione. Come evidenziato dai grafici di confronto generati in ambiente MATLAB (Fig. 29), la curva campionata risulta pressoché sovrapponibile alla funzione teorica, eliminando i macro-errori di quantizzazione presenti nella baseline. I risultati numerici confermano l'efficacia della risoluzione a 10 bit. Tale livello di accuratezza è fondamentale per i Vision Transformers, dove piccole deviazioni nelle funzioni di attivazione possono accumularsi attraverso i numerosi layer, degradando la precisione finale della classificazione.




```
MAE: 0.0005023378
Errore Massimo: 0.0012504682
```

Figura 29 – Confronto MATLAB risultati V2

A validare l'accuratezza analitica dell'acceleratore, è stata condotta un'analisi degli errori di quantizzazione dell'architettura ottimizzata V2, assumendo come riferimento ideale il modello software utilizzato precedente nella versione baseline. I risultati numerici, ottenuti dal testbench di coerenza, evidenziano un incremento della precisione del sistema. Nella configurazione baseline, l'approssimazione mostrava un MAE pari a 0.002399 e un Errore Massimo di 0.018546. Con l'introduzione dell'architettura V2, il MAE crolla drasticamente a 0.000502, registrando un miglioramento di quasi cinque volte. Ancor più significativo è il comportamento dell'errore massimo, il quale scende a soli 0.001250, riducendosi di quasi quindici volte rispetto al design primitivo. Tale abbattimento dell'errore residuo dimostra che la fittezza della nuova griglia di campionamento cattura fedelmente i transitori non lineari della curva asintotica. L'incremento di precisione numerica, ottenuto a costo hardware nullo grazie all'impiego efficiente delle BRAM native, assicura la perfetta convergenza dei pesi numerici durante l'inferenza di Transformer e reti neurali profonde.

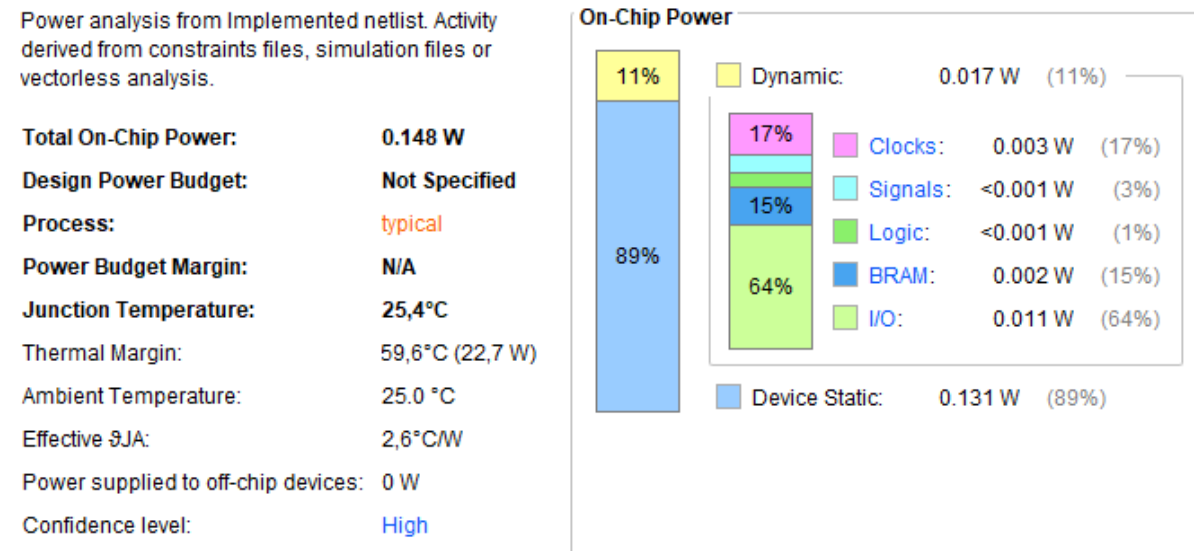


Figura 30 – Report di potenza V2 (SAIF relativo a TB coerenza)

A seguito dell'integrazione del file SAIF, il grado di confidenza del report è passato a High, rivelando una potenza dinamica effettiva di 0.017 W, inferiore di oltre il 50% rispetto alla stima iniziale (Fig.30).

Tale riduzione è attribuibile principalmente a una ricalibrazione del consumo dinamico relativo ai blocchi di I/O, che nell'analisi iniziale risultava sovrastimato. Il dato ottenuto, confermando un consumo di potenza totale di 0.148 W, garantisce un ampio margine termico rispetto alle specifiche del target FPGA, confermando l'efficienza energetica della soluzione implementata per l'accelerazione della funzione di attivazione.

Dopo aver eseguito il secondo testbench di robustezza, si osservano le forme d'onda di Fig. 31. La simulazione post-layout a 250 MHz conferma la piena stabilità dell'architettura ottimizzata a fronte di input stressanti che saturano i limiti nominali della LUT. La traccia dell'input mostra una rampa lineare estesa su un dominio triplo rispetto allo standard, spaziando da -12.0 a +12.0, simulando una condizione di input imprevedibile proveniente da framework esterni.

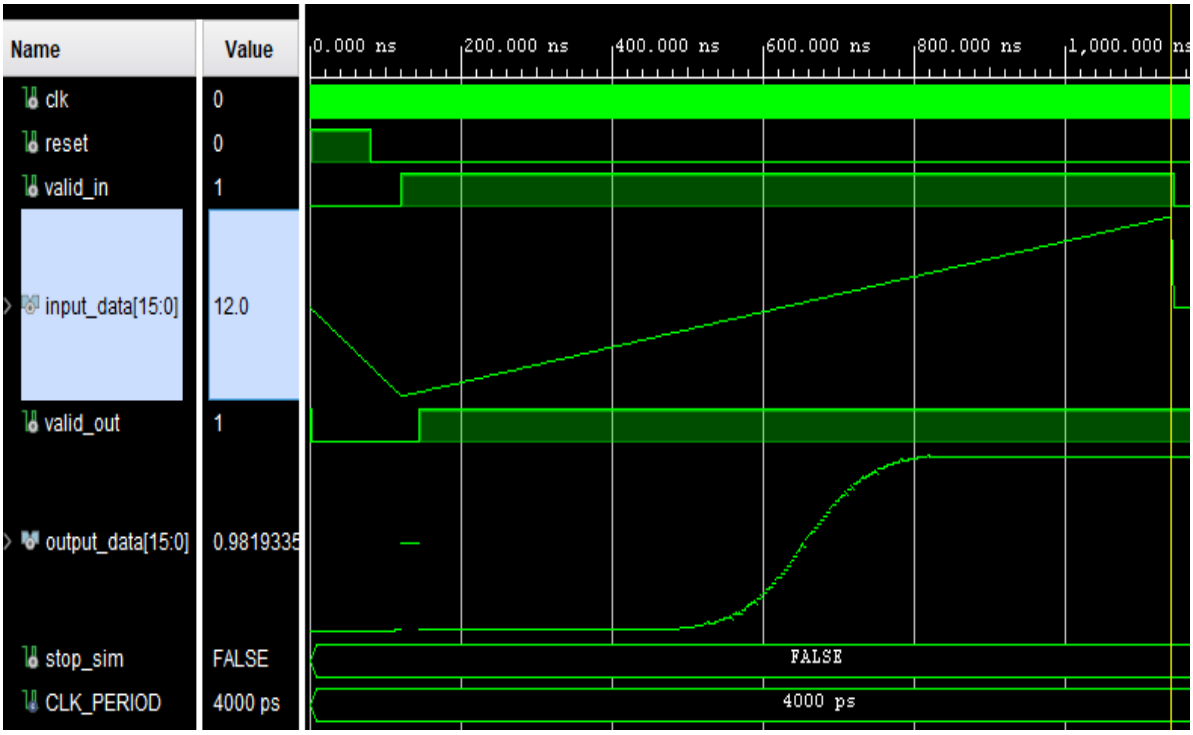


Figura 31 – Forma d'onda analogica in simulazione V2 (TB robustezza)

Dopo una latenza deterministica di 6 cicli di clock dall'asserzione di valid_in, coerente con l'architettura a 6 stadi implementata, il segnale *output_data* descrive la caratteristica curva a 'S' della sigmoide. È cruciale osservare come l'uscita non subisca rollover o errori di wrap-around quando l'ingresso eccede i limiti di campionamento utile. La logica di pre-processing intercetta correttamente gli overflow, ancorando deterministicamente l'output ai valori di saturazione asintotica validando così l'efficacia e la robustezza del clipping hardware. In Fig. 32 appare evidente come la saturazione del segnale di ingresso minimizzi l'attività di commutazione all'interno della pipeline di calcolo. Una volta raggiunto il valore di soglia, i segnali di ingresso si mantengono costanti, eliminando le transizioni logiche non necessarie nei blocchi di elaborazione a valle. Questo risultato attesta l'efficienza della strategia di clipping adottata: oltre a garantire la corretta saturazione della funzione, essa agisce come un meccanismo di risparmio energetico passivo, riducendo lo switching dinamico proporzionalmente al numero di campioni in ingresso che eccedono il range operativo.

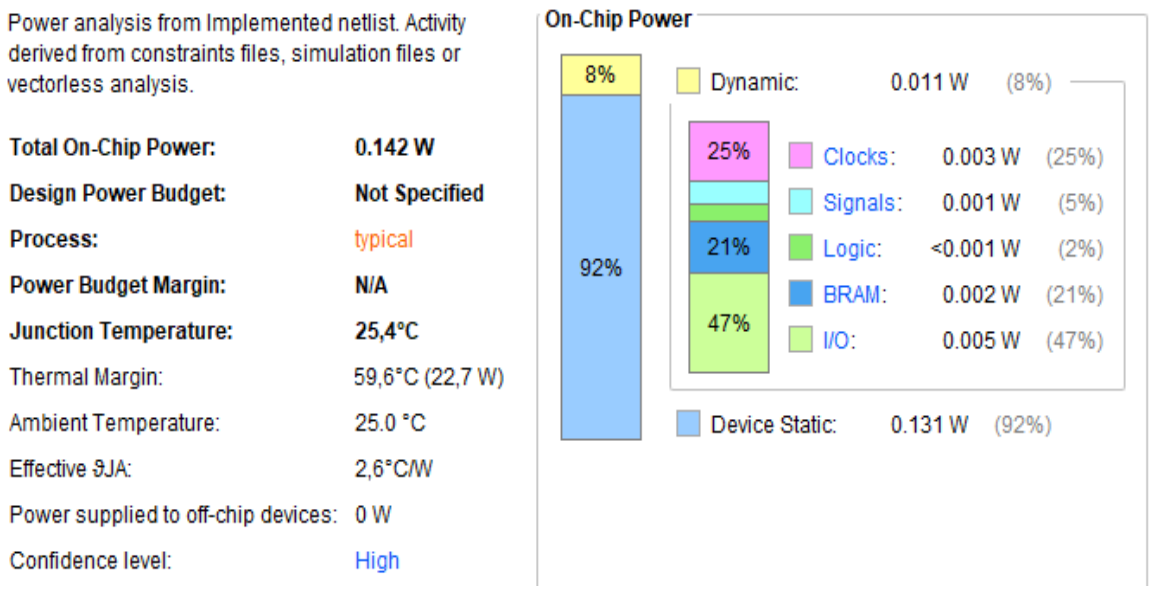


Figura 32 – Report di potenza V2 (SAIF relativo a TB robustezza)

In Fig. 33 si evidenzia la pipeline a 6 stadi dell'architettura V2, mentre in Fig. 34 si può apprezzare la forma d'onda analogica. Il confronto visivo tra le forme d'onda simulate per il design a 256 campioni e quello ottimizzato a 1024 campioni rivela un'apparente maggiore rumorosità nella versione ad alta risoluzione. Tale fenomeno è di natura puramente visuale e non indica una degradazione del segnale digitale.

Ciò è dovuto alla quadruplicazione della risoluzione della LUT, che introduce transizioni di valore infinitesimali ad ogni singolo ciclo di clock. Quando il simulatore renderizza l'intero dominio temporale della sigmoide, queste micro-transizioni digitali, fittamente addensate nel tempo, vengono interpretate visivamente come una linea scalettata, a differenza della linea apparentemente più pulita dei 256 campioni, che deriva da salti digitali più netti e distanti. Gli spike verticali osservabili sono artefatti di renderizzazione del simulatore in modalità visualizzazione analogica e non corrispondono a disturbi nel circuito hardware reale. La curva approssimata, in questo caso, segue il modello analitico con una granularità più fine.

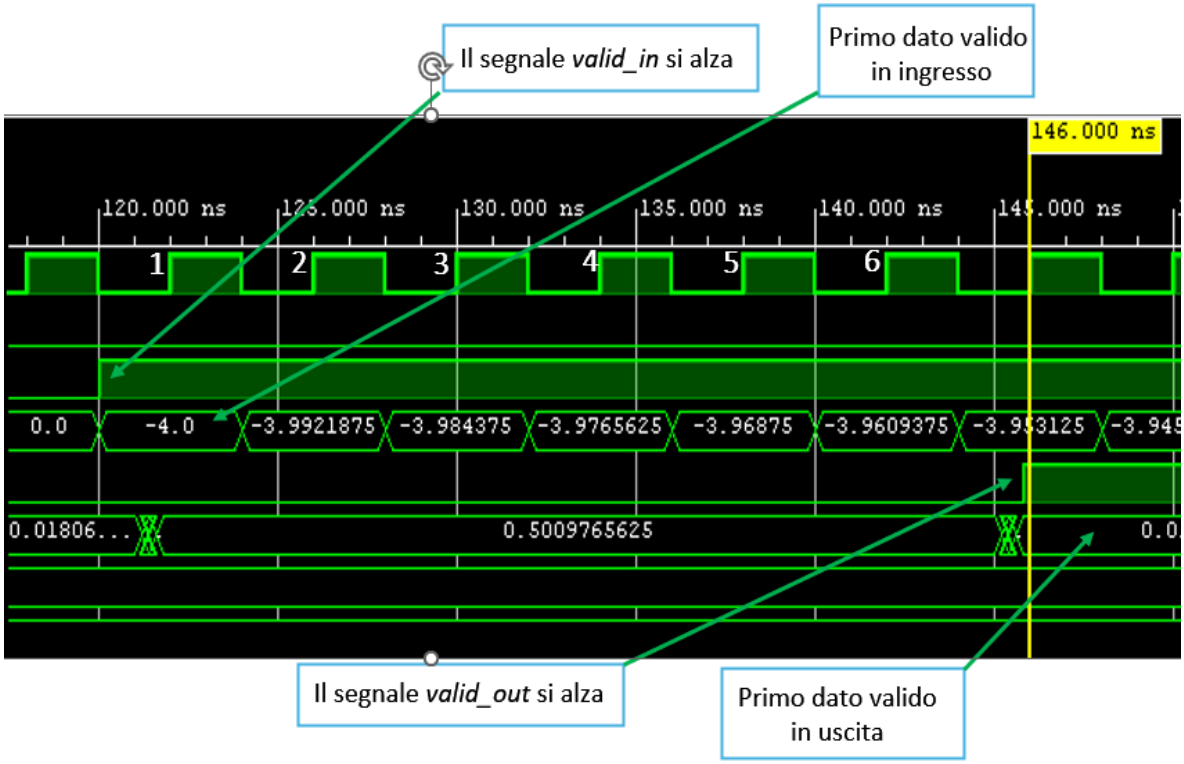


Figura 33 – Simulazione timing post-layout V2 (TB alta risoluzione)

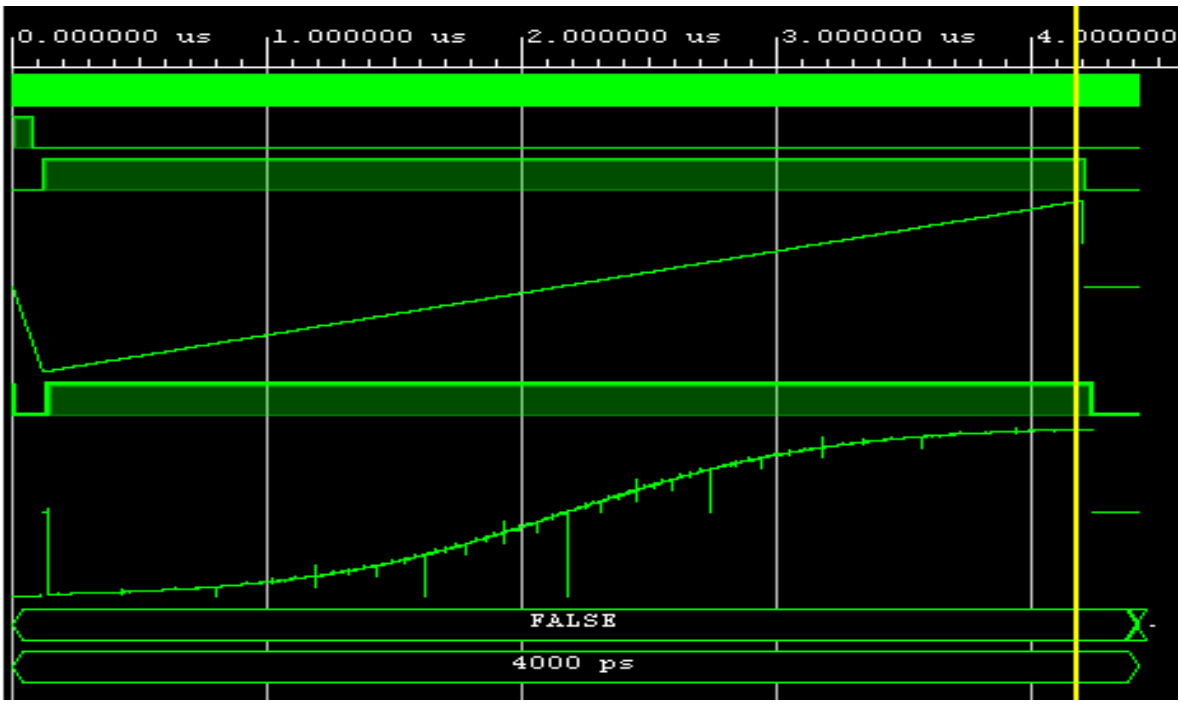


Figura 34 – Forma d'onda analogica in simulazione V2 (TB alta risoluzione)

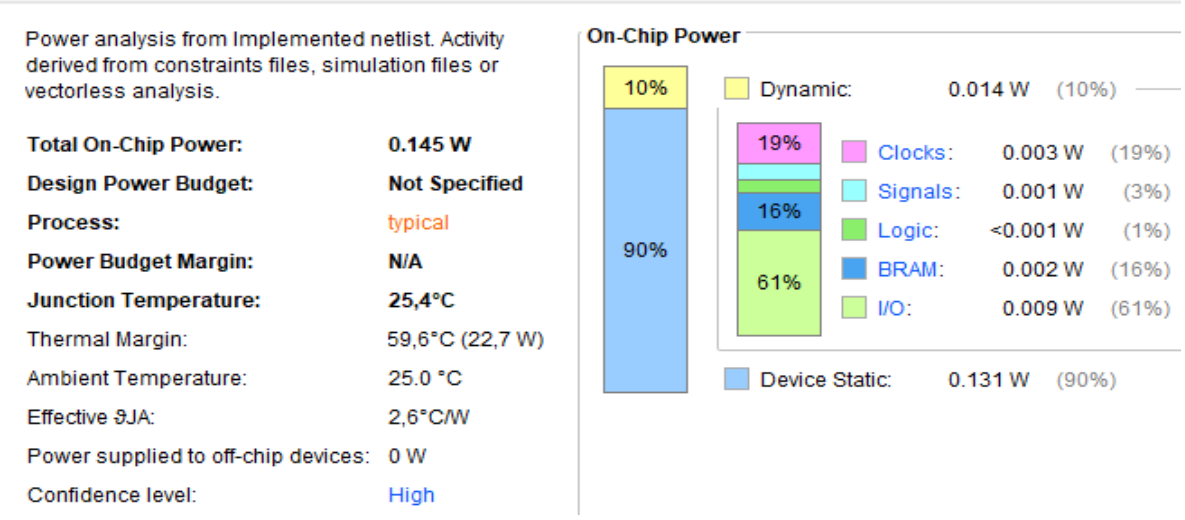


Figura 35 – Report di potenza V2 (SAIF relativo a TB alta risoluzione)

Il testbench esteso su 1024 campioni, che copre l'intero dominio di definizione della funzione sigmoide implementato nella ROM, ha fornito la validazione finale dell'efficienza energetica dell'architettura v2. Il report di potenza post-SAIF, visibile in Fig. 35, conferma che l'ottimizzazione del design verso una risoluzione quattro volte superiore non ha introdotto alcuna penalizzazione energetica, mantenendo la potenza dinamica ferma a 0.014 W, ma garantendo un sensibile guadagno in termini di precisione algoritmica. Tale stabilità del consumo energetico, pur a fronte di una maggiore densità di dati processati, evidenzia l'efficacia del pipelining adottato e l'ottimizzazione dell'interfaccia con l'approccio BRAM based.

3.2 Progettazione architettura avanzata V3

La versione avanzata dell'acceleratore hardware (V3) rappresenta un'evoluzione ulteriormente ottimizzata rispetto all'architettura baseline, progettata con l'obiettivo di migliorare l'accuratezza numerica del calcolo della funzione sigmoide, mantenendo al contempo contenuta l'occupazione di area sul chip. L'ottimizzazione si basa su tre strategie sequenziali sul data-path:

- Saturazione Dinamica (Clipping): Poiché per valori dell'input esterni all'intervallo $[-8, +8]$ l'uscita della sigmoide tende asintoticamente verso i valori limite 0 e 1 (con un errore di quantizzazione trascurabile nel formato a 16 bit), è stato integrato un modulo di clipping d'ingresso. Tutti i valori superiori a +8 o inferiori a -8 vengono saturati a tali estremi, confinando la successiva area di calcolo a un range dinamico ristretto.
- Dimezzamento della Memoria di Mappatura: Sfruttando il clipping, la LUT interna è stata progettata per memorizzare esclusivamente i valori della funzione appartenenti al semiasse positivo, ovvero nell'intervallo $[0\,;\,+8]$, codificati nel formato ad alta precisione Q1.15 (1 bit di segno e 15 per la parte decimale, sufficiente per rappresentare l'intervallo di uscita da 0.5 a 1). Questo ha permesso di dimezzare istantaneamente i requisiti di memorizzazione rispetto a una mappatura d'intervallo completa.
- Ricostruzione per Simmetria: Per i valori d'ingresso originariamente negativi appartenenti al range $[-8\,;\,0]$, l'architettura sfrutta la proprietà di simmetria centrale della sigmoide, espressa dalla relazione $\sigma(x) = 1 - \sigma(-x)$. Un'apposita logica combinatoria a valle della ROM si occupa di invertire il segno dell'input per l'indirizzamento e, successivamente, di complementare l'uscita letta, ricostruendo l'intera forma d'onda in modo duale.

3.2.1 Package Q1.15

Poiché i valori d'uscita memorizzati nella LUT per il range $[0\,;\,8]$ sono rigorosamente compresi tra 0.5 e 1.0, la parte intera del numero non necessita di bit dedicati (il valore limite 1.0 asintotico viene approssimato dal massimo valore frazionario rappresentabile). Per questa ragione, al fine di massimizzare la precisione numerica sul bus a 16 bit, è stato adottato il formato in fixed-point Q1.15. In questa specifica configurazione 1 bit è riservato al segno (MSB, fissato a 0 essendo i valori sempre positivi), mentre 15 bit sono dedicati interamente alla parte frazionaria.

Grazie a questa scelta, la risoluzione frazionaria della memoria aumenta, garantendo un errore di quantizzazione del coefficiente praticamente trascurabile rispetto al calcolo in virgola mobile a doppia precisione di Python.

Lo script Python, riportato in Appendice A.7, esegue la conversione dei valori floating-point nel formato esadecimale Q1.15 per l'importazione diretta in ambiente Xilinx Vivado secondo i seguenti passi algoritmici:

1. Campionamento dell'Input: Calcolo del valore della sigmoide per 1024 punti distribuiti uniformemente nell'intervallo [0, 8], con un passo di quantizzazione dell'input pari a $\Delta x = \frac{8}{1024} = 0.0078125$.
2. Scaling fixed-point: Moltiplicazione del valore floating-point ottenuto per il fattore di scala 2^{15} (ovvero 32768), per traslare la frazione nel dominio degli interi.
3. Troncamento: Applicazione dell'operazione di arrotondamento all'intero più vicino e successivo cast a tipo intero a 16 bit senza segno.
4. Conversione Esadecimale: Formattazione del valore intero in una stringa esadecimale a 4 cifre (il valore d'inizio scala $\sigma(0) = 0.5$) viene convertito nell'intero 16384, che corrisponde alla stringa esadecimale 4000).

I 1024 valori esadecimali così generati vengono inseriti nel package *Sigmoid_Values_1024_0_8.vhd,* riportato in Appendice B.11. Questo file viene successivamente richiamato all'interno del codice VHDL per inizializzare in modo statico e automatico i blocchi di memoria BRAM della FPGA in fase di sintesi, costituendo il nucleo della LUT dell'architettura V3.

3.2.2 Blocco di pre-processing avanzato

Il modulo *Pre_Processing_V3.vhd* costituisce lo stadio d'ingresso dell'acceleratore hardware. Il suo scopo è duplice: da un lato implementa le operazioni geometrico-matematiche necessarie a sfruttare la simmetria della funzione d'attivazione, dall'altro ottimizza la risoluzione del data-path d'ingresso per pilotare in sicurezza la LUT, minimizzando l'occupazione di memoria.

La logica di questo blocco è illustrata in Fig. 36, lo schematico post-sintesi in Fig. 37, mentre il codice che lo descrive è riportato integralmente in Appendice B.12.

Per sostenere frequenze operative elevate e spezzare i cammini critici della logica combinatoria, il modulo è stato progettato secondo un'architettura pipeline sincrona a due stadi operante sul fronte di salita del clock.

L'elaborazione avviene in maniera strettamente sequenziale attraverso due barriere di registri dedicate:

- Stadio I: Estrazione del Valore Assoluto e Tracciamento del Segno

Al primo fronte di salita del clock, il bus d'ingresso *data_in* (espresso in formato in fixed-point Q5.11) viene acquisito. Una rete combinatoria verifica il bit di segno del dato per calcolarne il valore assoluto:

  - Qualora il dato sia negativo, viene applicata l'operazione di inversione in complemento a due (-*data_in*). Per evitare anomalie di overflow sul valore minimo rappresentabile a 16 bit, l'operando viene preventivamente ridimensionato a 17 bit tramite l'operatore *resize*. Contestualmente, il flag di segno interno viene asserito a 1.
  - Qualora il dato sia positivo o nullo, il vettore viene esteso a 17 bit preservandone il valore e il flag di segno viene posto a 0.

I risultati di questo primo stadio vengono stabilizzati all'interno dei registri *abs_val* e *neg_reg*, mentre il segnale di controllo d'ingresso viene memorizzato nel registro *valid_reg*.

- Stadio II: Condizionamento di Clipping e Slicing dell'Indirizzo

Al successivo fronte di clock, la logica combinatoria del secondo stadio opera sui valori precedentemente registrati. Il valore assoluto *abs_va*l viene confrontato con la costante di saturazione rigida *CLIP_VAL*, il cui valore numerico di 16384 rappresenta la soglia di stabilità asintotica +8.0 nel formato Q5.11:

  - Se abs_val eccede o eguaglia la soglia, la variabile d'ambiente clipped viene forzata al valore limite di 8.0.

- In caso contrario, la variabile acquisisce i bit del valore assoluto d'origine.
- A valle del multiplexer di clipping, viene eseguito lo slicing per la determinazione dell'indirizzo di uscita. Il bus *addr_out* viene generato estraendo la finestra di bit clipped (*13 downto 4*). Questa selezione esclude il bit di segno (nullo dopo il valore assoluto) e scarta i 4 bit frazionari meno significativi (*3 downto 0*). L'operazione attua un troncamento della precisione frazionaria che permette di mappare l'intero intervallo utile $[0, 8]$ su un range d'indirizzamento a 10 bit, ideale per una ROM da 1024 parole.

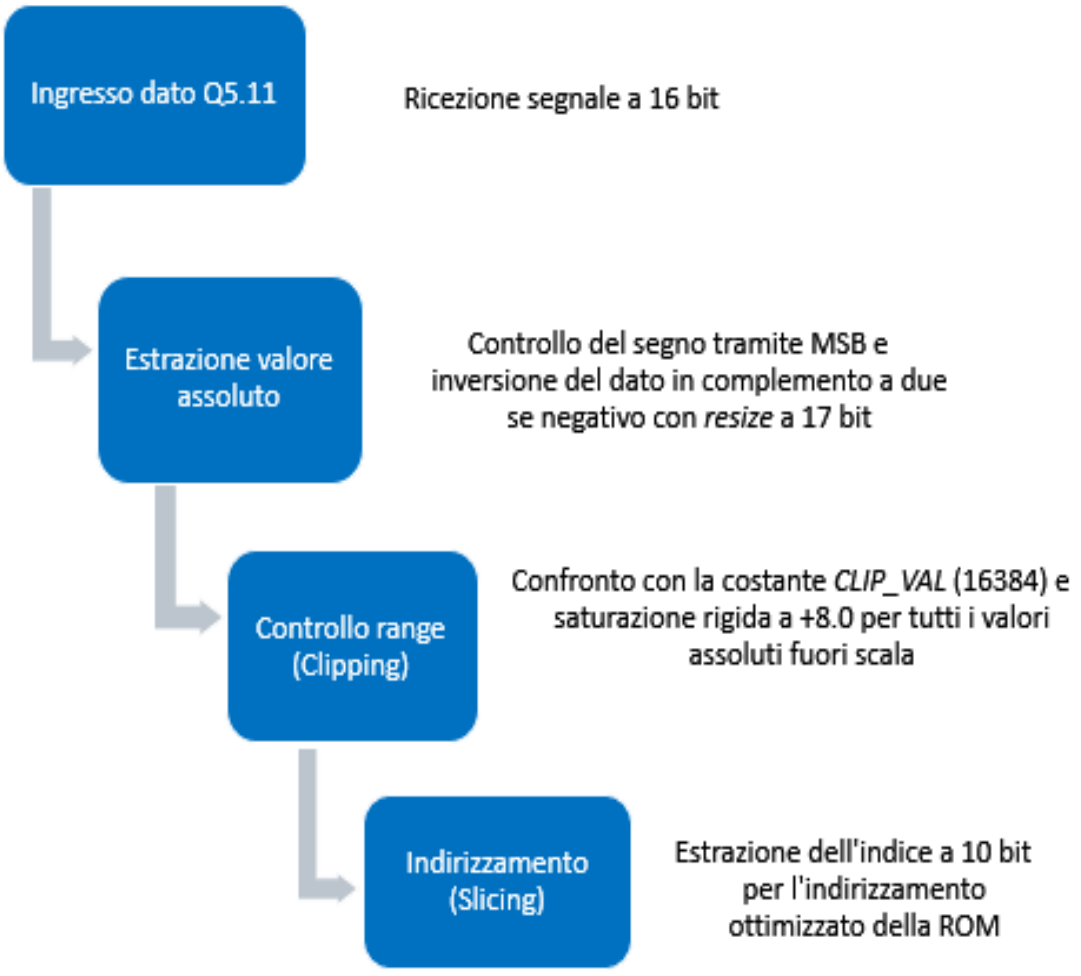


Figura 36 – Logica blocco di pre-processing avanzato

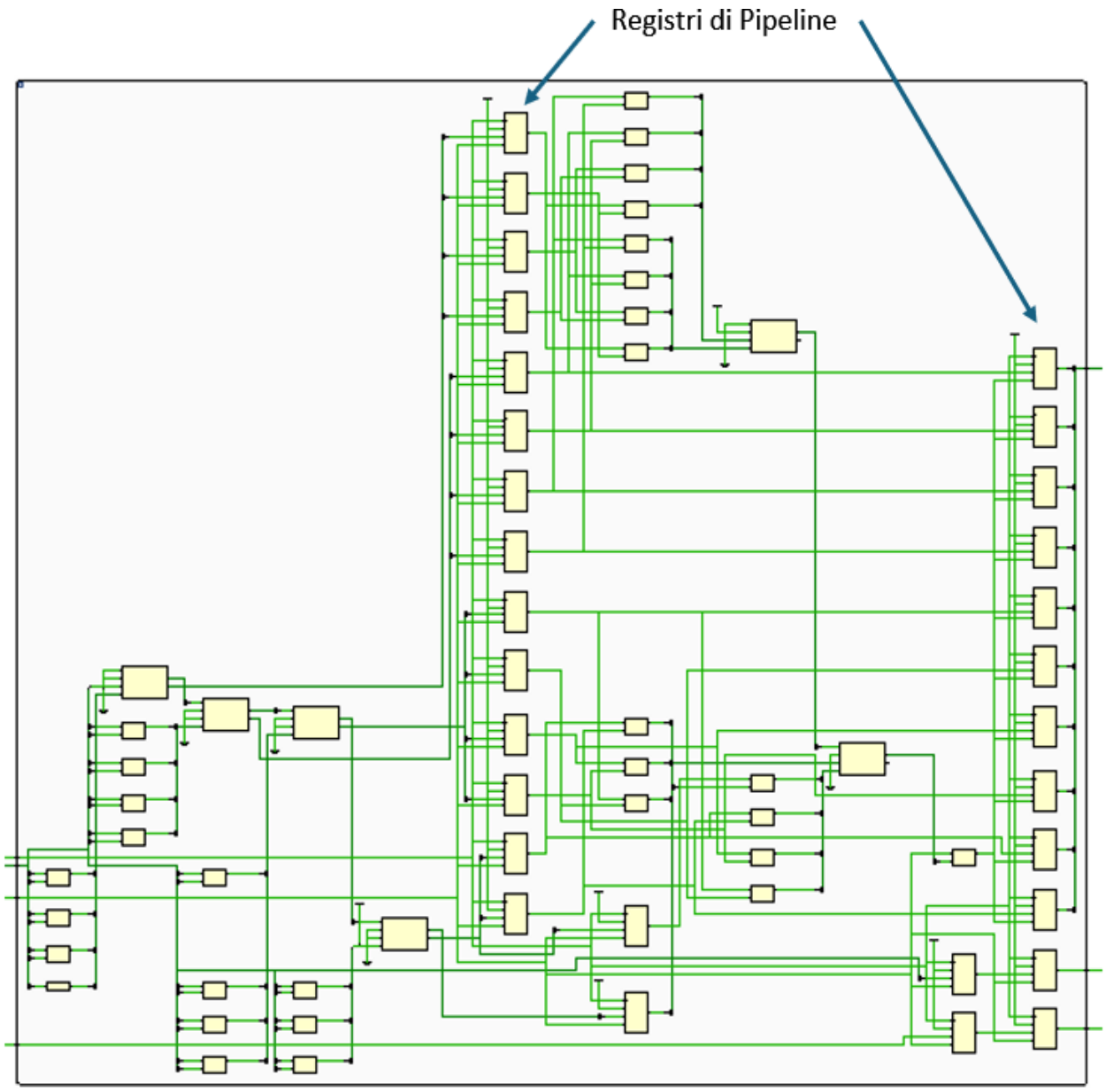


Figura 37 – Schematico post-layout blocco di pre-processing avanzato

Poiché la logica combinatoria di clipping e slicing introduce ritardi di propagazione sul data-path prima del campionamento finale su *addr_out*, i rispettivi segnali di controllo associati al record in transito devono subire lo stesso identico ritardo di pipeline.

Come visibile dal codice, i segnali *neg_reg* e *valid_reg* fungono da stadi di transito: nel medesimo istante in cui la logica del secondo stadio calcola l'indirizzo e lo deposita sul registro di uscita *addr_out*, i flag di controllo vengono aggiornati sui registri d'uscita finali *is_neg_out* e *valid_out*.

Questo schema assicura che l'indirizzo e le sue informazioni di controllo (validità e segno originario) emergano dal blocco di pre-processing in perfetto sincronismo temporale dopo esattamente due cicli di clock di latenza, pronti per essere interpretati correttamente dal blocco di memoria e di ricostruzione della simmetria a valle.

3.2.3 Blocco di memoria avanzato

Il modulo *Sigmoid_LUT_V3.vhd* costituisce il nucleo di calcolo e memorizzazione dell'acceleratore ottimizzato. Al fine di ridurre al minimo il consumo di risorse logiche distribuite sulla FPGA, la (LUT) contenente i 1024 coefficienti della funzione nell'intervallo $[0, 8]$, precalcolati in formato Q1.15, è stata forzata alla sintesi su blocchi di memoria dedicati on-chip tramite l'attributo *rom_style := "block".* L'architettura del data-path è stata progettata introducendo una pipeline a tre stadi sincroni, per sfruttare nativamente l'accoppiamento tra i registri d'interfaccia e la struttura fisica delle BRAM di Xilinx. Nel primo stadio, l'indirizzo d'ingresso e i relativi segnali di controllo (*is_neg_in e valid_in*) vengono campionati sul fronte di salita del clock per isolare i delay di routing provenienti dal blocco precedente.

Nel secondo stadio, l'indirizzo memorizzato effettua l'accesso sincrono alla memoria; il dato letto viene intercettato direttamente all'uscita della matrice dal registro nativo integrato nella BRAM (*DOA_REG*), azzerando i ritardi di propagazione interni, mentre i flag di controllo avanzano in parallelo per preservare la coerenza temporale del record. Nel terzo e ultimo stadio, la logica combinatoria attua in un solo ciclo di clock la ricostruzione matematica della funzione sfruttando la proprietà di simmetria della sigmoide: se il flag di segno originario indica che l'input era negativo, il valore letto viene sottratto in logica d'interi dalla costante ONE (fissata a 32767, equivalente all'unità nel formato Q1.15), invertendo specularmente la curva; in caso contrario, il coefficiente viene trasferito inalterato. Questo schema di pipeline strutturato garantisce il calcolo accurato dell'attivazione e l'allineamento perfetto del segnale *valid_out* sul bus d'uscita dopo una latenza fissa di tre cicli di clock, massimizzando il throughput complessivo. Lo schematico post-implementazione di questo blocco è mostrato in Fig. 38 mentre il codice è riportato integralmente in Appendice B.13.

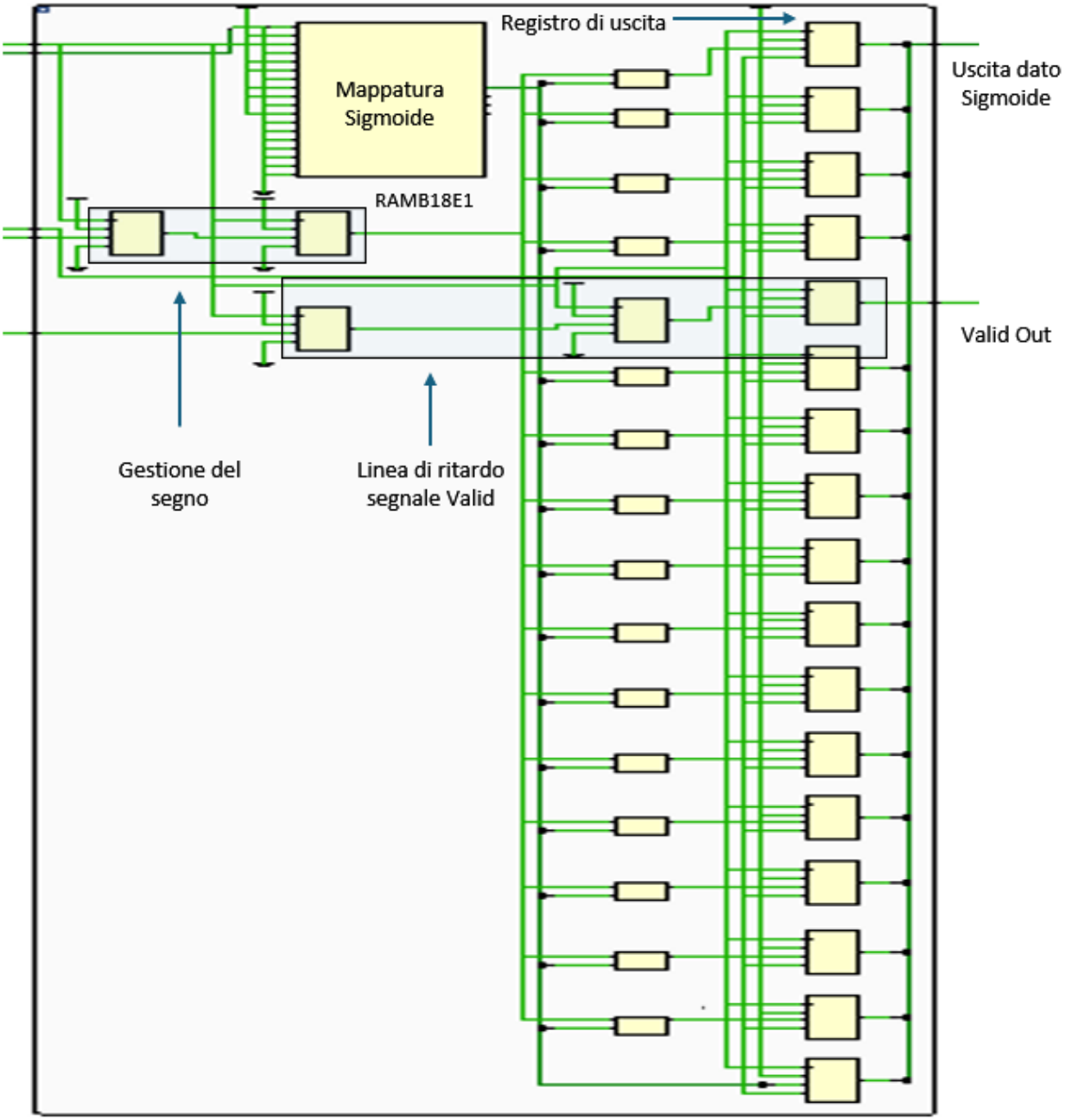


Figura 38 – Schematico post-layout blocco di memoria avanzato

3.2.4 Report post-layout e risultati architettura V3

La corretta sincronizzazione dei flussi di dato e la stabilità temporale dell'architettura V3 trovano un riscontro diretto nell'analisi del transitorio di attivazione mostrato nella simulazione in Fig. 39.

Come evidenziato dalle annotazioni grafiche, l'inizio del ciclo computazionale avviene con l'asserzione del segnale di controllo *valid_in* e la contestuale presentazione del primo campione valido sul bus d'ingresso. Coerentemente con la struttura implementata e finalizzata al bilanciamento dei cammini critici, il fronte d'onda attraversa sequenzialmente i 7 stadi di pipeline totali distribuiti nell'architettura, dopo di che il segnale di abilitazione d'uscita *valid_out* si commuta ad alto, indicando la disponibilità del dato finalizzato. Al successivo fronte di campionamento utile, il bus *output_data* si assesta sul valore quantizzato di 0.0179443, corrispondente al valore analitico ideale di $\sigma(-4.0)$, calcolato in fixed-point Q1.15.

Questa simulazione a livello di netlist post-implementation valida l'allineamento in fase dei segnali e l'assoluta assenza di glitch combinatori o transitori spuri al momento del campionamento esterno, certificando l'IP Core custom come un'architettura deterministica e stabile per elaborazioni in tempo reale a 250 MHz.

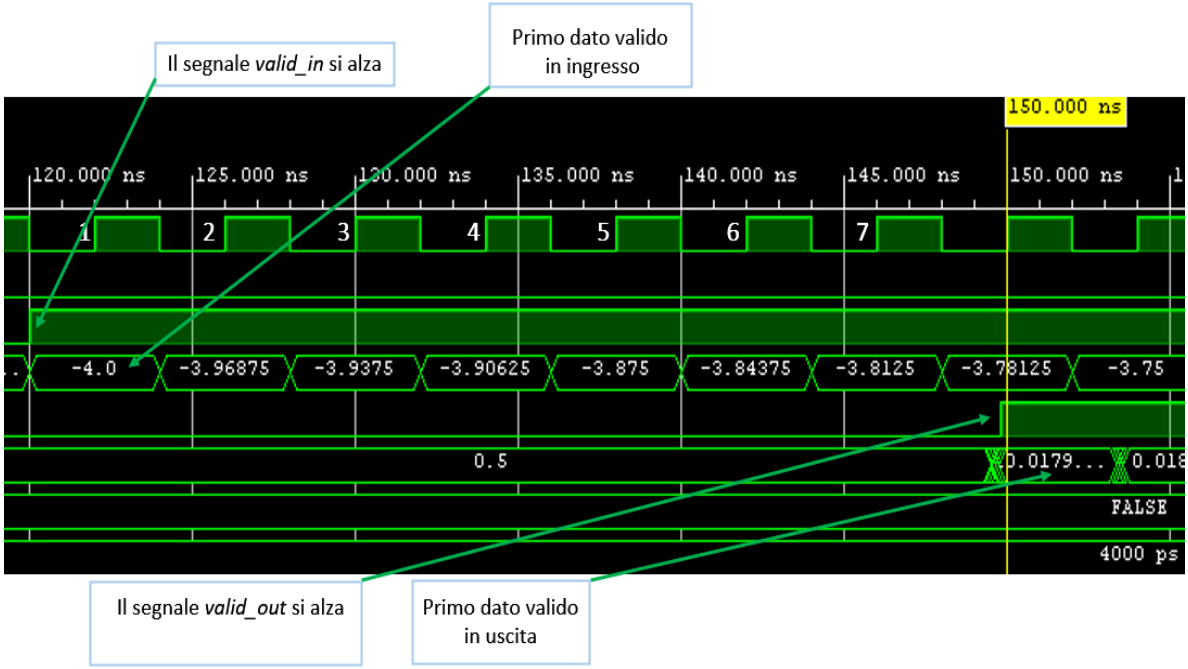


Figura 39 – Simulazione timing post-layout V3 (TB coerenza)

In Fig. 40 si può osservare la forma d'onda analogica.

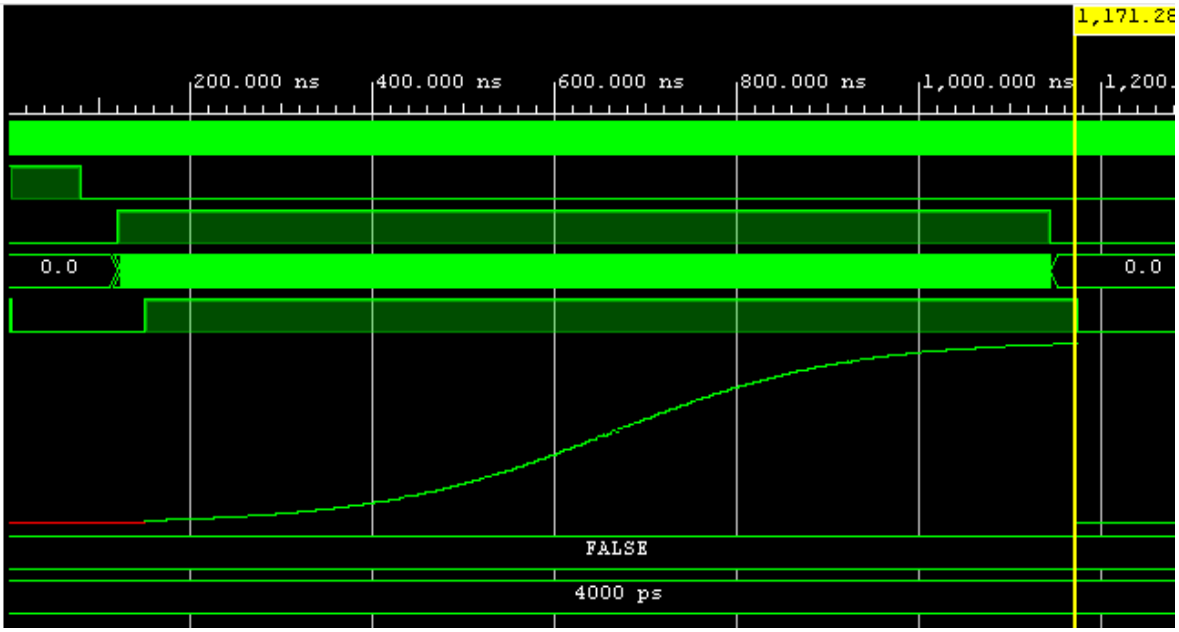
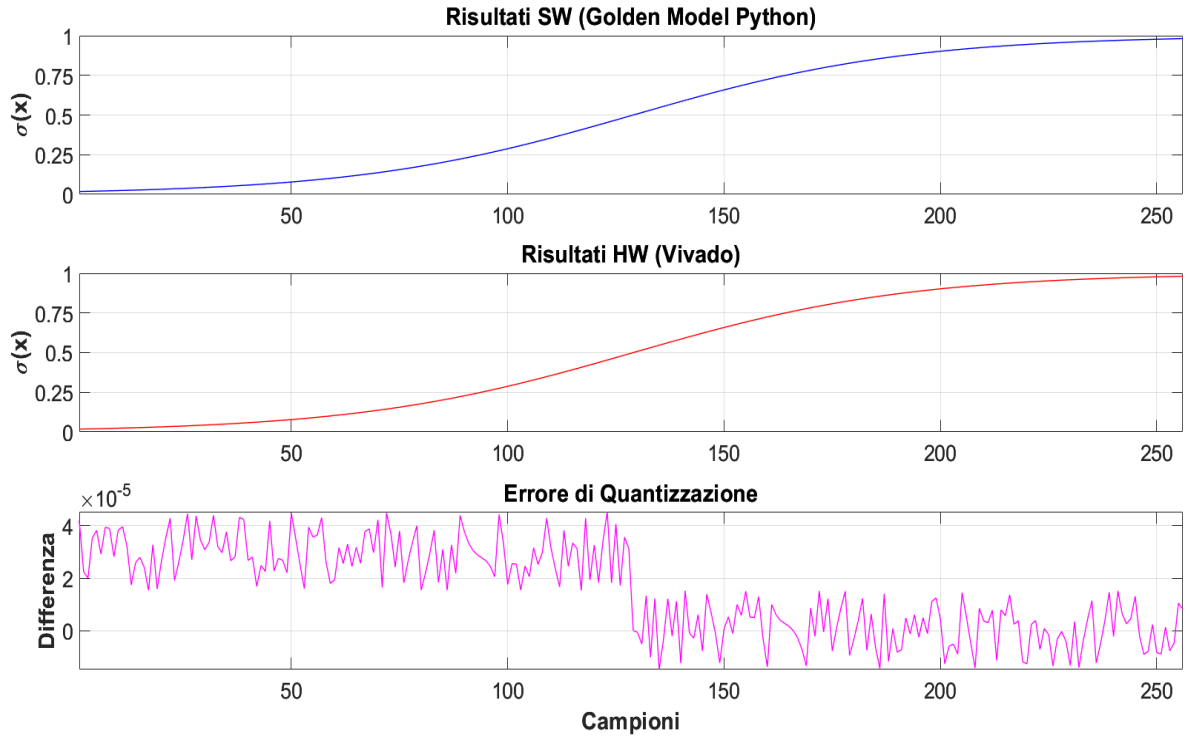


Figura 40 – Forma d'onda analogica simulazione V3 (TB coerenza)

La precisione dell'architettura V3 emerge dall'affinamento finale dei parametri di quantizzazione, i cui risultati di verifica in ambiente MATLAB sono documentati in Fig. 41. L'ottimizzazione del bit-width e il mantenimento dei bit di segno lungo l'intero data-path, hanno permesso di abbattere il MAE al valore infinitesimo di 0.0000185. Parallelamente, l'errore massimo registrato sull'intero dominio di test si attesta ad appena 0.0000452, riducendo lo scostamento locale di oltre due ordini di grandezza rispetto all'implementazione precedente.

Questo livello di accuratezza, dimostra che la funzione di attivazione sigmoide realizzata in hardware è virtualmente indistinguibile dal modello software teorico in floating-point: l'azzeramento del rumore di quantizzazione previene la degradazione del gradiente durante i calcoli di inferenza nei layer dei Transformers, garantendo le massime prestazioni predittive della rete senza gravare in maniera significativa sui consumi energetici o sulle risorse logiche della FPGA. I risultati della sintesi logica e del piazzamento post-layout dell'architettura V3, visualizzabili in Fig. 42, evidenziano un profilo di occupazione delle risorse estremamente ottimizzato, confermando l'efficienza della strategia di calcolo basata sulla simmetria.




```
MAE: 0.0000185288
Errore Massimo: 0.0000452953
```

Figura 41 – Confronto MATLAB risultati V3

Il core hardware richiede appena 39 Slice LUT (0.02% delle risorse totali del dispositivo). Tale dato dimostra che l'introduzione dei blocchi aritmetici per l'estrazione del valore assoluto e per la successiva inversione asintotica della curva non grava in alcun modo sulla logica programmabile della FPGA.

Sul fronte delle risorse sequenziali, l'ampio ricorso a tecniche di pipelining e l'integrazione dei buffer di isolamento d'interfaccia (Input/Output Buffering) comportano l'allocazione di 85 Slice Registers (0.03%). Questo minimo incremento di registri rappresenta il fulcro del bilanciamento temporale del modulo. Infine, il dato relativo alle memorie fisse evidenzia l'impiego di una frazione di BRAM pari a 0.5, corrispondente a un singolo blocco hardware nativo da 18 Kb. L'architettura V3 riesce così a memorizzare l'intera profondità di 1024 locazioni della BRAM unicamente per il semiasse positivo, dimezzando l'occupazione di memoria rispetto a un mapping lineare standard.

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)	Bonded IOB (285)	BUFGCTRL (32)
N Sigmoid_Accelerator_TOP_V3	39	85	26	39	0.5	36	1
	0.03%	0.03%	0.08%	0.03%	0.14%	12.63%	3.13%

Figura 42 – Report di utilizzo V3

La fattibilità fisica dell'architettura V3 alla frequenza operativa di target di 250 MHz è interamente validata dal Design Timing Summary post-layout (Fig. 43).

L'analisi dei vincoli di temporizzazione mostra un WNS pari a 0.453 ns, a fronte di un Total Negative Slack (TNS) rigorosamente nullo su tutti gli 88 endpoint attivi del circuito. Questo margine positivo attesta che, nonostante l'elevata densità di logica combinatoria introdotta all'interno dello stesso ciclo di clock per il calcolo della simmetria e del valore assoluto, il data-path completa la propagazione dei segnali in anticipo rispetto al tempo limite di campionamento, scongiurando fenomeni di metastabilità.

Setup

Worst Negative Slack (WNS):	0,453 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	88

Hold

Worst Hold Slack (WHS):	0,108 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	88

All user specified timing constraints are met.

Figura 43 – Report di timing V3

Parallelamente, la stabilità del sistema rispetto alle corse critiche (*race conditions*) è presidiata da uno Slack di Hold (WHS) ampiamente sicuro e pari a **0.108 ns**, supportato da un Total Hold Slack (THS) di 0.000 ns (Fig. 43). I risultati temporali post-routing confermano, in definitiva, la totale robustezza elettrica dell'IP Core, garantendo performance deterministiche ad alta frequenza. La caratterizzazione energetica dell'acceleratore V3 è stata eseguita tramite l'analisi di potenza post-implementazione alla frequenza di target di 250 MHz.

In Fig.44 è presentata l'analisi di potenza in modalità *vectorless*, mentre i risultati visibili nel report di Fig. 45 evidenziano un profilo di consumo estremamente contenuto, con una potenza totale stimata in 0.156 W e un incremento della temperatura di giunzione pari a soli 0.4°C rispetto alla temperatura ambiente di 25.0°C. L'aspetto di maggior rilievo risiede nella quota di potenza dinamica, che si attesta ad appena 0.025 W (16% del consumo complessivo). IL 73% della potenza dinamica (0.018 W) è dovuto esclusivamente alle strutture di Input/Output necessarie al testing del modulo stand-alone.

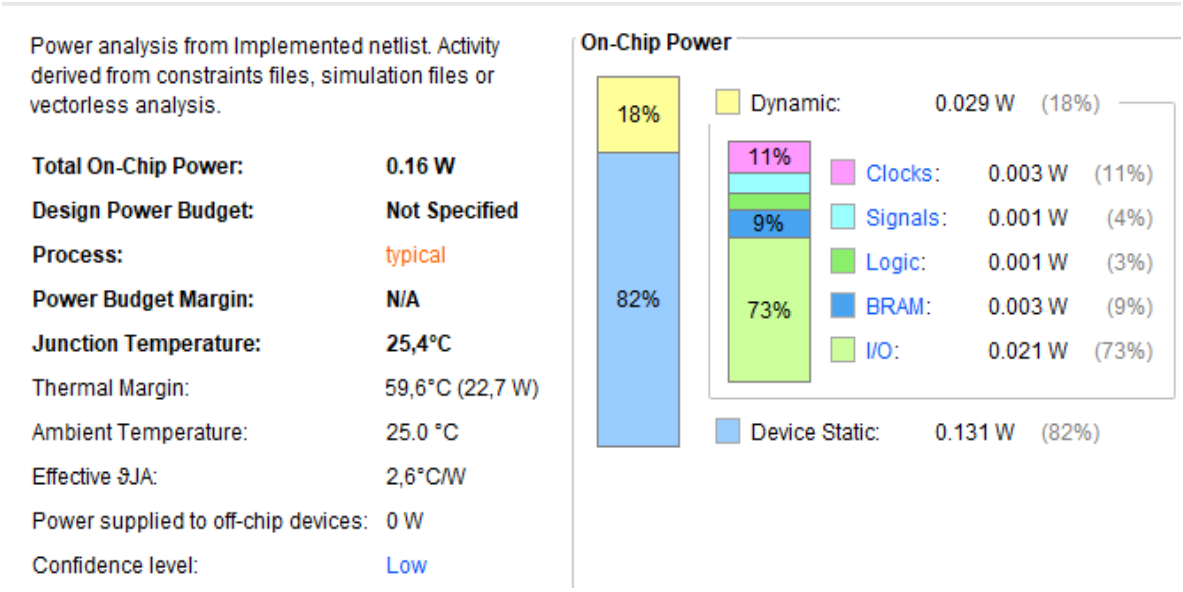


Figura 44 – Report di potenza V3 (vectorless)

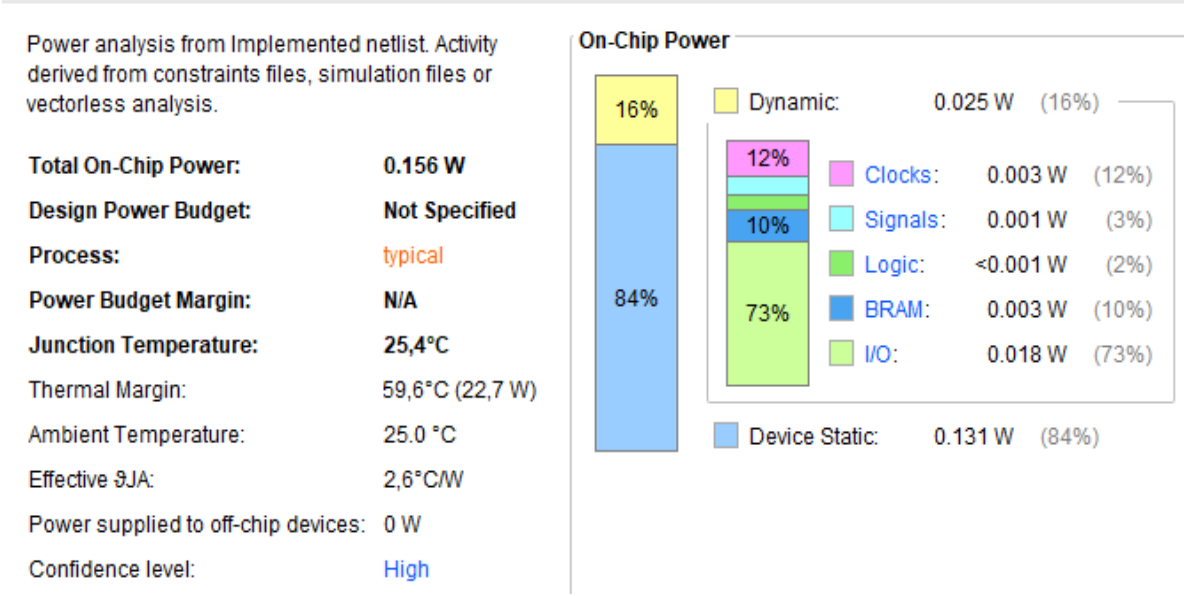


Figura 45 – Report di potenza V3 (SAIF relativo a TB coerenza)

Capitolo 4 – Confronto architetture

Questo capitolo è dedicato alla valutazione sperimentale e all'esplorazione dei compromessi architetturali delle soluzioni sviluppate. Il percorso di analisi si articola come segue:

- Analisi comparativa interna: In prima istanza, viene formalizzato un confronto diretto e standardizzato alla frequenza operativa di 250 MHz tra l'architettura baseline, la Versione 2 e l'ottimizzata Versione 3. Le tre soluzioni hardware vengono messe a sistema analizzando i vettori di merito fondamentali: l'occupazione dell'area di silicio (Slice LUT e Registers), i margini temporali di sicurezza (WNS) e la drastica evoluzione dell'accuratezza analitica (MAE).
- Strategie di parallelismo e throughput: Successivamente, l'orizzonte progettuale viene esteso dalla singola istanza computazionale alla scalabilità di sistema. Viene analizzata la replica spaziale del modulo V3 per il processing parallelo dei vettori di attivazione, con l'uso strategico delle memorie BRAM in modalità Dual-Port per raddoppiare la banda di sottomodulo senza duplicazione delle risorse fisiche di memorizzazione.
- Validazione su dataset reale: Al fine di testare la robustezza del core hardware in scenari operativi reali ed embedded, l'acceleratore viene investito da stimoli d'ingresso estratti direttamente da tensori di attivazione derivanti da una rete neurale pre-addestrata in ambiente PyTorch. Tale stress-test permette di validare sul campo la logica di pre-processing e clipping deterministico a fronte di distribuzioni statistiche reali e potenziali outlier.
- Benchmark eterogeneo (FPGA vs CPU): Il capitolo si conclude con una analisi comparativa che confronta l’architettura finale sviluppata con l’architettura general-purpose (CPU), soppesandone i rispettivi trade-off in termini di latenza deterministica, flessibilità software ed efficienza energetica, consolidando i vantaggi competitivi della logica programmabile nel panorama dei Transformers e del DL.

4.1 Analisi delle diverse implementazioni

Dall'analisi comparativa mostrata in Tabella II si effettuano delle considerazioni sulle 3 architetture viste precedentemente.

Tabella II – Confronto Architetture

Architettura	V1	V2	V3
Frequenza di Clock operativa (MHz)	250	250	250
Frequenza massima validata (MHz)	307	285	260
Profondità LUT	256	1024	1024
Risoluzione Campioni	256	1024	2048
Intervallo valori di $\sigma(x)$ mappati	$[-4; 4]$	$[-4; 4]$	$[-8; 8]$
Slice LUTs	43	16	39
Slice Registers (Flip-Flop)	53	65	85
BRAM (18Kb)	0	1	1
Throughput per ciclo di clock	1	1	1
Throughput reale (operazioni/s)	250 M	250 M	250 M
Profondità Pipeline	5	6	7
Latenza iniziale (ns)	20	24	28
WNS (ns)	1,394	0,773	0,453
Potenza Totale in TB coerenza (mW)	144	148	156
Errore di quantizzazione medio (MAE)	$\simeq 2{,}39 \cdot 10^{-3}$	$\simeq 5 \cdot 10^{-4}$	$\simeq 1{,}85 \cdot 10^{-5}$
Errore massimo assoluto	$\simeq 1{,}85 \cdot 10^{-2}$	$\simeq 1{,}25 \cdot 10^{-3}$	$\simeq 4{,}52 \cdot 10^{-5}$
Protezione Extra-Range	No	Moderata	Totale

4.2 Parallelismo

Per superare i limiti di throughput imposti da un'elaborazione seriale, il design è stato esteso verso un paradigma di calcolo parallelo. La strategia adottata prevede l'istanziazione di quattro core indipendenti all'interno di un nuovo Top module di sistema (*Sigmoid_MultiCore_Top.vhd*), riportato in Appendice B.16. Questo approccio sfrutta costrutti VHDL di tipo *generate* per garantire una scalabilità dinamica dell'architettura. Tale estensione verrà definita versione “V3.1”.

Il risultato è un acceleratore ad alta densità in cui ogni istanza opera in modo deterministico, condividendo il segnale di controllo di ingresso e fornendo un output aggregato su un bus dati esteso, trasformando di fatto un'unità di calcolo isolata in un sistema di elaborazione vettoriale.

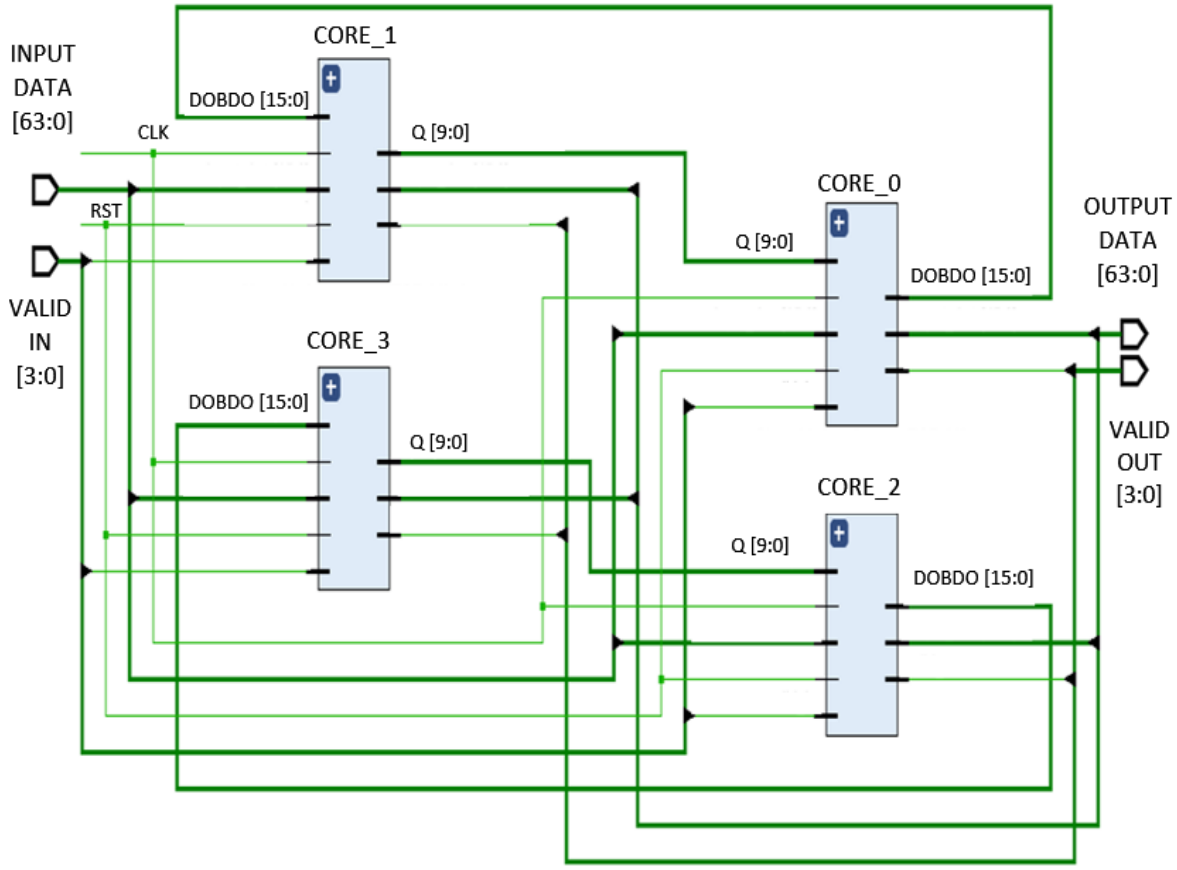


Figura 46 – Schematico post-layout architettura parallela V3.1 (4 moduli)

Nell'analisi comparativa tra il core singolo e l'architettura parallela, si è osservato un passaggio da una frequenza operativa massima di 250 MHz a 200 MHz. Tale variazione è riconducibile all'aumentato carico sulle risorse di routing dell'FPGA e alla maggiore complessità nel bilanciamento dei segnali di controllo su un'area occupata più estesa. La validazione dell'architettura parallela avviene mediante l'analisi di 4 moduli. In Fig. 46 viene illustrato lo schema RTL dell'architettura con parallelismo 4.

Per validare il comportamento dell'architettura parallela in uno scenario ad alto throughput, è stato sviluppato un testbench interamente parametrico, riportato in Appendice B.17, progettato per stressare il sistema con flussi di dati eterogenei. A differenza di una verifica statica ancorata a un numero fisso di istanze, l'ambiente di simulazione è stato ingegnerizzato per essere completamente scalabile tramite costanti di configurazione (*NUM_CORES* e *DATA_WIDTH*), consentendo di monitorare simultaneamente e in modo dinamico l'integrità dei segnali su un numero arbitrario di rami di elaborazione paralleli (impostato a 64 core nella configurazione di validazione corrente).

La logica di generazione degli stimoli si appoggia alle librerie standard STD.TEXTIO, simulando un carico di lavoro reale mediante la lettura automatizzata da un file di memoria esterno (*stimoli_input_baseline_p4.mem*). I dati vengono deserializzati e mappati dinamicamente sul bus vettoriale di ingresso (*input_data*), permettendo di osservare la propagazione del vettore dei segnali di abilitazione *valid_in* e la conseguente risposta temporale di tutti i core istanziati. Tale setup ha permesso di isolare e analizzare le latenze introdotte dal routing fisico, fornendo evidenza della corretta sincronizzazione della pipeline vettoriale.

La Fig. 47 illustra la simulazione temporale post-implementazione, dove è possibile apprezzare la stabilità dei campionamenti e l'allineamento deterministico dei risultati in uscita con processing parallelo di dati vettorializzati su 4 canali.

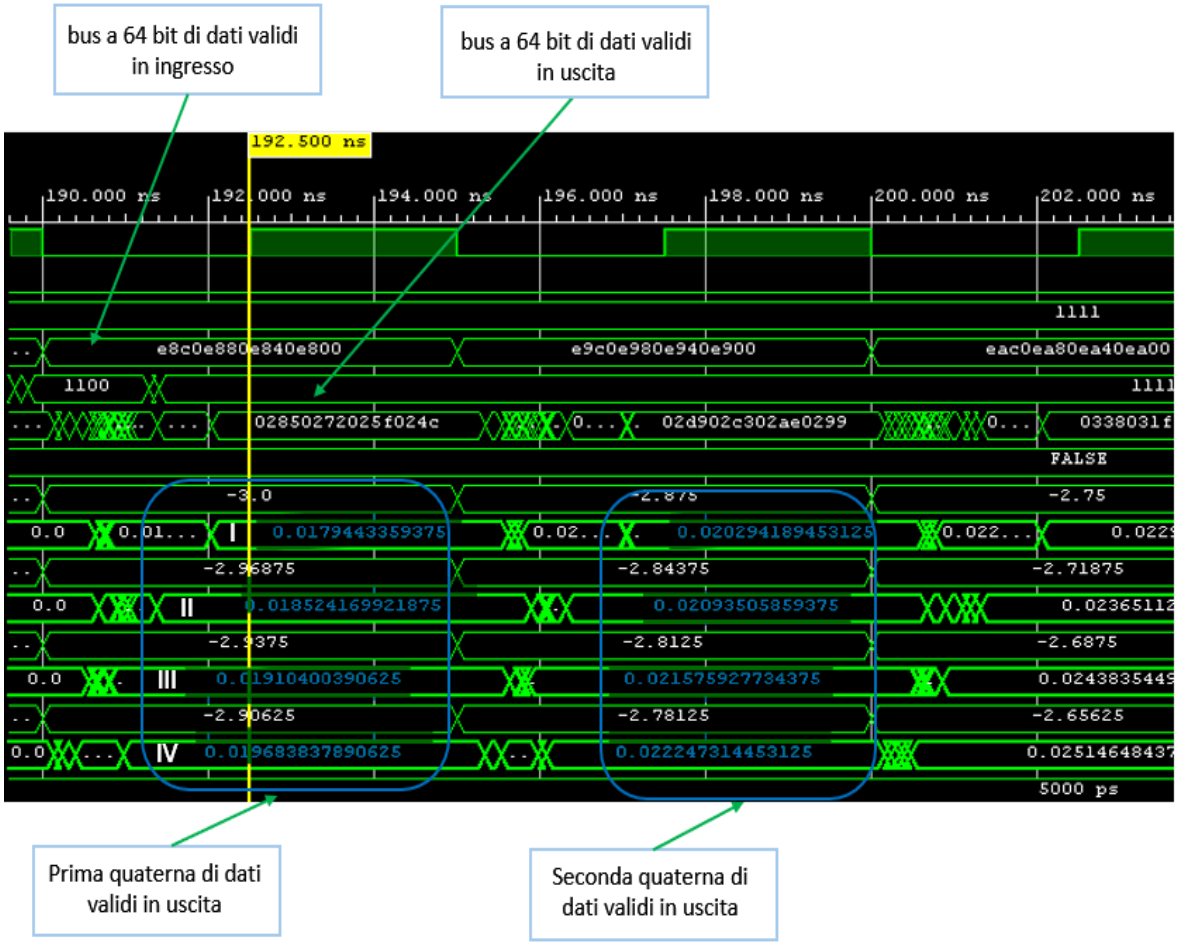


Figura 47 – Simulazione timing post-layout V3.1 (4 moduli)

Per garantire la continuità metodologica, l'architettura parallela è stata sottoposta ai medesimi stimoli di test utilizzati per la validazione delle versioni a singolo modulo. È stato utilizzato il set di 256 campioni di ingresso della versione baseline per coerenza, con un range compreso tra -4 e +3.96875.

La Fig. 48 illustra la ricostruzione della curva sigmoidea tramite l'analisi analogica delle uscite dei quattro core. Le incertezze grafiche visibili nelle transizioni (*skew*) sono una rappresentazione in dominio analogico dei micro-ritardi di propagazione dovuti al routing tra i diversi canali paralleli.

Tuttavia, il segnale di *valid_out* funge da filtro di campionamento sincrono, garantendo che vengano acquisiti esclusivamente i valori stabili e corretti al termine della pipeline di elaborazione, confermando la piena equivalenza funzionale rispetto al modello di riferimento.

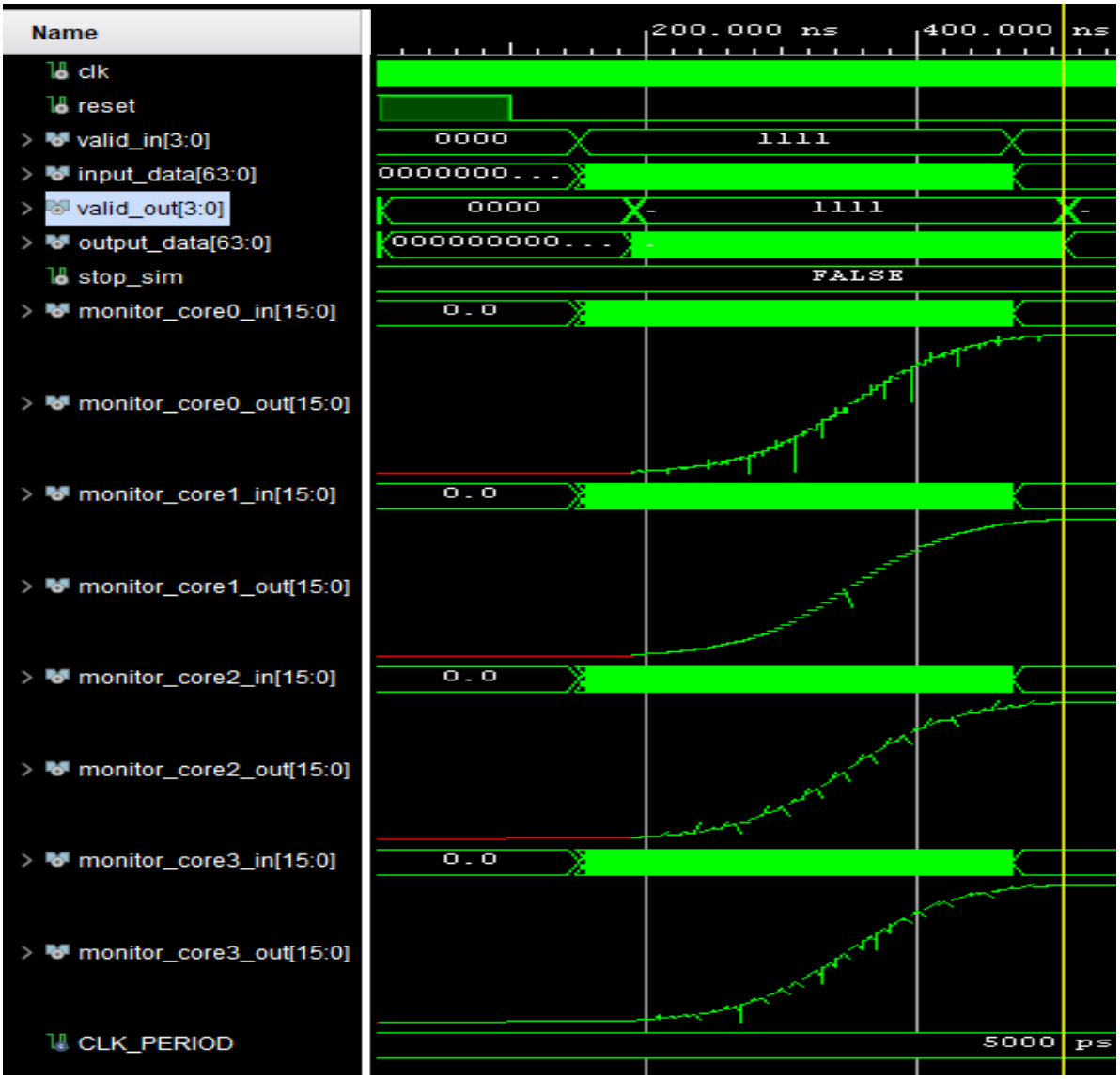


Figura 48 – Forme d'onda analogiche in simulazione V3.1 (4 moduli)

L'analisi dell'occupazione delle risorse, visibile in Fig. 49, conferma l'elevata efficienza e la modularità dell'architettura parallela proposta. Il report evidenzia una distribuzione simmetrica del carico logico tra le quattro istanze, con un'occupazione complessiva delle risorse estremamente contenuta. Un' ottimizzazione di rilievo riguarda l'architettura della memoria: l'intero set di valori della funzione sigmoide, necessario per il funzionamento di tutti e quattro i core, è stato compattato in soli 2 blocchi BRAM da 18Kb (1 blocco da 36Kb diviso in 2). Tale scelta di memory mapping è particolarmente efficace in quanto centralizzare la LUT dei valori della sigmoide in una memoria True Dual-Port condivisa permette di servire simultaneamente più unità di calcolo.

Questo approccio non solo dimezza l'occupazione di risorse BRAM sul chip, ma garantisce un accesso deterministico e a latenza minima per ciascun core, evitando la duplicazione ridondante di dati e minimizzando la congestione del routing locale.

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)	Bonded IOB (285)	BUFGCTRL (32)
Sigmoid_MultiCore_Top	122	340	101	122	1	138	1
CORE_0	31	85	30	31	0.5	0	0
CORE_1	31	85	27	31	0	0	0
CORE_2	31	85	32	31	0.5	0	0
CORE_3	31	85	29	31	0	0	0

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)	Bonded IOB (285)	BUFGCTRL (32)
Sigmoid_MultiCore_Top	0.09%	0.13%	0.30%	0.09%	0.27%	48.42%	3.13%
CORE_0	0.02%	0.03%	0.09%	0.02%	0.14%	0.00%	0.00%
CORE_1	0.02%	0.03%	0.08%	0.02%	0.00%	0.00%	0.00%
CORE_2	0.02%	0.03%	0.10%	0.02%	0.14%	0.00%	0.00%
CORE_3	0.02%	0.03%	0.09%	0.02%	0.00%	0.00%	0.00%

Figura 49 – Report di utilizzo V3.1 (4 moduli)

L'analisi temporale, i cui risultati sono riportati in Fig. 50, conferma il totale rispetto dei vincoli temporali definiti con un numero di *Failing Endpoints* pari a zero.

La solidità del sistema è certificata da:

- **WNS pari a 0,349 ns:** Questo valore positivo indica che il segnale ha ampiamente rispettato il vincolo di Setup entro il periodo di clock impostato a 5 ns (200 MHz). Il design si presenta pertanto altamente robusto e il percorso critico gode di un margine di tolleranza significativo. Ciò garantisce che il sistema possa operare in condizioni di temperatura e tensione variabili senza incorrere in errori di campionamento.
- **WHS pari a 0,105 ns:** Anche il vincolo di Hold risulta pienamente soddisfatto. Questo margine positivo assicura che il dato, una volta catturato da un registro, rimanga stabile per un tempo sufficiente a evitare la corruzione logica, confermando l'assenza di rischi di *race conditions* nel circuito.

Il *Total Negative Slack* (TNS) e il *Total Hold Slack* (THS) pari a zero confermano che l'architettura non presenta alcun punto di debolezza latente nell'intero set di 416 endpoint analizzati. Tali metriche attestano la validità della pipeline progettata e la corretta gestione del clock distribuito tra i quattro core paralleli, garantendo un'operatività deterministica e affidabile

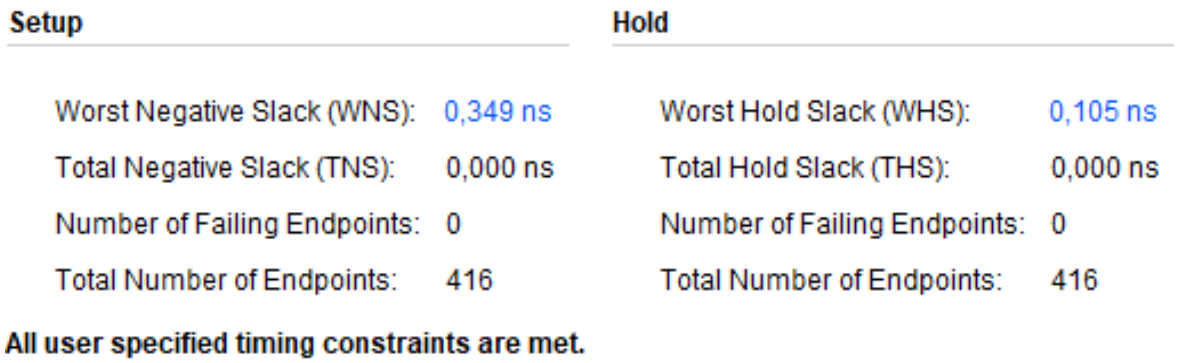


Figura 50 – Report di timing V3.1 (4 moduli)

Il consumo di potenza totale appare estremamente contenuto, pari a 0,269 W in analisi *vectorless* (Fig. 51), e a 0,188 W. post-SAIF (Fig. 52).

L'analisi evidenzia la seguente distribuzione del carico energetico:

- Potenza Dinamica (30%): Il consumo attivo, legato alla commutazione dei segnali e dei clock, si attesta a 0,057 W. L'impatto totale di *Clocks*, *Signals* e *BRAM* sulla potenza dinamica è solo del 15%, confermando l'efficacia delle tecniche di gating e della struttura di memoria scelta.
- Potenza Statica (70%): Il consumo di base (leakage) è di 0,131 W, un valore standard per la tecnologia utilizzata, che domina il bilancio energetico totale in un design a così bassa densità di risorse.
- Margine Termico: Il sistema opera a una temperatura di giunzione di 25,5°C, con un margine termico molto ampio. Ciò garantisce la massima affidabilità operativa e la totale assenza di necessità di sistemi di raffreddamento attivi.

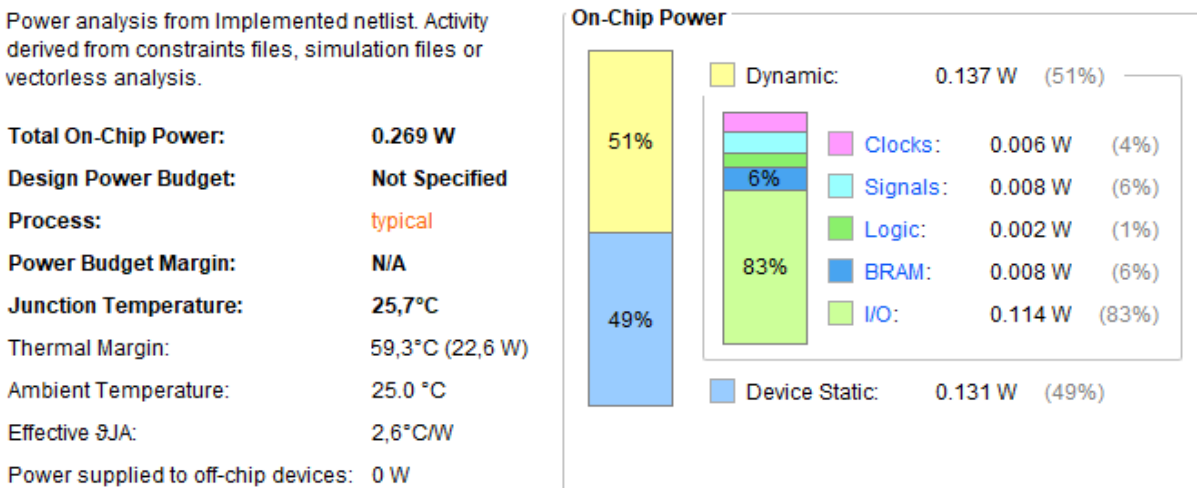


Figura 51 – Report di potenza V3.1 (4 moduli, vectorless)

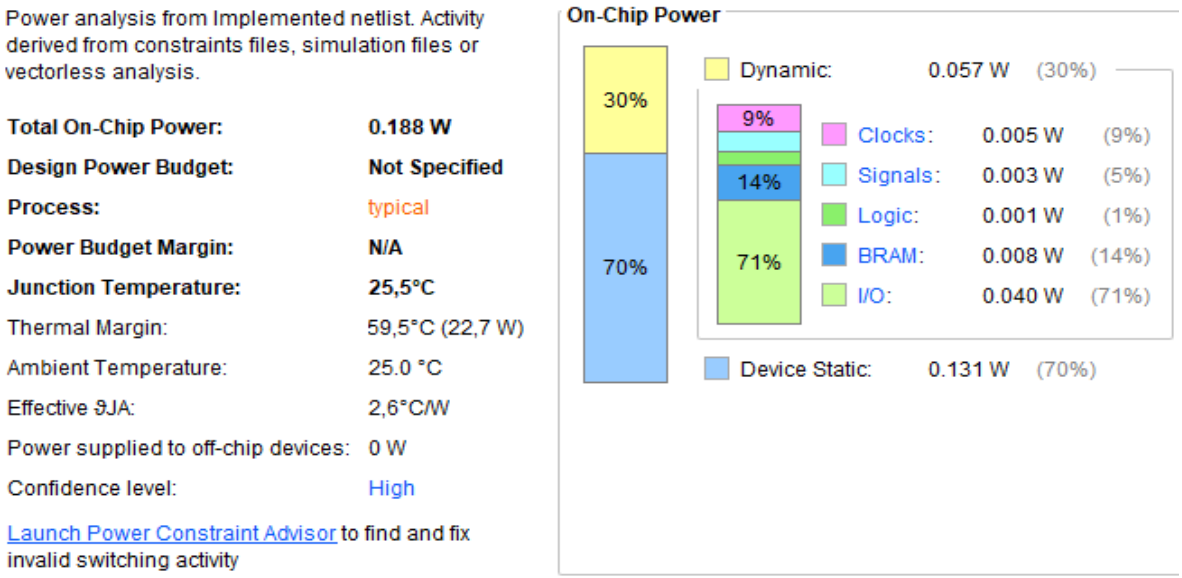


Figura 52 – Report di potenza V3.1 (4 moduli, SAIF relativo a TB coerenza)

La Tabella III fornisce un quadro comparativo tra la versione single-core (V3), e l'architettura vettoriale V3.1) con grado di parallelismo 2 e 4. Si evidenzia come, a fronte di un incremento lineare delle risorse (LUT e BRAM), il throughput reale passi da 200 a 800 milioni di operazioni al secondo.

Tabella III – Modulo singolo vs parallelismo 2 e 4

Architettura	V3 (Modulo Singolo)	V3.1 (Parallelismo 2)	V3.1 (Parallelismo 4)
Frequenza di Clock (MHz)	200	200	200
Slice LUTs	39	61	122
Slice Registers / Flip-Flop	85	170	340
BRAM 36 Kb	0.5	0.5 (True Dual Port)	1 (True Dual Port)
Throughput per ciclo di clock	1	2	4
Throughput (operazioni/s)	200 M	400 M	800 M
Profondità Pipeline	7	7	7
Latenza iniziale (ns)	35	35	35
WNS (ns)	0,874	0,561	0,349
Potenza statica (mW)	131	131	131
Potenza dinamica in TB coerenza (mW)	25	36	57
Potenza totale in TB coerenza (mW)	156	167	188
Errore di quantizzazione medio (MAE)	$\simeq 1{,}85 \cdot 10^{-5}$	$\simeq 1{,}85 \cdot 10^{-5}$	$\simeq 1{,}85 \cdot 10^{-5}$

4.3 Comportamento su dataset reale

Per la validazione dell'architettura hardware proposta, si è proceduto all'acquisizione dei dati generati dal modello software di riferimento. Data l'elevata dimensionalità dei tensori prodotti dal modello software, sviluppato in ambiente PyTorch/TensorFlow [23], è stato estratto un subset significativo di dati ai fini della simulazione RTL su Vivado. In Fig. 53 è riassunta la procedura.

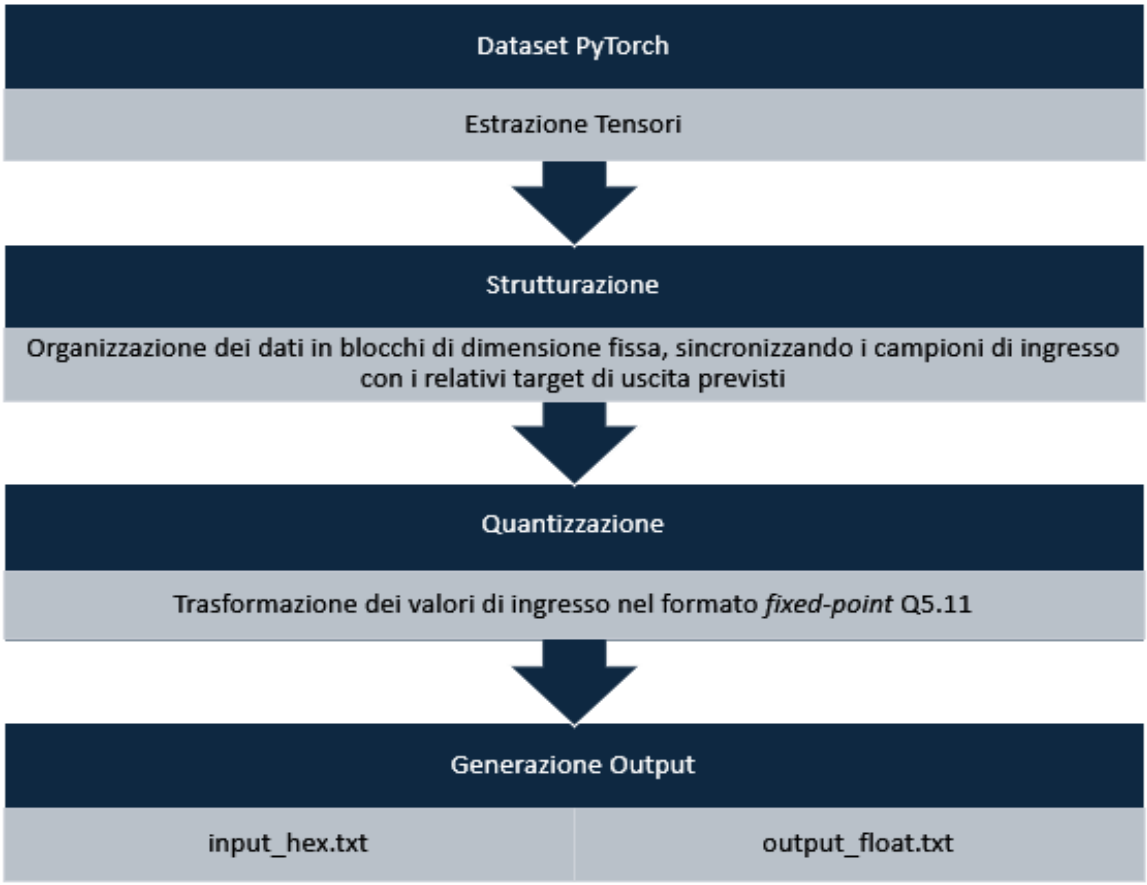
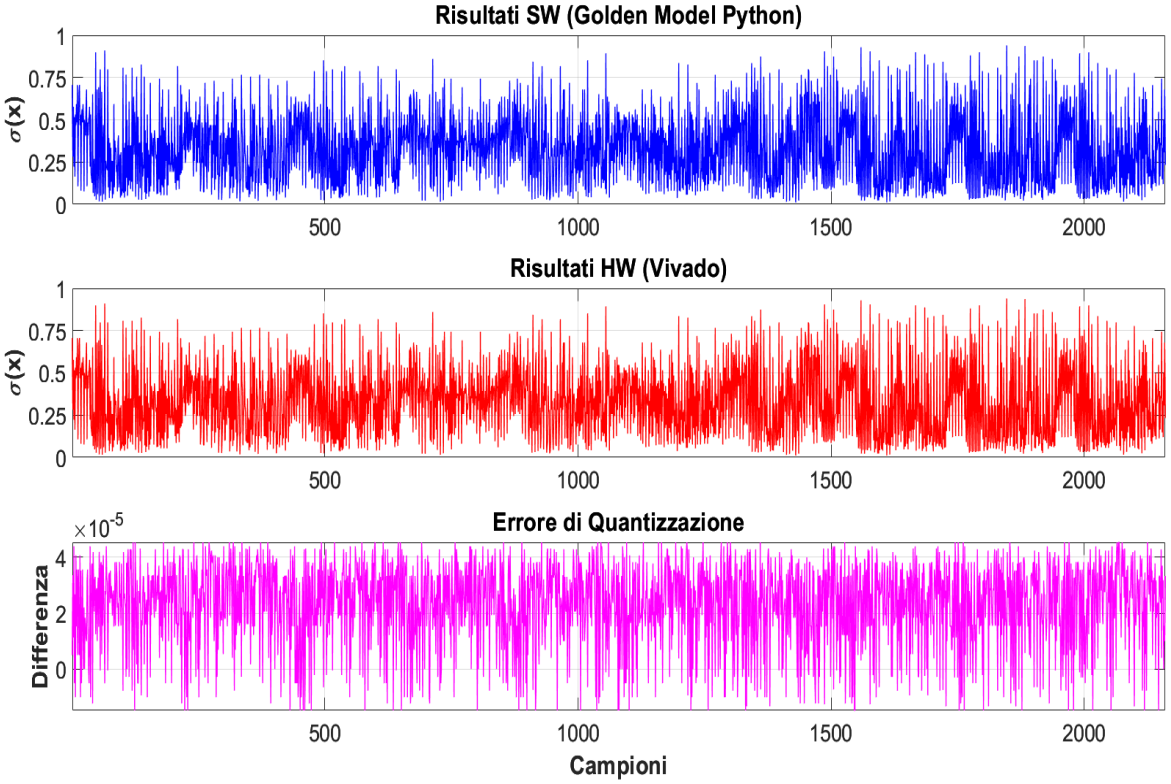


Figura 53 – Conversione dataset reale

Lo script *clean_dataset.py,* riportato in Appendice A.8, quantizza gli input nel formato fixed-point Q5.11 per rispecchiare fedelmente le condizioni operative dell'hardware, mentre mantiene i valori di riferimento in uscita nel formato floating-point originale. Questa scelta permette di quantificare l'errore di quantizzazione introdotto dall'architettura hardware rispetto al modello matematico ideale, fornendo una metrica oggettiva della perdita di precisione dovuta alla rappresentazione a 16 bit.

Il confronto tra l'output hardware e il riferimento software (in float) permette dunque di validare non solo la correttezza logica del calcolo, ma anche la robustezza numerica dell'intera catena di elaborazione. In seguito sono stati presi i primi 2160 campioni del dataset reale come stimolo per il testbench, inseriti nel file *stimoli_real.mem,* e i corrispettivi risultati calcolati via sw nel file *2160_outputs_float.txt* per il confronto dell'errore su MATLAB (Fig. 54). Il testbench è stato adeguato per questa architettura e riportato integralmente in Appendice B.18. In Fig. 55 si può osservare la simulazione temporale post-layout adoperando 4 moduli paralleli, mentre in relativo report di potenza post-SAIF è visibile in Fig. 56.




```
MAE: 0.0000249010
Errore Massimo: 0.0000452973
```

Figura 54 – Confronto MATLAB risultati V3.1 (4 moduli) con dataset reale

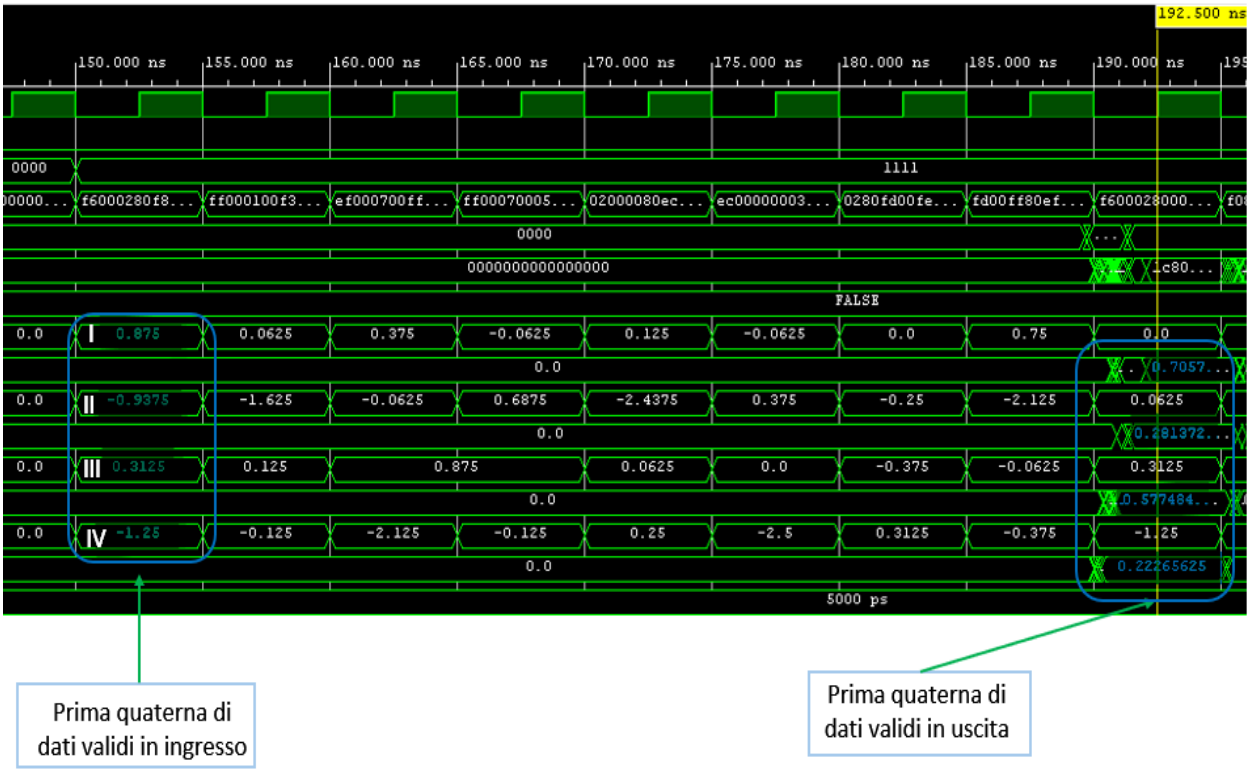


Figura 55 – Simulazione timing post-layout V3.1 (4 moduli) con dataset reale

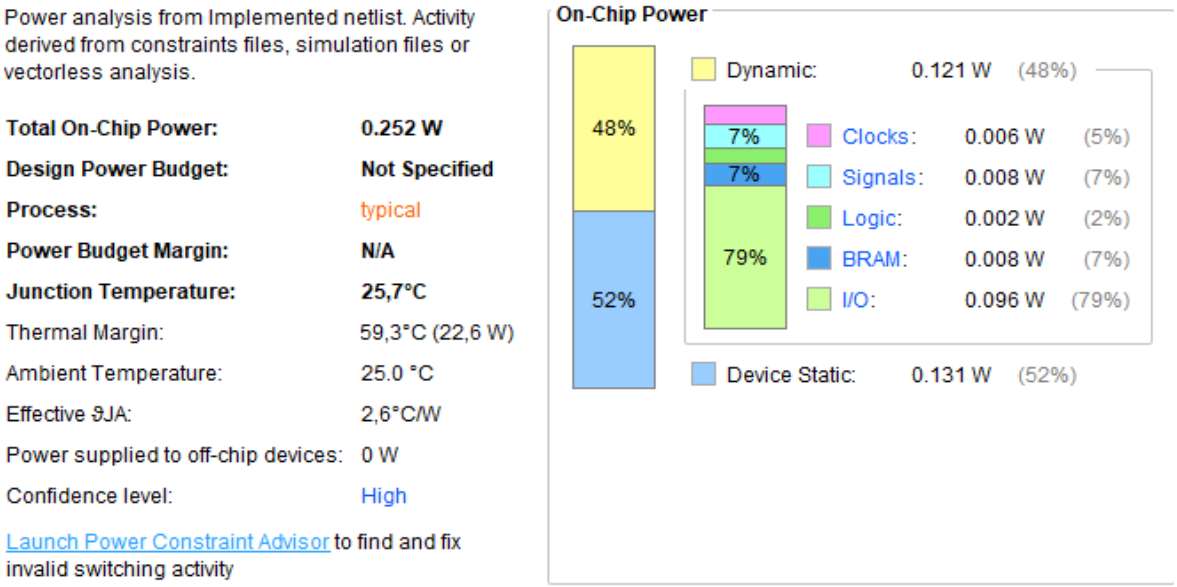


Figura 56 – Report di potenza V3.1 (4 moduli, SAIF relativo a TB dataset reale)

4.4 Scalabilità

L'architettura proposta, implementata mediante il costrutto VHDL *generate*, presenta una scalabilità strutturale nativa che consente di istanziare un numero arbitrario di core di calcolo all'interno del medesimo ecosistema logico. Per validare le prestazioni del sistema su larga scala, testando configurazioni estese da 8 fino a 1024 unità, si è adottata una metodologia di sintesi ***Out-Of-Context*** **(OOC)**.

Tale approccio è risultato determinante per isolare il modulo Top level dai vincoli di pin-assignment e dai ritardi indotti dall'interfaccia fisica con i dispositivi I/O del target FPGA. Focalizzando il processo di sintesi esclusivamente sulla logica computazionale e sui percorsi critici, è stato possibile isolare il comportamento intrinseco in scenari ad altissima densità logica. Ciò ha permesso di estrapolare metriche attendibili in merito all'utilizzo delle risorse (LUT, BRAM, Distributed RAM) e all'evoluzione del (WNS), delineando con precisione i limiti operativi dell'architettura.

Attraverso lo stress-test incrementale fino a 1024 istanze, è stato possibile identificare non solo il punto di ottimo, ma anche la soglia critica di saturazione in cui la congestione del routing globale diventa il fattore limitante le prestazioni. I risultati documentati nei report seguenti offrono una visione dettagliata del trade-off tra densità computazionale, consumo energetico e stabilità temporale, confermando la robustezza dell'IP Core anche in configurazioni massivamente parallele che eccederebbero le capacità di un design integrato convenzionale. Nelle figure che seguono sono mostrate le analisi di occupazione risorse, timing e potenza relativi ai diversi valori di parallelismo impiegati.

Al fine di validare l’efficienza dell’architettura proposta, è stata condotta una procedura di sintesi e implementazione incrementale, variando il numero di istanze del core di calcolo da 8 fino a 512 unità. Questa analisi sistematica permette di osservare la linearità del consumo di risorse e la stabilità temporale del sistema all'aumentare della densità logica.

Nelle Fig. 57 – 66 sono riportati i report di utilizzo, timing e potenza relativi a tali configurazioni, che testimoniano il mantenimento dei vincoli temporali (WNS positivo) e l'efficienza nell'allocazione delle risorse su target FPGA.

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
Sigmoid_MultiCore_Top	31	85	18	31	0.5
	0.02%	0.03%	0.05%	0.02%	0.14%

Setup

Worst Negative Slack (WNS):	0,874 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	104

Hold

Worst Hold Slack (WHS):	0,143 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	104

All user specified timing constraints are met.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.128 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**25,3°C**
Thermal Margin:	59,7°C (22,7 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	2,6°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	High

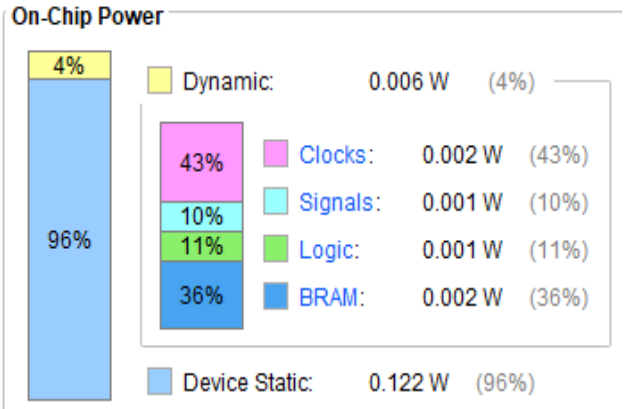


Figura 57 – Analisi OOC modulo singolo

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
Sigmoid_MultiCore_Top	61	170	39	61	0.5
	0.05%	0.06%	0.12%	0.05%	0.14%

Setup

Worst Negative Slack (WNS):	0,674 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	208

Hold

Worst Hold Slack (WHS):	0,145 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	208

All user specified timing constraints are met.

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	**0.133 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**25,3°C**
Thermal Margin:	59,7°C (22,7 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	2,6°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	High

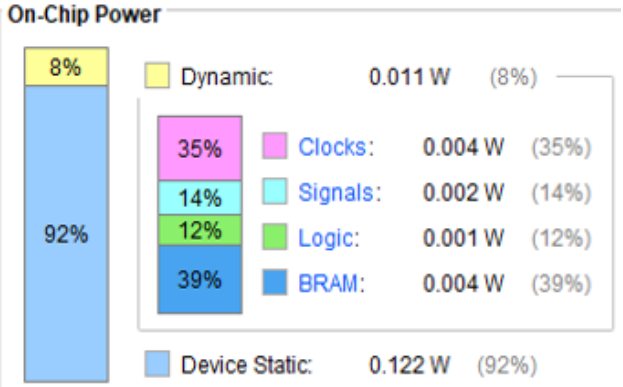


Figura 58 – Analisi OOC 2 moduli

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
Sigmoid_MultiCore_Top	122	340	90	122	1

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
Sigmoid_MultiCore_Top	0.09%	0.13%	0.27%	0.09%	0.27%

Setup

Worst Negative Slack (WNS):	0,246 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	416

Hold

Worst Hold Slack (WHS):	0,129 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	416

All user specified timing constraints are met.

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power:	**0.144 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**25,4°C**
Thermal Margin:	59,6°C (22,7 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	2,6°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	High

On-Chip Power

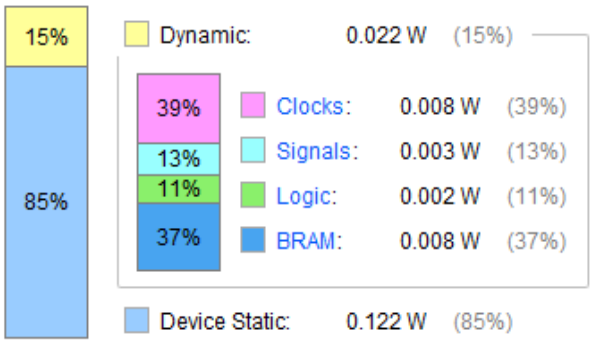


Figura 59 – Analisi OOC 4 moduli

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
Sigmoid_MultiCore_Top	244	680	139	244	2
	0.18%	0.25%	0.41%	0.18%	0.55%

Setup

Worst Negative Slack (WNS):	0,289 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	832

Hold

Worst Hold Slack (WHS):	0,098 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	832

All user specified timing constraints are met.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.166 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**25,4°C**
Thermal Margin:	59,6°C (22,7 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	2,6°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	High

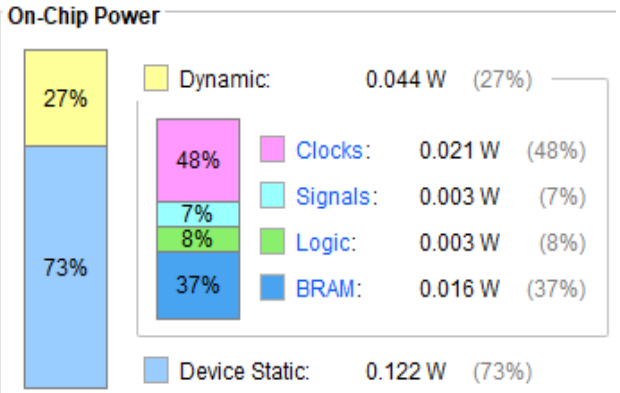


Figura 60 – Analisi OOC 8 moduli

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
Sigmoid_MultiCore_Top	488	1360	272	488	4

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
Sigmoid_MultiCore_Top	0.36%	0.51%	0.81%	0.36%	1.10%

Setup

Worst Negative Slack (WNS):	0,265 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	1664

Hold

Worst Hold Slack (WHS):	0,098 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	1664

All user specified timing constraints are met.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.19 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**25,5°C**
Thermal Margin:	59,5°C (22,7 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	2,6°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	High

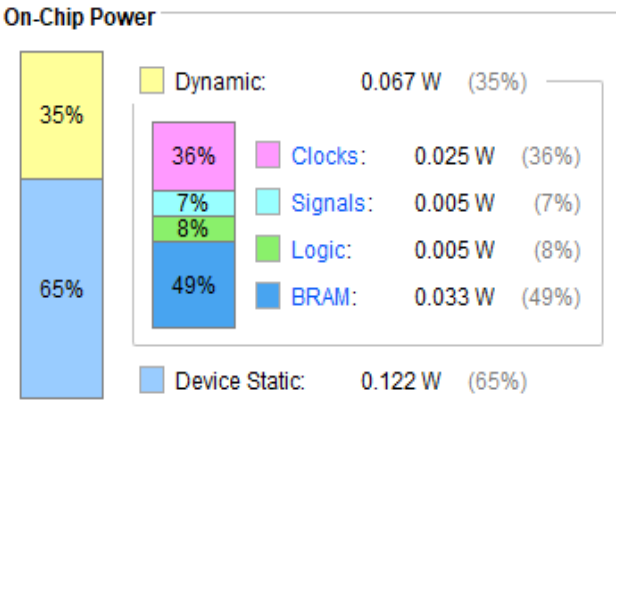


Figura 61 – Analisi OOC 16 moduli

L'analisi non si limita a verificare la scalabilità della struttura attuale, ma mira a delineare i confini di un'integrazione a livello di sistema (System-on-Chip). In quest'ottica di dimensionamento, la configurazione a 32 moduli paralleli viene individuata come lo *sweet spot* ingegneristico dell'architettura. Con un parallelismo del dato fissato a 16 bit, tale configurazione consente di saturare nativamente un bus di comunicazione AXI a 512 bit, garantendo il perfetto bilanciamento tra il throughput computazionale dell'acceleratore e la larghezza di banda della memoria senza introdurre colli di bottiglia o logiche di instradamento complesse.

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
> N Sigmoid_MultiCore_Top	976	2720	568	976	8

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
> N Sigmoid_MultiCore_Top	0.73%	1.01%	1.69%	0.73%	2.19%

Setup

Worst Negative Slack (WNS):	0,430 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	3328

Hold

Worst Hold Slack (WHS):	0,084 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	3328

All user specified timing constraints are met.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.238 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**25,6°C**
Thermal Margin:	59,4°C (22,6 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	2,6°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	High

Launch Power Constraint Advisor to find and fix invalid switching activity

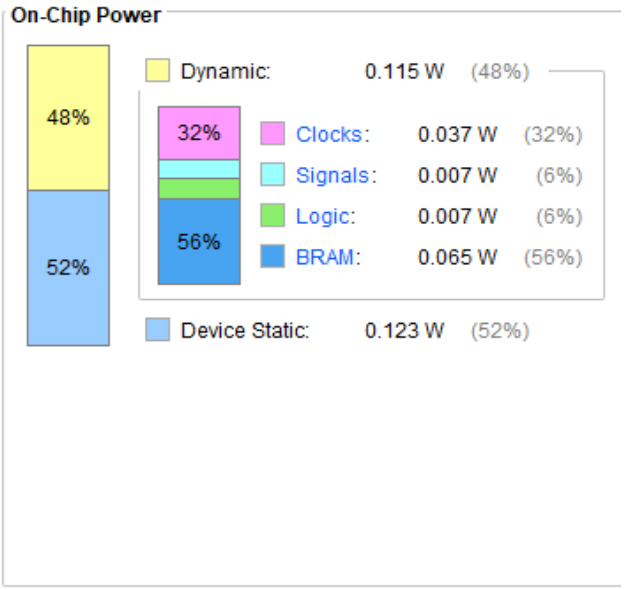


Figura 62 – Analisi OOC 32 moduli

Per scenari applicativi ad altissime prestazioni che richiedono il massimo parallelismo massivo, lo studio prevede la scalabilità del sistema fino a un limite massimo di 64 moduli paralleli, configurazione pensata per interfacciarsi con un bus AXI da 1024 bit e capitalizzare le ampie risorse di routing e di interconnessione offerte dalle architetture AMD Xilinx UltraScale+.

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
Sigmoid_MultiCore_Top	1953	5440	1162	1953	16

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
Sigmoid_MultiCore_Top	1.45%	2.02%	3.45%	1.45%	4.38%

Setup

Worst Negative Slack (WNS):	0,165 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	6656

Hold

Worst Hold Slack (WHS):	0,083 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	6656

All user specified timing constraints are met.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.333 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**25,9°C**
Thermal Margin:	59,1°C (22,5 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	2,6°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	High

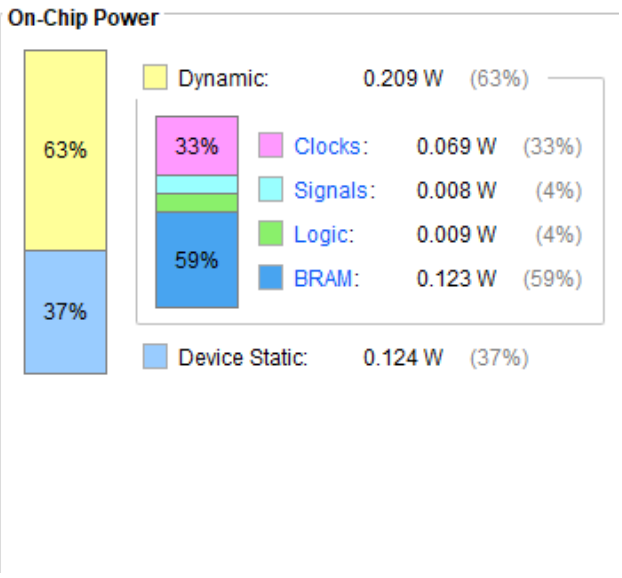


Figura 63 – Analisi OOC 64 moduli

Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (3365 0)	LUT as Logic (134600)	Block RAM Tile (365)
Sigmoid_MultiCore_Top	3907	10880	2279	3907	32
	2.90%	4.04%	6.77%	2.90%	8.77%

Setup

Worst Negative Slack (WNS):	0,252 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	13312

Hold

Worst Hold Slack (WHS):	0,085 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	13312

All user specified timing constraints are met.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**0.508 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**26,3°C**
Thermal Margin:	58,7°C (22,4 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	2,6°C/W
Power supplied to off-chip devices:	0 W
Confidence level:	High

Launch Power Constraint Advisor to find and fix invalid switching activity

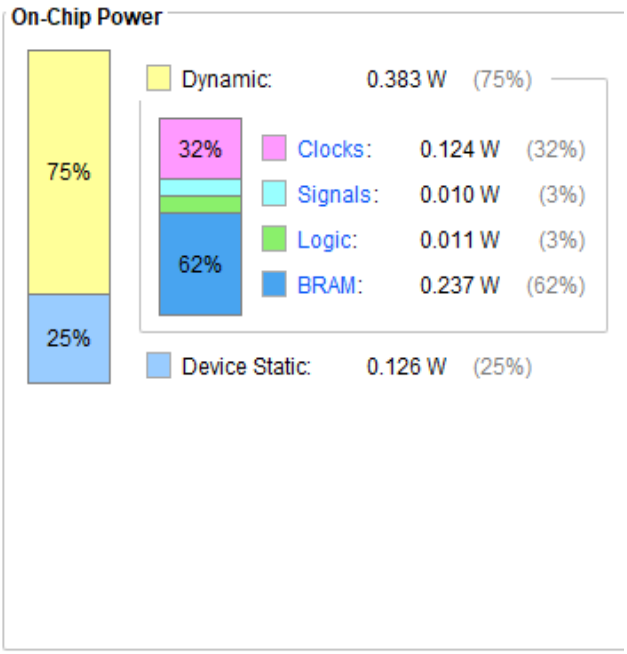


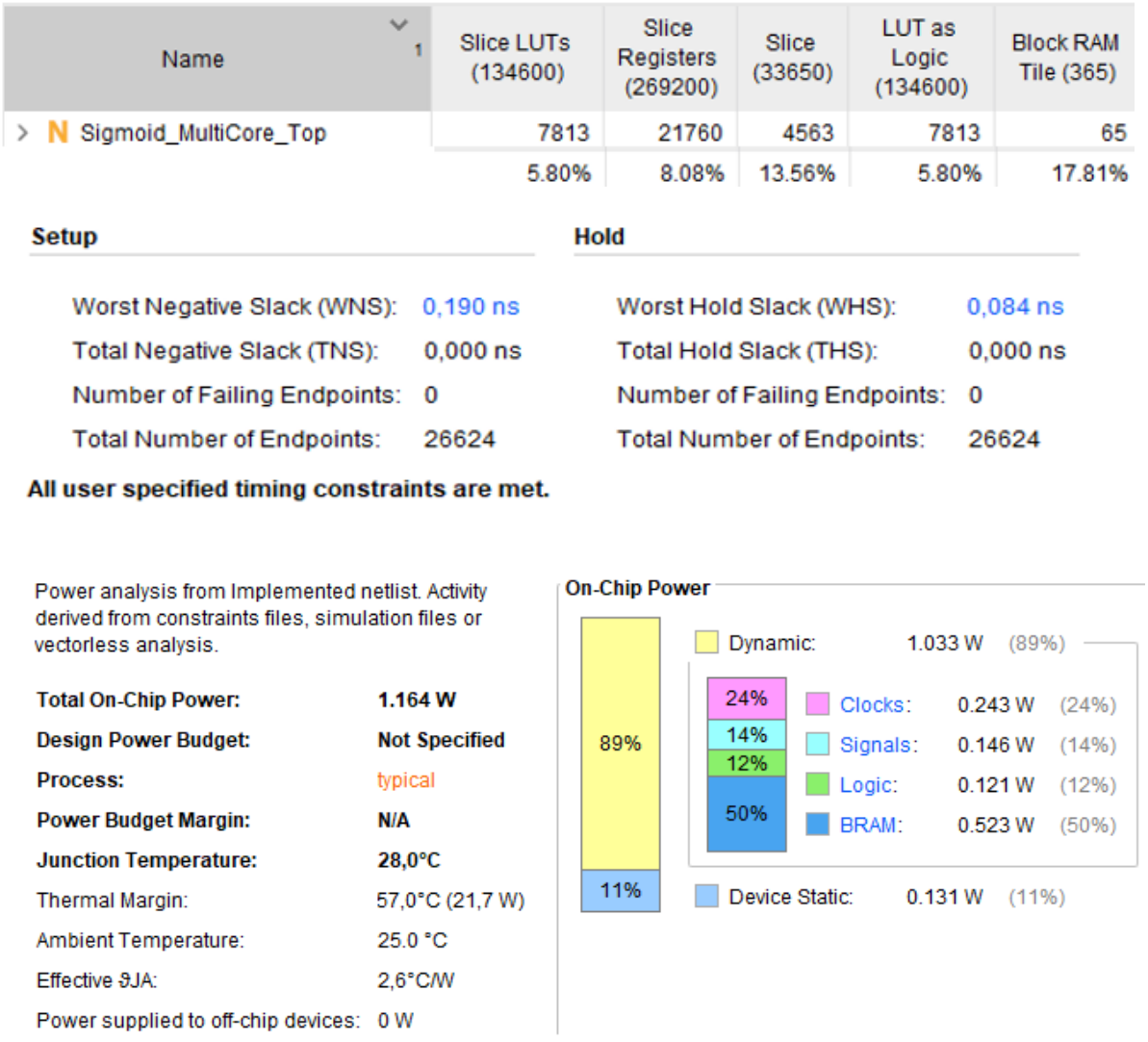
Figura 64 – Analisi OOC 128 moduli

Al di là delle configurazioni standard di sistema, l'analisi si spinge intenzionalmente verso scenari con configurazioni massive a 128, 256, 512 e 1024 unità parallele.

Questa progressione geometrica punta a esplorare i limiti fisici intrinseci dell'architettura e in termini di area e frequenza massima di clock. I risultati dimostrano che l'IP Core in sé non rappresenterà mai il collo di bottiglia del sistema.

La natura intrinsecamente parallela e la struttura pipelined della logica di calcolo garantiscono un throughput teorico tale da elaborare i dati a una velocità superiore rispetto a quella con cui una normale infrastruttura di memoria può fornirli. L'eventuale limite prestazionale viene quindi delocalizzato dall'acceleratore e spostato esclusivamente sulla massima larghezza di banda del bus di sistema o sulla congestione delle linee di routing dell'FPGA ad alta densità.

Si evidenzia come l'estensione del parallelismo massivo a 128 moduli sia un obiettivo condizionato e vincolato **esclusivamente alla transizione al formato a 8 bit** (Q4.4 in ingresso e Q1.7 in uscita). Mantenendo l'attuale configurazione a 16 bit, infatti, un bus dati AXI4-Stream ad alte prestazioni con ampiezza massima nativa di 1024 bit potrebbe ospitare un limite teorico di soli 64 flussi paralleli per singolo ciclo di clock.



Name	Slice LUTs (134600)	Slice Registers (269200)	Slice (33650)	LUT as Logic (134600)	Block RAM Tile (365)
Sigmoid_MultiCore_Top	7813	21760	4563	7813	65
	5.80%	8.08%	13.56%	5.80%	17.81%

Setup

Worst Negative Slack (WNS):	0,190 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	26624

Hold

Worst Hold Slack (WHS):	0,084 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	26624

All user specified timing constraints are met.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**1.164 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**28,0°C**
Thermal Margin:	57,0°C (21,7 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	2,6°C/W
Power supplied to off-chip devices:	0 W

On-Chip Power

Figura 65 – Analisi OOC 256 moduli

Di conseguenza, la quantizzazione a 8 bit diventa un **vincolo architetturale stringente** per consentire il packing di 128 elementi d'ingresso nello stesso fronte di clock, saturando l'intera banda disponibile e sbloccando il picco massimo di throughput del sistema.

Name	Slice LUTs	Slice Registers	Slice (3365	LUT as Logic	Block RAM Tile (365)
> N Sigmoid_MultiCore_Top	15629	43520	9027	15629	130

Name	Slice LUTs	Slice Registers	Slice (3365	LUT as Logic	Block RAM Tile (365)
> N Sigmoid_MultiCore_Top	11.61%	16.17%	26.83%	11.61%	35.62%

Setup

Worst Negative Slack (WNS):	0,141 ns
Total Negative Slack (TNS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	53248

Hold

Worst Hold Slack (WHS):	0,083 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	53248

All user specified timing constraints are met.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**2.144 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**30,5°C**
Thermal Margin:	54,5°C (20,7 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	2,6°C/W
Power supplied to off-chip devices:	0 W

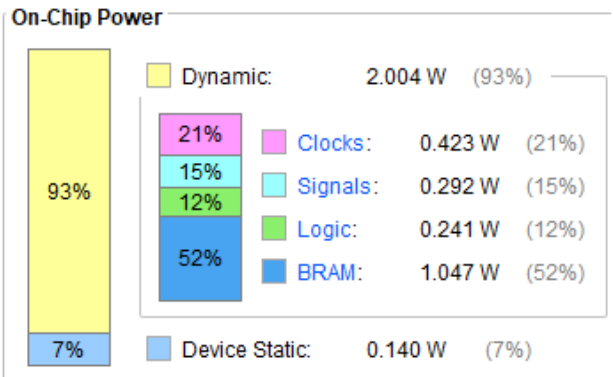


Figura 66 – Analisi OOC 512 moduli

Le architetture a 256 e 512 moduli rispondono a una finalità prettamente speculativa di *Design Space Exploration* (DSE), volta a indagare la scalabilità della struttura logica e a mappare i limiti fisici di saturazione delle risorse sul silicio. L'obiettivo di testare la configurazione a 1024 moduli è l'identificazione del punto di rottura del sistema, ovvero la soglia critica in cui la saturazione del routing globale impedisce la corretta convergenza temporale.

Per queste tre specifiche configurazioni ad altissima densità, non si è proceduto alla generazione dei file SAIF tramite simulazione temporale dinamica. Tale scelta è stata dettata dall'eccessiva onerosità dei tempi di computazione richiesti dal simulatore per la gestione di un numero così elevato di nodi interconnessi, a fronte di un ridotto valore aggiunto in termini di accuratezza. Pertanto, l'analisi dei consumi per i modelli a 256 e 512 core si affida alla stima statistica di tipo *vectorless* integrata nel Power Report di Xilinx Vivado. Tale metodologia basata su modelli predittivi statici permette, in questa fase di stress test architetturale, di estrapolare e validare con sufficiente confidenza il trend asintotico di crescita della potenza sia statica che dinamica.

L'analisi dei dati a 1024 istanze parallele, mostrata in Fig. 67 evidenzia il limite fisico del dispositivo di riferimento e conferma, per contrasto, la robustezza operativa delle configurazioni ottimizzate nell'intervallo 1-512 core.

Name	Slice LUTs	Slice Registers	F7 Muxes	F8 Muxes	Slice (33650	LUT as Logic	Block RAM Tile (365)
Sigmoid_MultiCore_Top	79765	93508	16170	6762	28987	79765	185.5

Name	Slice LUTs	Slice Registers	F7 Muxes	F8 Muxes	Slice (33650	LUT as Logic	Block RAM Tile (365)
Sigmoid_MultiCore_Top	59.26%	34.74%	24.03%	20.10%	86.14%	59.26%	50.82%

Setup

Worst Negative Slack (WNS):	-0,053 ns
Total Negative Slack (TNS):	-0,277 ns
Number of Failing Endpoints:	7
Total Number of Endpoints:	110906

Hold

Worst Hold Slack (WHS):	0,084 ns
Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	110906

Timing constraints are not met.

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

Total On-Chip Power:	**5.115 W**
Design Power Budget:	**Not Specified**
Process:	typical
Power Budget Margin:	**N/A**
Junction Temperature:	**38,2°C**
Thermal Margin:	46,8°C (17,7 W)
Ambient Temperature:	25.0 °C
Effective ϑJA:	2,6°C/W
Power supplied to off-chip devices:	0 W

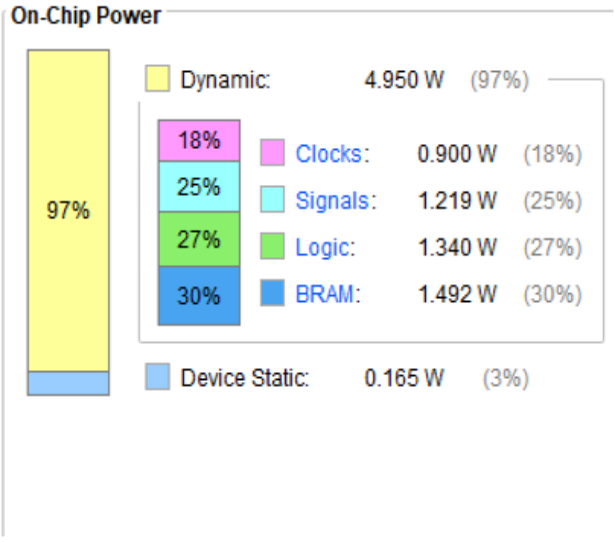


Figura 67 – Analisi OOC 1024 moduli

Nella seguente tabella vengono classificati i diversi livelli di parallelismo sintetizzati in modalità OOC:

Tabella IV – Analisi scalabilità in OOC

Moduli paralleli	**Frequenza operativa (MHz)**	**LUT**	**FF**	**BRAM 36 Kb**	**WNS (ns)**	**Potenza Totale (W)**	**Potenza dinamica/ statica (%)**
1	200	31	85	0.5	0.874	0.127	4/96
2	200	61	170	0.5	0,674	0.133	8/92
4	200	122	340	1	0,246	0.144	15/85
8	200	244	680	2	0,289	0.166	27/73
16	200	488	1360	4	0,265	0.190	35/65
32	200	976	2720	8	0,430	0.238	48/52
64	200	1953	5440	16	0,165	0.333	63/37
128	200	3907	10880	32	0,252	0.508	75/25
256	200	7813	21760	65	0,190	1.164	89/11
512	200	15629	43520	130	0,141	2.144	93/7
1024	200	79765	93508	185,5 + DR	<0	5.115	97/3

L'analisi evidenzia una scalabilità lineare delle risorse logiche fino alla configurazione da 512 unità, confermando l'efficacia del costrutto *generate* nell'istanziazione delle unità parallele.

Si osservano inoltre le seguenti caratteristiche:

- **Gestione risorse di memoria (BRAM):** Fino a 512 PE, l'allocazione delle BRAM segue una progressione lineare. Il valore di 65 BRAM in corrispondenza dei 256 PE, anziché il valore teorico di 64, indica un intervento di ottimizzazione euristica

da parte del sintetizzatore, che ha allocato un blocco supplementare per bilanciare il routing locale. Superata la soglia delle 512 unità, il sistema incontra il limite fisico di saturazione delle risorse di memoria dedicata, costringendo Vivado a ricorrere alla logica distribuita (LUT-RAM) per la gestione dei restanti dati, causa principale dell'impennata nel numero di LUT occupate nel passaggio a 1024 PE.

- **Comportamento del WNS:** Mantiene un andamento positivo e stabile nell'intervallo 4-512 PE, garantendo la chiusura temporale alla frequenza operativa di 200 MHz. La drastica transizione a un WNS negativo in corrispondenza dei 1024 PE segnala il superamento della soglia critica: la congestione del routing globale, esacerbata dall'uso forzato di LUT-RAM e dall'elevata densità di interconnessioni, impedisce il raggiungimento del target temporale, rendendo tale configurazione non implementabile in condizioni operative nominali.
- **Evoluzione del consumo energetico:** Il consumo di potenza mostra una crescita non lineare, che accelera sensibilmente oltre i 512 PE. È opportuno distinguere tra la componente statica, legata alla dispersione di corrente del silicio, e la componente dinamica, strettamente proporzionale alla frequenza di commutazione e al numero di elementi attivi. L'incremento da 2,144 W a 5,115 W nel passaggio a 1024 PE evidenzia come l'elevata densità logica e il conseguente aumento del toggle rate del routing congestionato portino a una dissipazione energetica che supera l'efficienza richiesta per un sistema embedded, rendendo evidente che il punto di trade-off ottimale tra densità computazionale e costo energetico è situato nel range 128-512 PE.

Si sottolinea che l'implementazione fino a 1024 istanze non rappresenta una configurazione d'uso reale, bensì uno stress test finalizzato a validare i limiti strutturali e la robustezza del codice. La scalabilità del sistema garantisce la massima flessibilità di configurazione, lasciando all'utilizzatore finale l'onere di dimensionare l'IP Core in base alle specifiche risorse hardware e ai vincoli applicativi del proprio progetto.

4.5 Confronto FPGA vs CPU

Al fine di valutare quantitativamente le prestazioni computazionali e l'efficienza energetica dell'IP Core sviluppato su FPGA, sono state condotte operazioni di benchmarking comparativo. L'architettura hardware è stata messa a confronto con un'implementazione software equivalente dell'algoritmo di attivazione (sigmoide), eseguita in ambiente MATLAB. La piattaforma computazionale di riferimento (Host PC) utilizzata per i test software adotta un processore commerciale AMD Ryzen 5 7535H. Tale CPU, basata su architettura a 6 core fisici e 12 core logici (thread), è caratterizzata da una frequenza di clock di base di 3.30 GHz con capacità di accelerazione dinamica (Turbo Boost) fino a 4.59 GHz.
Per isolare l'impatto delle diverse strategie di gestione dei thread tipiche delle moderne CPU general-purpose, il benchmark software è stato suddiviso in due scenari applicativi nettamente distinti:

- **Scenario Single-Thread (Loop Sequenziale):** L'algoritmo viene eseguito in modalità rigidamente seriale tramite un ciclo for nativo. Questo approccio costringe il sistema operativo ad allocare l'intero carico computazionale su un singolo core logico della CPU, consentendo di valutare le prestazioni pure in assenza di parallelismo software.
- **Scenario Multi-Thread (Approccio Vettorizzato):** Il calcolo viene eseguito sfruttando la vettorizzazione intrinseca di MATLAB, che delega l'operazione alle librerie matematiche ottimizzate di basso livello (BLAS/LAPACK). In questo modo, la CPU attiva l'Implicit Multithreading, distribuendo il carico di lavoro su tutti i core disponibili e sfruttando le istruzioni hardware SIMD (Single Instruction, Multiple Data).

Metodologicamente, ciascuno dei due script MATLAB opera in due fasi sequenziali e mutuamente esclusive, preservando l'integrità dei dati fisici rilevati:

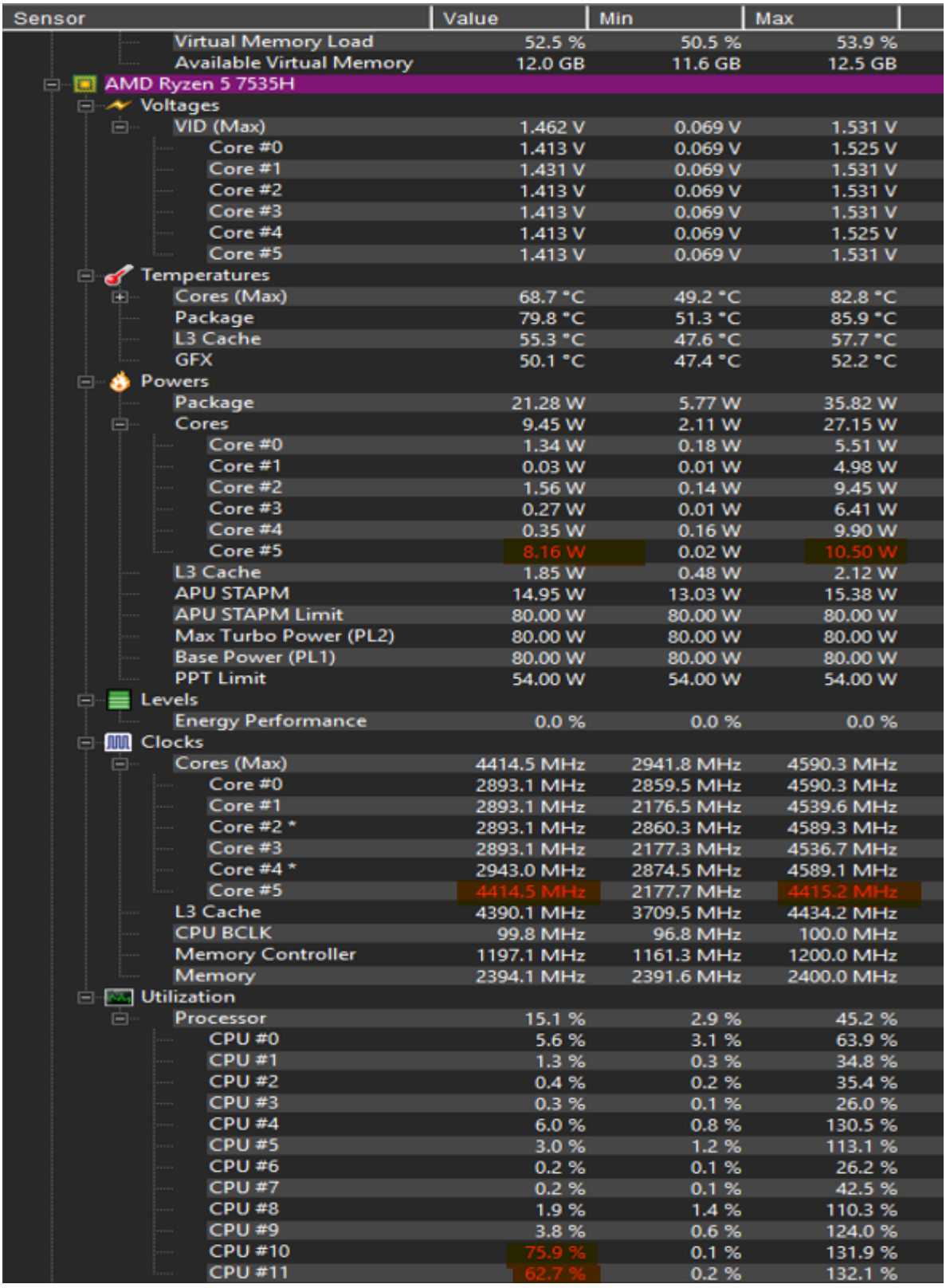
1. **Analisi Temporale:** Il codice esegue un’elaborazione su una porzione del dataset di input reale (composto da **216.000 campioni**), estratta con lo script riportato in Appendice A.9, misurando tramite cronometro software ad alta precisione (tic-toc) il tempo esatto di esecuzione espresso in millisecondi. Questa misura è isolata per evitare qualsiasi alterazione dovuta a fenomeni di deriva termica.

2. **Analisi Energetica (Stress Test):** Subito dopo la misura temporale, l'esecuzione viene posta in un ciclo continuo per una durata tassativa di 60 secondi. Questo intervallo esteso è stringente per consentire alla CPU e al sistema di dissipazione termica del notebook di raggiungere un regime di equilibrio termico. Durante questa fase, sono stati estrapolati i valori relativi alla potenza del processore tramite l'utility HWMonitor, isolando il consumo energetico a regime associato. Entrambi gli script sono riportati rispettivamente in Appendice C.6 E C.7.

Un elemento metodologico di fondamentale importanza nella valutazione delle architetture computazionali è il concetto di **determinismo della latenza**. I sistemi basati su CPU general-purpose, come l'AMD Ryzen 5 oggetto di questo test, sono intrinsecamente non deterministici. Il tempo richiesto da una CPU per elaborare un determinato pacchetto di dati risente di molteplici variabili dinamiche e stocastiche regolate dal sistema operativo e dall'hardware stesso, tra cui:

- L'attività di pianificazione dello scheduler (*OS scheduling*).
- La gestione delle interruzioni hardware e software (*hardware interrupts*).
- I cambi di contesto (*context switching*) tra i processi in background.
- I fenomeni di *cache miss* e i transitori termici legati alla variazione dinamica della frequenza.

A causa di questo intrinseco *jitter* temporale, i valori di latenza software registrati in questa tesi per gli scenari di Cold Run e Warm Run non devono essere intesi come valori assoluti e immutabili, bensì come medie statistiche calcolate su un set esteso di iterazioni consecutive. Tale approccio statistico si è reso necessario per filtrare il rumore di fondo del sistema operativo e isolare il reale comportamento dell'algoritmo.



Sensor	Value	Min	Max
Virtual Memory Load	52.5 %	50.5 %	53.9 %
Available Virtual Memory	12.0 GB	11.6 GB	12.5 GB
AMD Ryzen 5 7535H			
Voltages			
VID (Max)	1.462 V	0.069 V	1.531 V
Core #0	1.413 V	0.069 V	1.525 V
Core #1	1.431 V	0.069 V	1.531 V
Core #2	1.413 V	0.069 V	1.531 V
Core #3	1.413 V	0.069 V	1.531 V
Core #4	1.413 V	0.069 V	1.525 V
Core #5	1.413 V	0.069 V	1.531 V
Temperatures			
Cores (Max)	68.7 °C	49.2 °C	82.8 °C
Package	79.8 °C	51.3 °C	85.9 °C
L3 Cache	55.3 °C	47.6 °C	57.7 °C
GFX	50.1 °C	47.4 °C	52.2 °C
Powers			
Package	21.28 W	5.77 W	35.82 W
Cores	9.45 W	2.11 W	27.15 W
Core #0	1.34 W	0.18 W	5.51 W
Core #1	0.03 W	0.01 W	4.98 W
Core #2	1.56 W	0.14 W	9.45 W
Core #3	0.27 W	0.01 W	6.41 W
Core #4	0.35 W	0.16 W	9.90 W
Core #5	8.16 W	0.02 W	10.50 W
L3 Cache	1.85 W	0.48 W	2.12 W
APU STAPM	14.95 W	13.03 W	15.38 W
APU STAPM Limit	80.00 W	80.00 W	80.00 W
Max Turbo Power (PL2)	80.00 W	80.00 W	80.00 W
Base Power (PL1)	80.00 W	80.00 W	80.00 W
PPT Limit	54.00 W	54.00 W	54.00 W
Levels			
Energy Performance	0.0 %	0.0 %	0.0 %
Clocks			
Cores (Max)	4414.5 MHz	2941.8 MHz	4590.3 MHz
Core #0	2893.1 MHz	2859.5 MHz	4590.3 MHz
Core #1	2893.1 MHz	2176.5 MHz	4539.6 MHz
Core #2 *	2893.1 MHz	2860.3 MHz	4589.3 MHz
Core #3	2893.1 MHz	2177.3 MHz	4536.7 MHz
Core #4 *	2943.0 MHz	2874.5 MHz	4589.1 MHz
Core #5	4414.5 MHz	2177.7 MHz	4415.2 MHz
L3 Cache	4390.1 MHz	3709.5 MHz	4434.2 MHz
CPU BCLK	99.8 MHz	96.8 MHz	100.0 MHz
Memory Controller	1197.1 MHz	1161.3 MHz	1200.0 MHz
Memory	2394.1 MHz	2391.6 MHz	2400.0 MHz
Utilization			
Processor	15.1 %	2.9 %	45.2 %
CPU #0	5.6 %	3.1 %	63.9 %
CPU #1	1.3 %	0.3 %	34.8 %
CPU #2	0.4 %	0.2 %	35.4 %
CPU #3	0.3 %	0.1 %	26.0 %
CPU #4	6.0 %	0.8 %	130.5 %
CPU #5	3.0 %	1.2 %	113.1 %
CPU #6	0.2 %	0.1 %	26.2 %
CPU #7	0.2 %	0.1 %	42.5 %
CPU #8	1.9 %	1.4 %	110.3 %
CPU #9	3.8 %	0.6 %	124.0 %
CPU #10	75.9 %	0.1 %	131.9 %
CPU #11	62.7 %	0.2 %	132.1 %

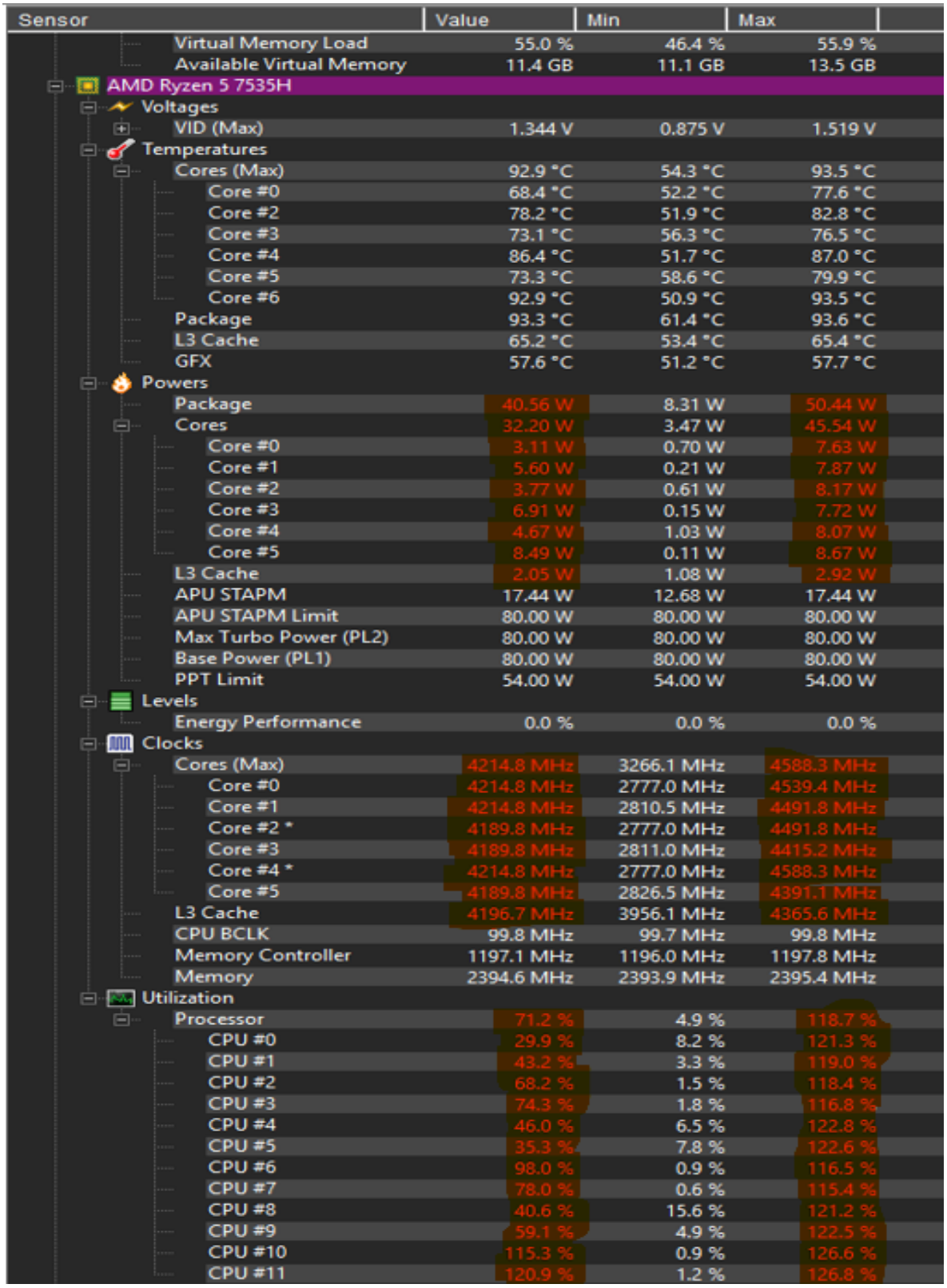
Cold Run

```
TEST 1: SINGLE-THREAD
Tempo di elaborazione: 5.63 ms
```

Warm Run

```
TEST 1: SINGLE-THREAD
Tempo di elaborazione: 2.96 ms
```

Figura 68 – Analisi elaborazione CPU Single-Thread



Sensor	Value	Min	Max
Virtual Memory Load	55.0 %	46.4 %	55.9 %
Available Virtual Memory	11.4 GB	11.1 GB	13.5 GB
AMD Ryzen 5 7535H			
Voltages			
VID (Max)	1.344 V	0.875 V	1.519 V
Temperatures			
Cores (Max)	92.9 °C	54.3 °C	93.5 °C
Core #0	68.4 °C	52.2 °C	77.6 °C
Core #2	78.2 °C	51.9 °C	82.8 °C
Core #3	73.1 °C	56.3 °C	76.5 °C
Core #4	86.4 °C	51.7 °C	87.0 °C
Core #5	73.3 °C	58.6 °C	79.9 °C
Core #6	92.9 °C	50.9 °C	93.5 °C
Package	93.3 °C	61.4 °C	93.6 °C
L3 Cache	65.2 °C	53.4 °C	65.4 °C
GFX	57.6 °C	51.2 °C	57.7 °C
Powers			
Package	40.56 W	8.31 W	50.44 W
Cores	32.20 W	3.47 W	45.54 W
Core #0	3.11 W	0.70 W	7.63 W
Core #1	5.60 W	0.21 W	7.87 W
Core #2	3.77 W	0.61 W	8.17 W
Core #3	6.91 W	0.15 W	7.72 W
Core #4	4.67 W	1.03 W	8.07 W
Core #5	8.49 W	0.11 W	8.67 W
L3 Cache	2.05 W	1.08 W	2.92 W
APU STAPM	17.44 W	12.68 W	17.44 W
APU STAPM Limit	80.00 W	80.00 W	80.00 W
Max Turbo Power (PL2)	80.00 W	80.00 W	80.00 W
Base Power (PL1)	80.00 W	80.00 W	80.00 W
PPT Limit	54.00 W	54.00 W	54.00 W
Levels			
Energy Performance	0.0 %	0.0 %	0.0 %
Clocks			
Cores (Max)	4214.8 MHz	3266.1 MHz	4588.3 MHz
Core #0	4214.8 MHz	2777.0 MHz	4539.4 MHz
Core #1	4214.8 MHz	2810.5 MHz	4491.8 MHz
Core #2 *	4189.8 MHz	2777.0 MHz	4491.8 MHz
Core #3	4189.8 MHz	2811.0 MHz	4415.2 MHz
Core #4 *	4214.8 MHz	2777.0 MHz	4588.3 MHz
Core #5	4189.8 MHz	2826.5 MHz	4391.1 MHz
L3 Cache	4196.7 MHz	3956.1 MHz	4365.6 MHz
CPU BCLK	99.8 MHz	99.7 MHz	99.8 MHz
Memory Controller	1197.1 MHz	1196.0 MHz	1197.8 MHz
Memory	2394.6 MHz	2393.9 MHz	2395.4 MHz
Utilization			
Processor	71.2 %	4.9 %	118.7 %
CPU #0	29.9 %	8.2 %	121.3 %
CPU #1	43.2 %	3.3 %	119.0 %
CPU #2	68.2 %	1.5 %	118.4 %
CPU #3	74.3 %	1.8 %	116.8 %
CPU #4	46.0 %	6.5 %	122.8 %
CPU #5	35.3 %	7.8 %	122.6 %
CPU #6	98.0 %	0.9 %	116.5 %
CPU #7	78.0 %	0.6 %	115.4 %
CPU #8	40.6 %	15.6 %	121.2 %
CPU #9	59.1 %	4.9 %	122.5 %
CPU #10	115.3 %	0.9 %	126.6 %
CPU #11	120.9 %	1.2 %	126.8 %

Cold Run

```
TEST 2: MULTI-THREAD VETTORIZZATO
Tempo di elaborazione: 2.55 ms
```

Warm Run

```
TEST 2: MULTI-THREAD VETTORIZZATO
Tempo di elaborazione: 0.74 ms
```

Figura 69 – Analisi elaborazione CPU Multi-Thread

Nelle Fig. 68 e 69 vengono analizzati i valori ricavati dai test, evidenziando il divario tra la prima esecuzione a freddo (Cold Run), penalizzata dall'inizializzazione dei thread e dalla compilazione Just-In-Time (JIT) di MATLAB, e l'esecuzione a regime (Warm Run), che beneficia del caricamento dei dati nella cache L3:

- Scenario Single-Thread (Loop For): La latenza media crolla da un valore di 5.63 ms (Cold Run) a un valore stabile a regime di 2.96 ms (Warm Run). In questa condizione, la telemetria mostra la saturazione del solo Core #5 (8.16 W a 4414.5 MHz), con un consumo complessivo del chip di 21.28 W e una temperatura di 79.8 °C.
- Scenario Multi-Thread (Vettorizzato): La latenza media passa da 2.55 ms (Cold Run) a 0.74 ms (Warm Run). La parallelizzazione spinge la CPU al suo limite termodinamico: l'assorbimento di pacchetto sale a 40.56 W (con picchi di 50.44 W), distribuito su tutti i 12 core logici a una frequenza di 4214.8 MHz. La temperatura di 93.3 °C indica l'attivazione delle soglie di *thermal throttling*.

L'implementazione su FPGA Artix-7, di canto opposto, offre un profilo prestazionale rigorosamente deterministico. Essendo un circuito hardware dedicato, l'IP Core esegue la computazione in un numero fisso e immutabile di cicli di clock La latenza hardware è ricavabile analiticamente tramite la relazione deterministica: $T_{\text{lat}} = N_{\text{clock}} \times T_{\text{clk}}$. Essa risulta totalmente indipendente dallo stato termico, dalla cronologia dei dati in ingresso o dalla presenza di thread concorrenti.

Nella tabella seguente viene proposto un confronto tra CPU e FPGA in termini di tempo di elaborazione, throughput e massimo consumo di potenza.

Tabella V – Confronto eterogeneo FPGA vs CPU

Architettura di Calcolo	Tempo di Elaborazione (ms)	Throughput (MSamples/s)	Consumo Massimo (W)
CPU ST (Loop For)	2.96 – 5.63 (variabile)	38.37 - 72.97	≈ 35.82
CPU MT (Vettorizzato)	0.74 – 2.55 (variabile)	99.54 - 308.57	≈ 50.44
IP Core FPGA 1 Modulo (OOC)	1.08 (fisso)	200	0.128
IP Core FPGA 2 Moduli (OOC)	0.54 (fisso)	400	0.133
IP Core FPGA 4 Moduli (OOC)	0.27 (fisso)	800	0.144
IP Core FPGA 8 Moduli (OOC)	0.135 (fisso)	1600	0.166
IP Core FPGA 16 Moduli (OOC)	0.0675 (fisso)	3200	0.190
IP Core FPGA 32 Moduli (OOC)	0.0338 (fisso)	6400	0.238
IP Core FPGA 64 Moduli (OOC)	0.0169 (fisso)	12800	0.333

Conclusioni

Il presente lavoro di tesi ha affrontato la progettazione, l'ottimizzazione e la caratterizzazione hardware su logica programmabile FPGA di un IP Core dedicato all'accelerazione della funzione di attivazione non lineare sigmoide. L'esigenza di tale sviluppo si colloca nel contesto delle moderne architetture di Intelligenza Artificiale basate su Transformer, dove la gestione efficiente delle componenti non lineari rappresenta uno dei più severi colli di bottiglia computazionali. La complessità intrinseca nel calcolo di funzioni trascendentali ostacola infatti il throughput complessivo dei sistemi general-purpose, rendendo necessaria la progettazione di architetture hardware dedicate capaci di operare in tempo reale e con consumi ridotti.

Attraverso un approccio ingegneristico di tipo incrementale, la ricerca ha condotto alla validazione di tre distinte release architetturali, ottimizzate progressivamente per bilanciare la precisione fixed-point e il footprint sul silicio. Se la versione baseline ha permesso di consolidare il datapath iniziale in formato Q5.11 su Distributed RAM, l'architettura si è evoluta nella versione V2 mediante l'introduzione di una logica di clipping hardware, fondamentale per neutralizzare i fenomeni di rollover numerico a fronte di dataset dinamici estesi provenienti da framework esterni. L'ottimizzazione culmina nell'architettura V3, la quale introduce il formato ad alta precisione decimale Q1.15 e capitalizza la proprietà di simmetria centrale della sigmoide. Questa scelta progettuale ha consentito di dimezzare l'occupazione fisica della LUT mappando esclusivamente il dominio positivo $[0,8]$ e delegando la ricostruzione speculare del ramo negativo a una logica combinatoria sincrona a ridosso delle BRAM on-chip. Grazie a una rigorosa struttura di pipelining a 7 stadi volta a isolare i ritardi di routing, il singolo core operativo ha dimostrato una perfetta convergenza temporale a una frequenza di clock di 250 MHz. L'efficacia energetica del design low-power è stata rigorosamente validata tramite annotazione post-implementazione dei vettori di commutazione reali (file SAIF), evidenziando un consumo dinamico effettivo estremamente contenuto.

L'efficacia complessiva di questo lavoro trova la sua massima espressione nella prospettiva di integrazione dell'IP Core all'interno di acceleratori complessi per modelli Transformer, quali Vision Transformer (ViT), BERT o Swin Transformer.

Sfruttando la variante parallela V3.1, l'architettura isolata è stata trasformata in un sistema multicore a parallelismo scalabile. Le procedure di stress-test condotte in modalità OOC fino a un limite di 1024 istanze hanno confermato la robustezza strutturale del codice HDL e permesso di mappare con precisione l'andamento asintotico delle risorse e della potenza dissipata sul silicio. L'IP Core sviluppato si dimostra così pronto per essere integrato e per operare in sinergia con i blocchi di calcolo lineare (matrici di attenzione e stadi di feed-forward), eliminando i tempi morti di computazione e massimizzando il throughput complessivo dell'acceleratore neurale. Nel benchmarking comparativo effettuato, rispetto alle architetture general purpose, l'acceleratore hardware proposto offre tre vantaggi strategici determinanti, cruciali per il deployment in scenari embedded ed edge computing:

1. **Latenza deterministica e pipeline sincrona:** A differenza delle CPU (come l'architettura AMD Ryzen oggetto di test), nelle quali il calcolo delle funzioni trascendentali è affidato a routine software sequenziali condizionate dalle fluttuazioni di scheduling del sistema operativo, l'IP Core su FPGA garantisce un flusso di dati continuo con una latenza fissa e strettamente prevedibile (pari a soli 7 cicli di clock).
2. **Risoluzione del collo di bottiglia delle non-linearità:** Sebbene le GPU siano straordinariamente efficienti nell'eseguire il calcolo tensoriale lineare parallelo massivo, esse subiscono pesanti penalizzazioni in termini di throughput quando la pipeline grafica o di calcolo deve arrestarsi per valutare le funzioni non lineari punto a punto. L'IP Core sviluppato scarica completamente l'unità di calcolo principale da questo onere trascendentale, elaborando i vettori di attivazione in parallelo e in tempo reale.
3. **Efficienza energetica:** Con una quota di potenza dinamica reale confinata nell'ordine dei milliwatt per singola unità operativa, l'architettura proposta esibisce un rapporto throughput-per-Watt drasticamente superiore rispetto ai processori commerciali e alle GPU discrete. Questa caratteristica rende l'IP Core ideale per l'esecuzione di modelli di Intelligenza Artificiale direttamente a bordo di sistemi embedded (dispositivi automotive, robotica, droni o dispositivi medicali), dove i vincoli termici e di autonomia energetica escludono a priori l'utilizzo di hardware general-purpose più energivoro.

In conclusione, l'IP Core sviluppato rappresenta un punto di giunzione ottimale tra la fedeltà algoritmica richiesta dai modelli di DL e la sostenibilità computazionale del silicio. I risultati documentati aprono la strada a promettenti sviluppi futuri, tra cui l'estensione del blocco di pre-processing e della gestione della simmetria per l'accelerazione hardware di ulteriori funzioni di attivazione. Sebbene la funzione sigmoide sia stata eletta come caso studio primario per la validazione del sistema, l'infrastruttura hardware progettata, basata su una precisa segmentazione in pipeline, logiche di clipping anti-rollover e gestione della simmetria, possiede una valenza metodologica generale. Essa costituisce infatti un'eccellente architettura di base, flessibile e prontamente riutilizzabile, per l'accelerazione di altre funzioni di attivazione non lineari tipiche dei modelli predittivi profondi. Con minime modifiche algoritmiche, limitate principalmente al ricalcolo dei valori di posizionamento nella ROM e all'adeguamento dei nodi di inversione del segno, l'IP Core può essere declinato per computare la Tangente Iperbolica (tanh) o le più recenti funzioni *smooth* indispensabili nei Transformers di ultima generazione, quali Swish e GeLU (Gaussian Error Linear Unit).

Infine, l'evoluzione più strategica dell'architettura risiede nell'aggiornamento del data-path per supportare una quantizzazione fixed-point a 8 bit (in formato Q4.4 per gli ingressi e Q1.7 per le uscite). Sotto il profilo dell'area logica, il passaggio a 8 bit dimezzerebbe l'occupazione di elementi sequenziali e combinatori, consentendo di istanziare una densità doppia di core nel medesimo spazio di silicio; sotto il profilo della banda, sbloccherebbe la possibilità di effettuare un data packing ultra-denso sfruttando interfacce ad alte prestazioni come i bus AXI4-Stream a 1024 bit nativi della famiglia UltraScale+. In questo scenario, l'architettura potrebbe accogliere simultaneamente ben 128 moduli operanti in parallelo nello stesso fronte di clock, raddoppiando il throughput globale rispetto alla configurazione a 16 bit e minimizzando l'impatto energetico complessivo: una simile sinergia tra compressione del dato e scalabilità strutturale proietta il design proposto come un pilastro tecnologico abilitante per l'esecuzione in tempo reale dei più complessi modelli generativi, ridefinendo i confini e le prestazioni della moderna Edge AI.

Bibliografia

[1] «A. Vaswani et al., “Attention is all you need,” in Proc. Conf. Neural Informat. Process. Syst., 2017, pp. 6000–6010».

[2] «D. Bahdanau et al., “Neural machine translation by jointly learning to align and translate,” in Proc. Int. Conf. Learn. Representations, 2015».

[3] «A. Parikh et al., “A decomposable attention model for natural language inference,” in Proc. Empir. Methods Natural Lang. Process., 2016, pp. 2249–2255».

[4] «A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in Proc. Int. Conf. Learn. Representations, 2021».

[5] «J.Devlin et al., “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics-Hum. Lang. Technol., 2019, pp. 4171–4186».

[6] «T. B. Brown et al., “Language models are few-shot learners,” in Proc. Conf. Neural Informat. Process. Syst., 2020, pp. 1877–1901».

[7] «K. He et al., “Deep residual learning for image recognition,” in Proc. Conf. Comput. Vis. Pattern Recognit., 2016, pp. 770–778».

[8] «S. Ren et al., “Faster R-CNN: Towards real-time object detection with region proposal networks,” in Proc. Conf. Neural Informat. Process. Syst., 2015, pp. 91–99».

[9] «Salman Khan, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah. 2022. Transformers in vision: A survey. ACM computing surveys (CSUR) 54, 10s (2022), 1–41».

[10] «Soufiane Hayou, Arnaud Doucet, and Judith Rousseau. 2019. On the Impact of the Activation function on Deep Neural Networks Training. In Proceedings of the 36th International Conference on Machine Learning».

[11] «Vladimír Kunc and Jiří Kléma. 2024. Three Decades of Activations: A Comprehensive Survey of 400 Activation Functions for Neural Networks. arXiv preprint arXiv:2402.09092 (2024)».

[12] «Samuel S Schoenholz, Justin Gilmer, Surya Ganguli, and Jascha Sohl-Dickstein. 2016. Deep information propagation. arXiv preprint arXiv:1611.01232 (2016)».

[13] «H. Cao et al., “Swin-Unet: Unet-like pure transformer for medical image segmentation,” 2021, arXiv:2105.05537».

[14] «Z. Liu et al., “Swin transformer: Hierarchical vision transformer using shifted windows,” in Proc. Int. Conf. Comput. Vis., 2021, pp. 10012–10022».

[15] «N. Carion et al., “End-to-end object detection with transformers,” in Proc. Eur. Conf. Comput. Vis., 2020, pp. 213–229».

[16] «X. Zhu et al., “Deformable DETR: Deformable transformers for endto- end object detection,” in Proc. Int. Conf. Learn. Representations, 2021».

[17] «S. Zheng et al., “Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers,” in Proc. Conf. Comput. Vis. Pattern Recognit., 2021, pp. 6881–6890».

[18] «Bin Ding, Huimin Qian, and Jun Zhou. 2018. Activation functions and their characteristics in deep neural networks. In 2018 Chinese control and decision conference (CCDC). IEEE, 1836–1841».

[19] «Yvuraj Singh, Madan Saini, et al. 2023. Impact and Performance Analysis of Various Activation Functions for Classification Problems. In 2023 IEEE International Conference on Contemporary Computing and Communications (InC4), Vol. 1. IEEE, 1–7».

[20] «Qinghe Zheng, Mingqiang Yang, Xinyu Tian, XiaochenWang, and DeqiangWang. 2020. Rethinking the Role of Activation Functions in Deep Convolutional Neural Networks for Image Classification. Engineering Letters 28, 1 (2020)».

[21] «Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. 2009. CIFAR-100 (Canadian Institute For Advanced Research). Technical Report. Technical Report».

[22] «Tiny ImageNet. 2006. Tiny ImageNet Visual Recognition Challenge. https://tinyimagenet.herokuapp.com/. (2006)».

[23] lucidrains, «https://github.com/lucidrains/vit-pytorch,» [Online].

[24] Y. C. T. W. W. Y. Y. S. Z.-H. J. F. E. T. J. F. a. S. Y. Li Yuan, «Tokens-to-token vit: Training vision transformers from scratch on imagenet. In Proceedings of the IEEE/CVF international conference on computer vision. 558–567. 2021».

[25] «Yang Liu, Yao Zhang, Yixin Wang, Feng Hou, Jin Yuan, Jiang Tian, Yang Zhang, Zhongchao Shi, Jianping Fan, and Zhiqiang He. 2023. A survey of visual transformers. IEEE Transactions on Neural Networks and Learning Systems (2023)».

[26] «M. Tommiska, "Efficient digital implementation of the sigmoid function for reprogrammable logic," in IEEE Computers and Digital Techniques, 2003».

[27] «K. Guo et al., "A Survey of FPGA-Based Accelerators for Convolutional Neural Networks," in IEEE Design & Test, 2017».

[28] «Parhami, B. (2010) Computer Arithmetic: Algorithms and Hardware Designs. 2nd Edition, Oxford University Press, New York».

[29] «J. Long et al., “Fully convolutional networks for semantic segmentation,” in Proc. Conf. Comput. Vis. Pattern Recognit., 2015, pp. 3431–3440».

[30] «H. Wang et al., “Max-deeplab: End-to-end panoptic segmentation with mask transformers,” in Proc. Conf. Comput. Vis. Pattern Recognit., 2021, pp. 5463–5474».

[31] «G. Prato et al., “Fully quantized transformer for machine translation,” in Findings Proc. Empir. Methods Natural Lang. Process.,2020, pp. 1–14».

[32] «X. Jiao et al., “Distilling BERT for natural language understanding,” in Findings Proc. Empir. Methods Natural Lang. Process., 2020, pp. 4163–4174».

[33] «S. Vassiliadis et al., "On the implementation of the sigmoid function," in Proceedings of the 6th Great Lakes Symposium on VLSI, 1996».

Appendice A: Implementazione software e generazione coefficienti (Python)

A.1 Script estrazione coefficienti LUT versione baseline (*Baseline_LUT_samples.py*)

```
import pandas as pd
import sys
import os

os.chdir(os.path.dirname(os.path.abspath(__file__)))

df = pd.read_excel('sigmoid.xlsx', skiprows=2)
y_values = df['Y'].fillna(0).tolist()[:256]

def to_q5_11_hex(val):

    val_float = float(str(val).replace(',', '.'))
    scaled = int(round(val_float * 2048))
    if scaled > 32767: scaled = 32767
    if scaled < -32768: scaled = -32768
    if scaled < 0: scaled = (1 << 16) + scaled
    return f'x"{scaled:04x}"'

vhdl_hex = [to_q5_11_hex(y) for y in y_values]

with open("LUT_Values.txt", "w") as f:
    for i in range(0, len(vhdl_hex), 8):
        line = ", ".join(vhdl_hex[i:i+8])
        if i + 8 < len(vhdl_hex):
            f.write(f"{line},\n")
        else:
            f.write(f"{line}\n")
```

A.2 Script estrazione input ordinati versione baseline (*Baseline_inputs.py*)

```
import pandas as pd
import os
import sys

os.chdir(os.path.dirname(os.path.abspath(__file__)))

def to_q5_11_hex(val):

    scaled = int(round(float(str(val).replace(',', '.')) * 2048))
    scaled = max(-32768, min(32767, scaled))
    return f"{scaled & 0xFFFF:04X}"

df = pd.read_excel('sigmoid.xlsx', skiprows=2).head(256)
with open('stimoli_input_baseline.mem', 'w') as f:
    for x in df['X']:
        f.write(to_q5_11_hex(x) + '\n')
```

A.3 Script modello di riferimento software sigmoide (*golden_model.py*)

```
import math

START = -4.0
SAMPLES = 256
STEP_SIZE = 0.03125

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

with open("sigmoid_outputs_sw.txt", "w") as f:
    for i in range(SAMPLES):
        x = START + (i * STEP_SIZE)
        y = sigmoid(x)
        f.write(f"{y:.15f}\n")
```

A.4 Script valori funzione sigmoide: generazione dei 1024 coefficienti della LUT Custom (*1024_values_lut.py*)

```
import math

def to_fixed_q5_11(val):

    scaled = int(round(val * 2048))
    return format(scaled & 0xFFFF, '04x')

def generate_values():
    res = 1024
    range_min = -4.0
    range_max = 4.0
    step = (range_max - range_min) / (res - 1)


    values = []
    for i in range(res):

        x = range_min + (i * step)

        sigmoid_x = 1 / (1 + math.exp(-x))

        hex_val = to_fixed_q5_11(sigmoid_x)

        values.append(f'x"{hex_val}"')

    with open("LUT_Values_V2.txt", "w") as f:
        for i in range(0, len(values), 8):
            comma = "," if i + 8 < len(values) else ""
            linea = "    " + ", ".join(values[i:i+8]) + comma + "\n"
            f.write(linea)

generate_values()
```

A.5 Script per generazione input ordinati nell'intervallo [-12 ; 12] per TB robustezza (*256_inputs_12.py*)

```
import numpy as np

num_samples = 256
range_min = -12.0
range_max = 12.0

def to_q5_11_hex(val):

    scaled_val = int(round(val * (2048)))

    if scaled_val < 0:
        scaled_val = (1 << 16) + scaled_val

    return f"{scaled_val & 0xFFFF:04X}"

inputs = np.linspace(range_min, range_max, num_samples)

with open("stimoli_input_12.mem", "w") as f:
    for val in inputs:
        hex_val = to_q5_11_hex(val)
        f.write(f"{hex_val}\n")
```

A.6 Script per generazione 1024 input ordinati nell'intervallo per TB Full ROM (*1024_inputs.py*)

```
import numpy as np

num_samples = 1024
range_min = -4.0
range_max = 4.0

def to_q5_11_hex(val):

    scaled_val = int(round(val * (2048)))

    if scaled_val < 0:
        scaled_val = (1 << 16) + scaled_val

    return f"{scaled_val & 0xFFFF:04X}"

inputs = np.linspace(range_min, range_max, num_samples)

with open("stimoli_input_1024.mem", "w") as f:
    for val in inputs:
        hex_val = to_q5_11_hex(val)
        f.write(f"{hex_val}\n")
```

A.7 Script per 1024 coefficienti sigmoide, Q1.15, range $[0\ ;\ 8]$ (*1024_values_0_8.py*)

```
import math

def to_q1_15(val):
    scaled = int(round(val * 32768))
    return format(scaled & 0xFFFF, '04x')

def generate_values():
    res = 1024
    range_min = 0.0
    range_max = 8.0
    step = (range_max - range_min) / res
    values = []
    for i in range(res):
        x = range_min + (i * step)
        sigmoid_x = 1 / (1 + math.exp(-x))
        hex_val = to_q1_15(sigmoid_x)
        values.append(f'x"{hex_val}"')

    with open("LUT_Values_V3.txt", "w") as f:
        for i in range(0, len(values), 8):
            comma = "," if i + 8 < len(values) else ""
            linea = "    " + ", ".join(values[i:i+8]) + comma + "\n"
            f.write(linea)

generate_values()
```

A.8 Script conversione dataset reale (*clean_dataset.py*)

```
import re

def to_q5_11(val):
    return f"{(int(round(val * 2048)) & 0xFFFF):04x}"

with open("dataset.txt", 'r', encoding='utf-8') as f:
    content = f.read().replace('−', '-').replace('—', '-')
valori = [float(x) for x in re.findall(r'-?\d*\.\d+|[+-]?\d+', content)]

with open("input_hex.hex", 'w') as h, open("output_float.txt", 'w') as f:

   # Prende a blocchi di 432 (216 input + 216 output)
    for i in range(0, len(valori), 432):
        for val in valori[i : i + 216]:
            h.write(to_q5_11(val) + "\n")
        for val in valori[i + 216 : i + 432]:
            f.write(f"{val:.6f}\n")
```

A.9 Script estrazione input da dataset reale per test CPU (*CPU_input_Test.py*)

```
   import re

with open("dataset.txt", "r", encoding="utf-8") as f:
    content = f.read().replace("−", "-").replace("—", "-")

valori = [float(x) for x in re.findall(r"-?\d*\.\d+|[+-]?\d+", content)]

with open("input_valori.txt", "w") as f:

    for b in range(min(1000, len(valori) // 432)):
        inizio = b * 432
        for val in valori[inizio : inizio + 216]:
            f.write(f"{val:.6f}\n")
```

Appendice B: Descrizione hardware, progettazione HDL e test (Vivado)

B.1 Definizione dei coefficienti sigmoide per LUT *(Sigmoid_Values.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


package Sigmoid_Values is

    type lut_type is array (0 to 255) of std_logic_vector(15 downto 0);

    constant SIGMOID_TABLE : lut_type :=

   (x"0000", x"002a", x"002a", x"002a", x"002a", x"002a", x"002a", x"002a",
    x"0034", x"0034", x"0034", x"0034", x"0034", x"0034", x"0034", x"003e",
    x"003e", x"003e", x"003e", x"003e", x"0048", x"0048", x"0048", x"0048",
    x"0048", x"0052", x"0052", x"0052", x"0052", x"005c", x"005c", x"005c",
    x"005c", x"0067", x"0067", x"0067", x"0071", x"0071", x"0071", x"007b",
    x"007b", x"007b", x"0085", x"0085", x"008f", x"008f", x"008f", x"0099",
    x"0099", x"00a3", x"00a3", x"00ae", x"00ae", x"00b8", x"00b8", x"00c2",
    x"00c2", x"00cb", x"00cb", x"00d5", x"00df", x"00df", x"00e8", x"00f2",
    x"00f2", x"00fc", x"0105", x"010f", x"010f", x"0119", x"0123", x"012c",
    x"0135", x"0135", x"013f", x"0149", x"0153", x"015d", x"0166", x"0170",
    x"0179", x"0183", x"018d", x"0196", x"019f", x"01a8", x"01b2", x"01c5",
    x"01cf", x"01d9", x"01e3", x"01ec", x"01fe", x"0207", x"0211", x"0224",
    x"022d", x"0237", x"0241", x"0255", x"025e", x"0270", x"0278", x"028a",
    x"0293", x"02a5", x"02b7", x"02c0", x"02d3", x"02dc", x"02ef", x"02f9",
    x"030c", x"031f", x"0329", x"033d", x"0347", x"035b", x"0365", x"0379",
    x"038d", x"0397", x"03ab", x"03b5", x"03ca", x"03d4", x"03e8", x"03fd",
    x"0407", x"041c", x"0426", x"043a", x"0445", x"0459", x"0463", x"0477",
    x"048b", x"0495", x"04a9", x"04b3", x"04c7", x"04d1", x"04e5", x"04f8",
    x"0502", x"0515", x"051e", x"0531", x"053a", x"054d", x"055f", x"0568",
    x"057a", x"0582", x"0594", x"059c", x"05b0", x"05ba", x"05c4", x"05d8",
    x"05e1", x"05eb", x"05fd", x"0607", x"0610", x"0622", x"062b", x"0635",
    x"063f", x"0649", x"0652", x"065c", x"066e", x"0678", x"0682", x"068c",
    x"0695", x"069f", x"06a8", x"06a8", x"06b2", x"06bc", x"06c5", x"06cf",
    x"06d8", x"06e2", x"06e2", x"06ec", x"06f6", x"0700", x"0700", x"070a",
    x"0713", x"0713", x"071d", x"0727", x"0727", x"0731", x"0731", x"073b",
    x"0745", x"0745", x"074f", x"074f", x"0759", x"0759", x"0763", x"0763",
    x"076c", x"076c", x"076c", x"0776", x"0776", x"0780", x"0780", x"0780",
    x"078a", x"078a", x"078a", x"0794", x"0794", x"0794", x"079f", x"079f",
    x"079f", x"07a9", x"07a9", x"07a9", x"07a9", x"07b3", x"07b3", x"07b3",
    x"07b3", x"07bd", x"07bd", x"07bd", x"07bd", x"07bd", x"07c7", x"07c7",
    x"07c7", x"07c7", x"07c7", x"07c7", x"07d1", x"07d1", x"07d1", x"07d1",
    x"07d1", x"07d1", x"07d1", x"07d9", x"07d9", x"07d9", x"07d9", x"0800");


end package Sigmoid_Values;
```

B.2 Blocco di pre-processing baseline *(Pre_Processing.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Pre_Processing is
    Port (
         clk, reset, valid_in : in  STD_LOGIC;
         data_in : in  STD_LOGIC_VECTOR (15 downto 0);
         valid_out : out STD_LOGIC;
         addr_out : out UNSIGNED (7 downto 0)
         );
end Pre_Processing;

architecture Behavioral of Pre_Processing is
    constant OFFSET_VAL : signed(15 downto 0) := to_signed(8192, 16);
begin
    process(clk)
        variable temp_sum : signed(15 downto 0);
    begin
        if rising_edge(clk) then
            if reset = '1' then
                addr_out <= (others => '0');
                valid_out <= '0';
            else

                -- Stadio pipeline: calcolo indirizzo e registrazione
                valid_out <= valid_in;
                temp_sum := signed(data_in) + OFFSET_VAL;
                addr_out <= unsigned(temp_sum(13 downto 6));

            end if;
        end if;
    end process;

end Behavioral;
```

B.3 Blocco di memoria baseline *(Sigmoid_LUT.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

use work.Sigmoid_Values.all;

entity Sigmoid_LUT is
    Port (
         clk, reset, valid_in : in  STD_LOGIC;
         addr_in : in  UNSIGNED (7 downto 0);
         valid_out : out STD_LOGIC;
         data_out : out STD_LOGIC_VECTOR (15 downto 0)
         );
end Sigmoid_LUT;

architecture Behavioral of Sigmoid_LUT is

    signal ROM_CONTENT : lut_type := SIGMOID_TABLE;
    attribute rom_style : string;
    attribute rom_style of ROM_CONTENT : signal is "distributed";

    -- Segnali Pipeline
    signal addr_reg : std_logic_vector(7 downto 0);
    signal v_pipe : std_logic;
```

```
begin

    process(clk)
    begin
        if rising_edge(clk) then

            -- I stadio pipeline: registro d'ingresso
            addr_reg <= std_logic_vector(addr_in);

            -- II stadio pipeline: registro d'uscita
            data_out <= ROM_CONTENT(to_integer(unsigned(addr_reg)));

            if reset = '1' then
                v_pipe <= '0';
                valid_out <= '0';
            else

                -- Pipeline bit di validità (profondità 2)
                v_pipe <= valid_in;
                valid_out <= v_pipe;
            end if;
        end if;
    end process;

end Behavioral;
```

B.4 Top module architettura baseline *(TOP_Design.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity TOP_Design is
    Port (
         clk, reset, valid_in : in  STD_LOGIC;
         input_data : in  STD_LOGIC_VECTOR (15 downto 0);
         valid_out : out STD_LOGIC;
         output_data : out STD_LOGIC_VECTOR (15 downto 0)
         );
end TOP_Design;

architecture Structural of TOP_Design is

    component Pre_Processing
        Port (
             clk, reset, valid_in : in  STD_LOGIC;
             data_in : in  STD_LOGIC_VECTOR (15 downto 0);
             valid_out : out STD_LOGIC;
             addr_out : out UNSIGNED (7 downto 0)
             );
    end component;

    component Sigmoid_LUT
        Port (
             clk, reset, valid_in : in  STD_LOGIC;
             addr_in : in  UNSIGNED (7 downto 0);
             valid_out : out STD_LOGIC;
             data_out : out STD_LOGIC_VECTOR (15 downto 0)
             );
    end component;

    -- Segnali Pipeline
    signal internal_addr : UNSIGNED (7 downto 0);
    signal reg_valid_in, internal_valid, reg_valid_out : STD_LOGIC;
    signal reg_input_data, reg_output_data : STD_LOGIC_VECTOR(15 downto 0);
```

```
begin

    -- I stadio pipeline: input buffering
    process(clk)
    begin
        if rising_edge(clk) then
            if reset = '1' then
                reg_input_data <= (others => '0');
                reg_valid_in   <= '0';
            else
                reg_input_data <= input_data;
                reg_valid_in <= valid_in;
            end if;
        end if;
    end process;


    -- II stadio pipeline: logica di pre-processing
    PP: Pre_Processing
        port map (clk => clk, reset => reset, valid_in => reg_valid_in, data_in
=> reg_input_data, valid_out => internal_valid, addr_out => internal_addr);

    -- III, IV stadio pipeline: accesso alla memoria e uscita
    MEM: Sigmoid_LUT
        port map (clk => clk,reset => reset, valid_in => internal_valid, addr_in
=> internal_addr, valid_out => reg_valid_out, data_out => reg_output_data);

    -- V stadio pipeline: output buffering
    process(clk)
         begin
            if rising_edge(clk) then
                if reset = '1' then
                    output_data <= (others => '0');
                    valid_out <= '0';
                else
                 output_data <= reg_output_data;
                 valid_out <= reg_valid_out;
                end if;
             end if;
        end process;

end Structural;
```

B.5 Testbench di coerenza *(TB_Sigmoid.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use STD.TEXTIO.ALL;
use IEEE.STD_LOGIC_TEXTIO.ALL;

entity TB_Sigmoid is
end TB_Sigmoid;

architecture Behavioral of TB_Sigmoid is

    component TOP_Design
        Port (
             clk, reset, valid_in: in  STD_LOGIC;
             input_data : in  STD_LOGIC_VECTOR (15 downto 0);
             valid_out : out STD_LOGIC;
             output_data : out STD_LOGIC_VECTOR (15 downto 0)
             );
    end component;
```

```
    signal clk, reset, valid_in : STD_LOGIC := '0';
    signal input_data : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');
    signal valid_out : STD_LOGIC;
    signal output_data : STD_LOGIC_VECTOR(15 downto 0);

    constant CLK_PERIOD : time := 4 ns;
    signal stop_sim : boolean := false;

begin

    uut: TOP_Design
      port map (clk=> clk, reset => reset, valid_in => valid_in, input_data =>
input_data, valid_out => valid_out, output_data => output_data);

    -- Processo di generazione clock
    clk_process : process
    begin
        while not stop_sim loop
            clk <= '0'; wait for CLK_PERIOD/2;
            clk <= '1'; wait for CLK_PERIOD/2;
        end loop;
        wait;
    end process;

    -- Processo di stimolo
    stim_read: process
        file file_input : text open read_mode is "stimoli_input_baseline.mem";
        variable v_line : line;
        variable v_data : std_logic_vector(15 downto 0);
    begin

        reset <= '1';
        valid_in <= '0';
        input_data <= (others => '0');
        wait for CLK_PERIOD * 20;

        -- Rilascio reset sincronizzato sul fronte di discesa
        wait until falling_edge(clk);
        reset <= '0';

        wait for CLK_PERIOD * 10;

        while not endfile(file_input) loop
            readline(file_input, v_line);
            hread(v_line, v_data);

            wait until falling_edge(clk);
            input_data <= v_data;
            valid_in <= '1';
        end loop;

        wait until falling_edge(clk);
        valid_in <= '0';

        -- Attesa svuotamento pipeline
        wait for CLK_PERIOD * 50;
        stop_sim <= true;
        wait;
    end process;


    -- Processo di scrittura risultati
    results_writer: process
        file file_output : text open write_mode is "risultati_HW_256.hex";
        variable v_line : line;
    begin
```

```
 wait until reset = '0';
        while not stop_sim loop

            -- Sincronizzazione sul fronte di salita
            wait until rising_edge(clk);
            if valid_out = '1' then
                hwrite(v_line, output_data);
                writeline(file_output, v_line);
            end if;
        end loop;
        wait;
    end process;

end Behavioral;
```

B.6 Package V2 *(Sigmoid_Values_1024.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


package Sigmoid_Values_1024 is

    type lut_type is array (0 to 1023) of std_logic_vector(15 downto 0);

    constant SIGMOID_TABLE : lut_type :=

   (x"0025", x"0025", x"0025", x"0026", x"0026", x"0026", x"0027", x"0027",
    x"0027", x"0027", x"0028", x"0028", x"0028", x"0029", x"0029", x"0029",
    x"002a", x"002a", x"002a", x"002b", x"002b", x"002b", x"002c", x"002c",
    x"002c", x"002d", x"002d", x"002d", x"002e", x"002e", x"002e", x"002f",
    x"002f", x"002f", x"0030", x"0030", x"0031", x"0031", x"0031", x"0032",
    x"0032", x"0032", x"0033", x"0033", x"0034", x"0034", x"0034", x"0035",
    x"0035", x"0036", x"0036", x"0036", x"0037", x"0037", x"0038", x"0038",
    x"0039", x"0039", x"0039", x"003a", x"003a", x"003b", x"003b", x"003c",
    x"003c", x"003d", x"003d", x"003d", x"003e", x"003e", x"003f", x"003f",
    x"0040", x"0040", x"0041", x"0041", x"0042", x"0042", x"0043", x"0043",
    x"0044", x"0044", x"0045", x"0045", x"0046", x"0046", x"0047", x"0047",
    x"0048", x"0049", x"0049", x"004a", x"004a", x"004b", x"004b", x"004c",
    x"004c", x"004d", x"004e", x"004e", x"004f", x"004f", x"0050", x"0051",
    x"0051", x"0052", x"0052", x"0053", x"0054", x"0054", x"0055", x"0056",
    x"0056", x"0057", x"0058", x"0058", x"0059", x"005a", x"005a", x"005b",
    x"005c", x"005c", x"005d", x"005e", x"005e", x"005f", x"0060", x"0060",
    x"0061", x"0062", x"0063", x"0063", x"0064", x"0065", x"0066", x"0066",
    x"0067", x"0068", x"0069", x"006a", x"006a", x"006b", x"006c", x"006d",
    x"006d", x"006e", x"006f", x"0070", x"0071", x"0072", x"0072", x"0073",
    x"0074", x"0075", x"0076", x"0077", x"0078", x"0079", x"0079", x"007a",
    x"007b", x"007c", x"007d", x"007e", x"007f", x"0080", x"0081", x"0082",
    x"0083", x"0084", x"0085", x"0086", x"0087", x"0088", x"0089", x"008a",
    x"008b", x"008c", x"008d", x"008e", x"008f", x"0090", x"0091", x"0092",
    x"0093", x"0094", x"0095", x"0096", x"0097", x"0098", x"0099", x"009a",
    x"009c", x"009d", x"009e", x"009f", x"00a0", x"00a1", x"00a2", x"00a4",
    x"00a5", x"00a6", x"00a7", x"00a8", x"00aa", x"00ab", x"00ac", x"00ad",
    x"00af", x"00b0", x"00b1", x"00b2", x"00b4", x"00b5", x"00b6", x"00b7",
    x"00b9", x"00ba", x"00bb", x"00bd", x"00be", x"00bf", x"00c1", x"00c2",
    x"00c4", x"00c5", x"00c6", x"00c8", x"00c9", x"00cb", x"00cc", x"00cd",
    x"00cf", x"00d0", x"00d2", x"00d3", x"00d5", x"00d6", x"00d8", x"00d9",
    x"00db", x"00dc", x"00de", x"00df", x"00e1", x"00e3", x"00e4", x"00e6",
    x"00e7", x"00e9", x"00eb", x"00ec", x"00ee", x"00f0", x"00f1", x"00f3",
    x"00f5", x"00f6", x"00f8", x"00fa", x"00fb", x"00fd", x"00ff", x"0101",
    x"0102", x"0104", x"0106", x"0108", x"0109", x"010b", x"010d", x"010f",
    x"0111", x"0113", x"0115", x"0116", x"0118", x"011a", x"011c", x"011e",
```

```
x"0120", x"0122", x"0124", x"0126", x"0128", x"012a", x"012c", x"012e",
x"0130", x"0132", x"0134", x"0136", x"0138", x"013a", x"013c", x"013e",
x"0140", x"0142", x"0145", x"0147", x"0149", x"014b", x"014d", x"014f",
x"0152", x"0154", x"0156", x"0158", x"015a", x"015d", x"015f", x"0161",
x"0164", x"0166", x"0168", x"016b", x"016d", x"016f", x"0172", x"0174",
x"0176", x"0179", x"017b", x"017e", x"0180", x"0182", x"0185", x"0187",
x"018a", x"018c", x"018f", x"0191", x"0194", x"0196", x"0199", x"019c",
x"019e", x"01a1", x"01a3", x"01a6", x"01a9", x"01ab", x"01ae", x"01b1",
x"01b3", x"01b6", x"01b9", x"01bb", x"01be", x"01c1", x"01c4", x"01c6",
x"01c9", x"01cc", x"01cf", x"01d1", x"01d4", x"01d7", x"01da", x"01dd",
x"01e0", x"01e3", x"01e5", x"01e8", x"01eb", x"01ee", x"01f1", x"01f4",
x"01f7", x"01fa", x"01fd", x"0200", x"0203", x"0206", x"0209", x"020c",
x"020f", x"0212", x"0215", x"0218", x"021b", x"021f", x"0222", x"0225",
x"0228", x"022b", x"022e", x"0231", x"0235", x"0238", x"023b", x"023e",
x"0242", x"0245", x"0248", x"024b", x"024f", x"0252", x"0255", x"0259",
x"025c", x"025f", x"0263", x"0266", x"0269", x"026d", x"0270", x"0273",
x"0277", x"027a", x"027e", x"0281", x"0285", x"0288", x"028b", x"028f",
x"0292", x"0296", x"0299", x"029d", x"02a1", x"02a4", x"02a8", x"02ab",
x"02af", x"02b2", x"02b6", x"02b9", x"02bd", x"02c1", x"02c4", x"02c8",
x"02cc", x"02cf", x"02d3", x"02d7", x"02da", x"02de", x"02e2", x"02e5",
x"02e9", x"02ed", x"02f0", x"02f4", x"02f8", x"02fc", x"02ff", x"0303",
x"0307", x"030b", x"030e", x"0312", x"0316", x"031a", x"031e", x"0321",
x"0325", x"0329", x"032d", x"0331", x"0335", x"0338", x"033c", x"0340",
x"0344", x"0348", x"034c", x"0350", x"0353", x"0357", x"035b", x"035f",
x"0363", x"0367", x"036b", x"036f", x"0373", x"0377", x"037b", x"037f",
x"0383", x"0386", x"038a", x"038e", x"0392", x"0396", x"039a", x"039e",
x"03a2", x"03a6", x"03aa", x"03ae", x"03b2", x"03b6", x"03ba", x"03be",
x"03c2", x"03c6", x"03ca", x"03ce", x"03d2", x"03d6", x"03da", x"03de",
x"03e2", x"03e6", x"03ea", x"03ee", x"03f2", x"03f6", x"03fa", x"03fe",
x"0402", x"0406", x"040a", x"040e", x"0412", x"0416", x"041a", x"041e",
x"0422", x"0426", x"042a", x"042e", x"0432", x"0436", x"043a", x"043e",
x"0442", x"0446", x"044a", x"044e", x"0452", x"0456", x"045a", x"045e",
x"0462", x"0466", x"046a", x"046e", x"0472", x"0476", x"047a", x"047d",
x"0481", x"0485", x"0489", x"048d", x"0491", x"0495", x"0499", x"049d",
x"04a1", x"04a5", x"04a9", x"04ad", x"04b0", x"04b4", x"04b8", x"04bc",
x"04c0", x"04c4", x"04c8", x"04cb", x"04cf", x"04d3", x"04d7", x"04db",
x"04df", x"04e2", x"04e6", x"04ea", x"04ee", x"04f2", x"04f5", x"04f9",
x"04fd", x"0501", x"0504", x"0508", x"050c", x"0510", x"0513", x"0517",
x"051b", x"051e", x"0522", x"0526", x"0529", x"052d", x"0531", x"0534",
x"0538", x"053c", x"053f", x"0543", x"0547", x"054a", x"054e", x"0551",
x"0555", x"0558", x"055c", x"055f", x"0563", x"0567", x"056a", x"056e",
x"0571", x"0575", x"0578", x"057b", x"057f", x"0582", x"0586", x"0589",
x"058d", x"0590", x"0593", x"0597", x"059a", x"059d", x"05a1", x"05a4",
x"05a7", x"05ab", x"05ae", x"05b1", x"05b5", x"05b8", x"05bb", x"05be",
x"05c2", x"05c5", x"05c8", x"05cb", x"05cf", x"05d2", x"05d5", x"05d8",
x"05db", x"05de", x"05e1", x"05e5", x"05e8", x"05eb", x"05ee", x"05f1",
x"05f4", x"05f7", x"05fa", x"05fd", x"0600", x"0603", x"0606", x"0609",
x"060c", x"060f", x"0612", x"0615", x"0618", x"061b", x"061d", x"0620",
x"0623", x"0626", x"0629", x"062c", x"062f", x"0631", x"0634", x"0637",
x"063a", x"063c", x"063f", x"0642", x"0645", x"0647", x"064a", x"064d",
x"064f", x"0652", x"0655", x"0657", x"065a", x"065d", x"065f", x"0662",
x"0664", x"0667", x"066a", x"066c", x"066f", x"0671", x"0674", x"0676",
x"0679", x"067b", x"067e", x"0680", x"0682", x"0685", x"0687", x"068a",
x"068c", x"068e", x"0691", x"0693", x"0695", x"0698", x"069a", x"069c",
x"069f", x"06a1", x"06a3", x"06a6", x"06a8", x"06aa", x"06ac", x"06ae",
x"06b1", x"06b3", x"06b5", x"06b7", x"06b9", x"06bb", x"06be", x"06c0",
x"06c2", x"06c4", x"06c6", x"06c8", x"06ca", x"06cc", x"06ce", x"06d0",
x"06d2", x"06d4", x"06d6", x"06d8", x"06da", x"06dc", x"06de", x"06e0",
x"06e2", x"06e4", x"06e6", x"06e8", x"06ea", x"06eb", x"06ed", x"06ef",
x"06f1", x"06f3", x"06f5", x"06f7", x"06f8", x"06fa", x"06fc", x"06fe",
x"06ff", x"0701", x"0703", x"0705", x"0706", x"0708", x"070a", x"070b",
x"070d", x"070f", x"0710", x"0712", x"0714", x"0715", x"0717", x"0719",
x"071a", x"071c", x"071d", x"071f", x"0721", x"0722", x"0724", x"0725",
x"0727", x"0728", x"072a", x"072b", x"072d", x"072e", x"0730", x"0731",
x"0733", x"0734", x"0735", x"0737", x"0738", x"073a", x"073b", x"073c",
x"073e", x"073f", x"0741", x"0742", x"0743", x"0745", x"0746", x"0747",
x"0749", x"074a", x"074b", x"074c", x"074e", x"074f", x"0750", x"0751",
```

```
    x"0753", x"0754", x"0755", x"0756", x"0758", x"0759", x"075a", x"075b",
    x"075c", x"075e", x"075f", x"0760", x"0761", x"0762", x"0763", x"0764",
    x"0766", x"0767", x"0768", x"0769", x"076a", x"076b", x"076c", x"076d",
    x"076e", x"076f", x"0770", x"0771", x"0772", x"0773", x"0774", x"0775",
    x"0776", x"0777", x"0778", x"0779", x"077a", x"077b", x"077c", x"077d",
    x"077e", x"077f", x"0780", x"0781", x"0782", x"0783", x"0784", x"0785",
    x"0786", x"0787", x"0787", x"0788", x"0789", x"078a", x"078b", x"078c",
    x"078d", x"078e", x"078e", x"078f", x"0790", x"0791", x"0792", x"0793",
    x"0793", x"0794", x"0795", x"0796", x"0796", x"0797", x"0798", x"0799",
    x"079a", x"079a", x"079b", x"079c", x"079d", x"079d", x"079e", x"079f",
    x"07a0", x"07a0", x"07a1", x"07a2", x"07a2", x"07a3", x"07a4", x"07a4",
    x"07a5", x"07a6", x"07a6", x"07a7", x"07a8", x"07a8", x"07a9", x"07aa",
    x"07aa", x"07ab", x"07ac", x"07ac", x"07ad", x"07ae", x"07ae", x"07af",
    x"07af", x"07b0", x"07b1", x"07b1", x"07b2", x"07b2", x"07b3", x"07b4",
    x"07b4", x"07b5", x"07b5", x"07b6", x"07b6", x"07b7", x"07b7", x"07b8",
    x"07b9", x"07b9", x"07ba", x"07ba", x"07bb", x"07bb", x"07bc", x"07bc",
    x"07bd", x"07bd", x"07be", x"07be", x"07bf", x"07bf", x"07c0", x"07c0",
    x"07c1", x"07c1", x"07c2", x"07c2", x"07c3", x"07c3", x"07c3", x"07c4",
    x"07c4", x"07c5", x"07c5", x"07c6", x"07c6", x"07c7", x"07c7", x"07c7",
    x"07c8", x"07c8", x"07c9", x"07c9", x"07ca", x"07ca", x"07ca", x"07cb",
    x"07cb", x"07cc", x"07cc", x"07cc", x"07cd", x"07cd", x"07ce", x"07ce",
    x"07ce", x"07cf", x"07cf", x"07cf", x"07d0", x"07d0", x"07d1", x"07d1",
    x"07d1", x"07d2", x"07d2", x"07d2", x"07d3", x"07d3", x"07d3", x"07d4",
    x"07d4", x"07d4", x"07d5", x"07d5", x"07d5", x"07d6", x"07d6", x"07d6",
    x"07d7", x"07d7", x"07d7", x"07d8", x"07d8", x"07d8", x"07d9", x"07d9",
    x"07d9", x"07d9", x"07da", x"07da", x"07da", x"07db", x"07db", x"07db");


end package Sigmoid_Values_1024;
```

B.7 Blocco di pre-processing intermedio *(Pre_Processing_V2.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Pre_Processing_V2 is
    Port (
         clk, reset, valid_in : in  STD_LOGIC;
         data_in : in  STD_LOGIC_VECTOR (15 downto 0);
         valid_out : out STD_LOGIC;
         addr_out : out UNSIGNED (9 downto 0)
         );
end Pre_Processing_V2;

architecture Behavioral of Pre_Processing_V2 is

    constant OFFSET_VAL : signed(15 downto 0) := to_signed(8192, 16);

    -- Segnali Pipeline
    signal sum_reg : signed(16 downto 0);
    signal valid_pipe : std_logic;

begin

    process(clk)
    begin
        if rising_edge(clk) then

            if reset = '1' then
                sum_reg <= (others => '0');
                addr_out <= (others => '0');
                valid_pipe <= '0';
                valid_out <= '0';
```

```
            else

              -- I stadio pipeline: offset
              sum_reg <= resize(signed(data_in), 17) + resize(OFFSET_VAL, 17);
              valid_pipe <= valid_in;

              -- II stadio pipeline: clipping e slicing
              valid_out <= valid_pipe;

                if sum_reg < 0 then
                    addr_out <= (others => '0');
                elsif sum_reg >= to_signed(16384, 17) or (sum_reg(16) = '0' and
sum_reg(15) = '1') then
                    addr_out <= to_unsigned(1023, 10);
                else
                    addr_out <= unsigned(sum_reg(13 downto 4));
                end if;
            end if;
        end if;
    end process;

end Behavioral;
```

B.8 Blocco di memoria intermedio *(Sigmoid_LUT_V2.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

use work.Sigmoid_Values_V2.all;

entity Sigmoid_LUT_V2 is
    Port (
         clk, reset, valid_in : in  STD_LOGIC;
         addr_in : in  UNSIGNED (9 downto 0);
         valid_out : out STD_LOGIC;
         data_out : out STD_LOGIC_VECTOR (15 downto 0)
         );
end Sigmoid_LUT_V2;

architecture Behavioral of Sigmoid_LUT_V2 is

    signal ROM_CONTENT : lut_type := SIGMOID_TABLE;
    attribute rom_style : string;
    attribute rom_style of ROM_CONTENT : signal is "block";

    -- Segnali Pipeline
    signal addr_reg : std_logic_vector(9 downto 0);
    signal v_pipe : std_logic;

begin

    process(clk)
    begin
        if rising_edge(clk) then

            -- I stadio pipeline: registro d'ingresso
            addr_reg <= std_logic_vector(addr_in);

            -- II stadio pipeline: registro d'uscita
            data_out <= ROM_CONTENT(to_integer(unsigned(addr_reg)));

            if reset = '1' then
```

```
                v_pipe <= '0';
                valid_out <= '0';
            else

                v_pipe <= valid_in;
                valid_out <= v_pipe;
            end if;
        end if;
    end process;

end Behavioral;
```

B.9 Top module architettura V2 *(Sigmoid_Accelerator_TOP_V2.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Sigmoid_Accelerator_TOP_V2 is
    Port (
         clk, reset, valid_in : in  STD_LOGIC;
         input_data : in  STD_LOGIC_VECTOR (15 downto 0);
         valid_out : out STD_LOGIC;
         output_data : out STD_LOGIC_VECTOR (15 downto 0)
         );
end Sigmoid_Accelerator_TOP_V2;

architecture Structural of Sigmoid_Accelerator_TOP_V2 is

    component Pre_Processing_V2
        Port (
             clk, reset, valid_in : in  STD_LOGIC;
             data_in : in  STD_LOGIC_VECTOR (15 downto 0);
             valid_out : out STD_LOGIC;
             addr_out : out UNSIGNED (9 downto 0)
             );
    end component;

    component Sigmoid_LUT_V2
        Port (
             clk, reset, valid_in : in  STD_LOGIC;
             addr_in : in  UNSIGNED (9 downto 0);
             valid_out : out STD_LOGIC;
             data_out : out STD_LOGIC_VECTOR (15 downto 0)
             );
    end component;

    -- Segnali Pipeline
    signal internal_addr : UNSIGNED (9 downto 0);
    signal reg_valid_in, internal_valid, reg_valid_out : STD_LOGIC;
    signal reg_input_data, reg_output_data : STD_LOGIC_VECTOR(15 downto 0);

begin

    -- I stadio pipeline: input buffering
    process(clk)
    begin
        if rising_edge(clk) then
            if reset = '1' then
                reg_input_data <= (others => '0');
                reg_valid_in <= '0';
            else
                reg_input_data <= input_data;
                reg_valid_in <= valid_in;
            end if;
```

```
        end if;
    end process;

    -- II, III stadio pipeline: logica di pre-processing
    PP: Pre_Processing_V2
        port map (clk => clk, reset => reset, valid_in => reg_valid_in, data_in
=> reg_input_data, valid_out => internal_valid, addr_out => internal_addr);

    -- IV, V stadio pipeline: accesso alla memoria e uscita
    MEM: Sigmoid_LUT_V2
        port map (clk => clk, reset => reset, valid_in => internal_valid,
addr_in => internal_addr, valid_out => reg_valid_out, data_out =>
reg_output_data);

    -- VI stadio pipeline: output buffering
    process(clk)
    begin
        if rising_edge(clk) then
            if reset = '1' then
                output_data <= (others => '0');
                valid_out <= '0';
            else
                output_data <= reg_output_data;
                valid_out <= reg_valid_out;
            end if;
        end if;
    end process;

end Structural;
```

B.10 Testbench di robustezza (*TB_Stability.vhd*)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use STD.TEXTIO.ALL;
use IEEE.STD_LOGIC_TEXTIO.ALL;

entity TB_Stability is
end TB_Stability;

architecture Behavioral of TB_Stability is

    component Sigmoid_Accelerator_TOP_V2
        Port (
             clk, reset, valid_in: in  STD_LOGIC;
             input_data : in  STD_LOGIC_VECTOR (15 downto 0);
             valid_out : out STD_LOGIC;
             output_data : out STD_LOGIC_VECTOR (15 downto 0)
             );
    end component;

    signal clk, reset, valid_in : STD_LOGIC := '0';
    signal input_data : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');
    signal valid_out : STD_LOGIC;
    signal output_data : STD_LOGIC_VECTOR(15 downto 0);

    constant CLK_PERIOD : time := 4 ns;
    signal stop_sim : boolean := false;

begin

    uut: Sigmoid_Accelerator_TOP_V2
      port map (clk=> clk, reset => reset, valid_in => valid_in, input_data =>
input data, valid out => valid out, output data => output data);
```

```
    -- Processo di generazione clock
    clk_process : process
    begin
        while not stop_sim loop
            clk <= '0'; wait for CLK_PERIOD/2;
            clk <= '1'; wait for CLK_PERIOD/2;
        end loop;
        wait;
    end process;

    -- Processo di stimolo
    stim_read: process
        file file_input : text open read_mode is "stimoli_input_12.mem";
        variable v_line : line;
        variable v_data : std_logic_vector(15 downto 0);
    begin

        reset <= '1';
        valid_in <= '0';
        input_data <= (others => '0');
        wait for CLK_PERIOD * 20;

        -- Rilascio reset sincronizzato sul fronte di discesa
        wait until falling_edge(clk);
        reset <= '0';

        wait for CLK_PERIOD * 10;

        while not endfile(file_input) loop
            readline(file_input, v_line);
            hread(v_line, v_data);

            wait until falling_edge(clk);
            input_data <= v_data;
            valid_in <= '1';
        end loop;

        wait until falling_edge(clk);
        valid_in <= '0';
        input_data <= (others => '0');

        -- Attesa svuotamento pipeline
        wait for CLK_PERIOD * 50;
        stop_sim <= true;
        wait;
    end process;

    -- Processo di scrittura risultati
    results_writer: process
        file file_output : text open write_mode is "risultati_HW_12.hex";
        variable v_line : line;
    begin

        wait until reset = '0';
        while not stop_sim loop

            -- Sincronizzazione sul fronte di salita
            wait until rising_edge(clk);
            if valid_out = '1' then
                hwrite(v_line, output_data);
                writeline(file_output, v_line);
            end if;
        end loop;
        wait;
    end process;

end Behavioral;
```

B.11 Package V3 *(Sigmoid_Values_1024_0_8.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

package Sigmoid_Values_1024_0_8 is

    type lut_type is array (0 to 1023) of std_logic_vector(15 downto 0);

    constant SIGMOID_TABLE : lut_type :=

      (x"4000", x"4040", x"4080", x"40c0", x"4100", x"4140", x"4180", x"41c0",
       x"4200", x"4240", x"4280", x"42c0", x"42ff", x"433f", x"437f", x"43bf",
       x"43ff", x"443e", x"447e", x"44be", x"44fd", x"453d", x"457d", x"45bc",
       x"45fc", x"463b", x"467a", x"46ba", x"46f9", x"4738", x"4777", x"47b6",
       x"47f5", x"4834", x"4873", x"48b2", x"48f1", x"4930", x"496e", x"49ad",
       x"49eb", x"4a2a", x"4a68", x"4aa6", x"4ae5", x"4b23", x"4b61", x"4b9f",
       x"4bdc", x"4c1a", x"4c58", x"4c95", x"4cd3", x"4d10", x"4d4e", x"4d8b",
       x"4dc8", x"4e05", x"4e42", x"4e7f", x"4ebb", x"4ef8", x"4f34", x"4f71",
       x"4fad", x"4fe9", x"5025", x"5061", x"509c", x"50d8", x"5114", x"514f",
       x"518a", x"51c5", x"5200", x"523b", x"5276", x"52b1", x"52eb", x"5325",
       x"5360", x"539a", x"53d4", x"540d", x"5447", x"5481", x"54ba", x"54f3",
       x"552c", x"5565", x"559e", x"55d6", x"560f", x"5647", x"567f", x"56b7",
       x"56ef", x"5727", x"575f", x"5796", x"57cd", x"5804", x"583b", x"5872",
       x"58a8", x"58df", x"5915", x"594b", x"5981", x"59b7", x"59ed", x"5a22",
       x"5a57", x"5a8c", x"5ac1", x"5af6", x"5b2a", x"5b5f", x"5b93", x"5bc7",
       x"5bfb", x"5c2f", x"5c62", x"5c95", x"5cc9", x"5cfc", x"5d2e", x"5d61",
       x"5d93", x"5dc6", x"5df8", x"5e2a", x"5e5b", x"5e8d", x"5ebe", x"5eef",
       x"5f20", x"5f51", x"5f81", x"5fb2", x"5fe2", x"6012", x"6042", x"6072",
       x"60a1", x"60d0", x"60ff", x"612e", x"615d", x"618c", x"61ba", x"61e8",
       x"6216", x"6244", x"6271", x"629f", x"62cc", x"62f9", x"6326", x"6352",
       x"637f", x"63ab", x"63d7", x"6403", x"642e", x"645a", x"6485", x"64b0",
       x"64db", x"6506", x"6530", x"655a", x"6584", x"65ae", x"65d8", x"6602",
       x"662b", x"6654", x"667d", x"66a6", x"66ce", x"66f7", x"671f", x"6747",
       x"676f", x"6796", x"67be", x"67e5", x"680c", x"6833", x"685a", x"6880",
       x"68a6", x"68cc", x"68f2", x"6918", x"693d", x"6963", x"6988", x"69ad",
       x"69d2", x"69f6", x"6a1b", x"6a3f", x"6a63", x"6a87", x"6aaa", x"6ace",
       x"6af1", x"6b14", x"6b37", x"6b5a", x"6b7c", x"6b9f", x"6bc1", x"6be3",
       x"6c05", x"6c26", x"6c48", x"6c69", x"6c8a", x"6cab", x"6ccc", x"6cec",
       x"6d0d", x"6d2d", x"6d4d", x"6d6d", x"6d8d", x"6dac", x"6dcb", x"6dea",
       x"6e09", x"6e28", x"6e47", x"6e65", x"6e84", x"6ea2", x"6ec0", x"6edd",
       x"6efb", x"6f18", x"6f36", x"6f53", x"6f70", x"6f8c", x"6fa9", x"6fc6",
       x"6fe2", x"6ffe", x"701a", x"7036", x"7051", x"706d", x"7088", x"70a3",
       x"70be", x"70d9", x"70f3", x"710e", x"7128", x"7142", x"715c", x"7176",
       x"7190", x"71a9", x"71c3", x"71dc", x"71f5", x"720e", x"7227", x"723f",
       x"7258", x"7270", x"7288", x"72a0", x"72b8", x"72d0", x"72e8", x"72ff",
       x"7316", x"732e", x"7345", x"735b", x"7372", x"7389", x"739f", x"73b5",
       x"73cc", x"73e2", x"73f7", x"740d", x"7423", x"7438", x"744e", x"7463",
       x"7478", x"748d", x"74a2", x"74b6", x"74cb", x"74df", x"74f3", x"7507",
       x"751b", x"752f", x"7543", x"7557", x"756a", x"757e", x"7591", x"75a4",
       x"75b7", x"75ca", x"75dc", x"75ef", x"7602", x"7614", x"7626", x"7638",
       x"764a", x"765c", x"766e", x"7680", x"7691", x"76a3", x"76b4", x"76c5",
       x"76d6", x"76e7", x"76f8", x"7709", x"7719", x"772a", x"773a", x"774a",
       x"775b", x"776b", x"777b", x"778b", x"779a", x"77aa", x"77b9", x"77c9",
       x"77d8", x"77e7", x"77f7", x"7806", x"7814", x"7823", x"7832", x"7841",
       x"784f", x"785d", x"786c", x"787a", x"7888", x"7896", x"78a4", x"78b2",
       x"78c0", x"78cd", x"78db", x"78e8", x"78f6", x"7903", x"7910", x"791d",
       x"792a", x"7937", x"7944", x"7950", x"795d", x"796a", x"7976", x"7982",
       x"798f", x"799b", x"79a7", x"79b3", x"79bf", x"79cb", x"79d7", x"79e2",
       x"79ee", x"79f9", x"7a05", x"7a10", x"7a1c", x"7a27", x"7a32", x"7a3d",
       x"7a48", x"7a53", x"7a5e", x"7a68", x"7a73", x"7a7e", x"7a88", x"7a93",
       x"7a9d", x"7aa7", x"7ab1", x"7abc", x"7ac6", x"7ad0", x"7ad9", x"7ae3",
       x"7aed", x"7af7", x"7b00", x"7b0a", x"7b14", x"7b1d", x"7b26", x"7b30",
       x"7b39", x"7b42", x"7b4b", x"7b54", x"7b5d", x"7b66", x"7b6f", x"7b78",
       x"7b80", x"7b89", x"7b92", x"7b9a", x"7ba3", x"7bab", x"7bb3", x"7bbc",
       x"7bc4", x"7bcc", x"7bd4", x"7bdc", x"7be4", x"7bec", x"7bf4", x"7bfc",
       x"7c03", x"7c0b", x"7c13", x"7c1a", x"7c22", x"7c29", x"7c31", x"7c38",
       x"7c3f", x"7c47", x"7c4e", x"7c55", x"7c5c", x"7c63", x"7c6a", x"7c71",
       x"7c78", x"7c7f", x"7c86", x"7c8c", x"7c93", x"7c9a", x"7ca0", x"7ca7",
       x"7cad", x"7cb4", x"7cba", x"7cc1", x"7cc7", x"7ccd", x"7cd3", x"7cda",
       x"7ce0", x"7ce6", x"7cec", x"7cf2", x"7cf8", x"7cfe", x"7d03", x"7d09",
```

```
        x"7d0f", x"7d15", x"7d1a", x"7d20", x"7d26", x"7d2b", x"7d31", x"7d36",
        x"7d3c", x"7d41", x"7d46", x"7d4c", x"7d51", x"7d56", x"7d5b", x"7d61",
        x"7d66", x"7d6b", x"7d70", x"7d75", x"7d7a", x"7d7f", x"7d84", x"7d89",
        x"7d8d", x"7d92", x"7d97", x"7d9c", x"7da0", x"7da5", x"7daa", x"7dae",
        x"7db3", x"7db7", x"7dbc", x"7dc0", x"7dc4", x"7dc9", x"7dcd", x"7dd1",
        x"7dd6", x"7dda", x"7dde", x"7de2", x"7de7", x"7deb", x"7def", x"7df3",
        x"7df7", x"7dfb", x"7dff", x"7e03", x"7e07", x"7e0a", x"7e0e", x"7e12",
        x"7e16", x"7e1a", x"7e1d", x"7e21", x"7e25", x"7e28", x"7e2c", x"7e30",
        x"7e33", x"7e37", x"7e3a", x"7e3e", x"7e41", x"7e45", x"7e48", x"7e4b",
        x"7e4f", x"7e52", x"7e55", x"7e59", x"7e5c", x"7e5f", x"7e62", x"7e65",
        x"7e69", x"7e6c", x"7e6f", x"7e72", x"7e75", x"7e78", x"7e7b", x"7e7e",
        x"7e81", x"7e84", x"7e87", x"7e8a", x"7e8d", x"7e90", x"7e92", x"7e95",
        x"7e98", x"7e9b", x"7e9e", x"7ea0", x"7ea3", x"7ea6", x"7ea8", x"7eab",
        x"7eae", x"7eb0", x"7eb3", x"7eb5", x"7eb8", x"7eba", x"7ebd", x"7ebf",
        x"7ec2", x"7ec4", x"7ec7", x"7ec9", x"7ecc", x"7ece", x"7ed0", x"7ed3",
        x"7ed5", x"7ed7", x"7eda", x"7edc", x"7ede", x"7ee0", x"7ee3", x"7ee5",
        x"7ee7", x"7ee9", x"7eeb", x"7eed", x"7ef0", x"7ef2", x"7ef4", x"7ef6",
        x"7ef8", x"7efa", x"7efc", x"7efe", x"7f00", x"7f02", x"7f04", x"7f06",
        x"7f08", x"7f0a", x"7f0c", x"7f0d", x"7f0f", x"7f11", x"7f13", x"7f15",
        x"7f17", x"7f18", x"7f1a", x"7f1c", x"7f1e", x"7f20", x"7f21", x"7f23",
        x"7f25", x"7f26", x"7f28", x"7f2a", x"7f2b", x"7f2d", x"7f2f", x"7f30",
        x"7f32", x"7f33", x"7f35", x"7f37", x"7f38", x"7f3a", x"7f3b", x"7f3d",
        x"7f3e", x"7f40", x"7f41", x"7f43", x"7f44", x"7f46", x"7f47", x"7f49",
        x"7f4a", x"7f4b", x"7f4d", x"7f4e", x"7f50", x"7f51", x"7f52", x"7f54",
        x"7f55", x"7f56", x"7f58", x"7f59", x"7f5a", x"7f5b", x"7f5d", x"7f5e",
        x"7f5f", x"7f61", x"7f62", x"7f63", x"7f64", x"7f65", x"7f67", x"7f68",
        x"7f69", x"7f6a", x"7f6b", x"7f6c", x"7f6e", x"7f6f", x"7f70", x"7f71",
        x"7f72", x"7f73", x"7f74", x"7f75", x"7f76", x"7f77", x"7f79", x"7f7a",
        x"7f7b", x"7f7c", x"7f7d", x"7f7e", x"7f7f", x"7f80", x"7f81", x"7f82",
        x"7f83", x"7f84", x"7f85", x"7f86", x"7f87", x"7f87", x"7f88", x"7f89",
        x"7f8a", x"7f8b", x"7f8c", x"7f8d", x"7f8e", x"7f8f", x"7f90", x"7f90",
        x"7f91", x"7f92", x"7f93", x"7f94", x"7f95", x"7f96", x"7f96", x"7f97",
        x"7f98", x"7f99", x"7f9a", x"7f9a", x"7f9b", x"7f9c", x"7f9d", x"7f9e",
        x"7f9e", x"7f9f", x"7fa0", x"7fa1", x"7fa1", x"7fa2", x"7fa3", x"7fa4",
        x"7fa4", x"7fa5", x"7fa6", x"7fa6", x"7fa7", x"7fa8", x"7fa8", x"7fa9",
        x"7faa", x"7faa", x"7fab", x"7fac", x"7fac", x"7fad", x"7fae", x"7fae",
        x"7faf", x"7fb0", x"7fb0", x"7fb1", x"7fb1", x"7fb2", x"7fb3", x"7fb3",
        x"7fb4", x"7fb4", x"7fb5", x"7fb6", x"7fb6", x"7fb7", x"7fb7", x"7fb8",
        x"7fb8", x"7fb9", x"7fba", x"7fba", x"7fbb", x"7fbb", x"7fbc", x"7fbc",
        x"7fbd", x"7fbd", x"7fbe", x"7fbe", x"7fbf", x"7fbf", x"7fc0", x"7fc0",
        x"7fc1", x"7fc1", x"7fc2", x"7fc2", x"7fc3", x"7fc3", x"7fc4", x"7fc4",
        x"7fc5", x"7fc5", x"7fc6", x"7fc6", x"7fc7", x"7fc7", x"7fc7", x"7fc8",
        x"7fc8", x"7fc9", x"7fc9", x"7fca", x"7fca", x"7fca", x"7fcb", x"7fcb",
        x"7fcc", x"7fcc", x"7fcc", x"7fcd", x"7fcd", x"7fce", x"7fce", x"7fce",
        x"7fcf", x"7fcf", x"7fd0", x"7fd0", x"7fd0", x"7fd1", x"7fd1", x"7fd1",
        x"7fd2", x"7fd2", x"7fd3", x"7fd3", x"7fd3", x"7fd4", x"7fd4", x"7fd4",
        x"7fd5", x"7fd5", x"7fd5", x"7fd6", x"7fd6", x"7fd6", x"7fd7", x"7fd7",
        x"7fd7", x"7fd8", x"7fd8", x"7fd8", x"7fd8", x"7fd9", x"7fd9", x"7fd9",
        x"7fda", x"7fda", x"7fda", x"7fdb", x"7fdb", x"7fdb", x"7fdb", x"7fdc",
        x"7fdc", x"7fdc", x"7fdd", x"7fdd", x"7fdd", x"7fdd", x"7fde", x"7fde",
        x"7fde", x"7fde", x"7fdf", x"7fdf", x"7fdf", x"7fdf", x"7fe0", x"7fe0",
        x"7fe0", x"7fe0", x"7fe1", x"7fe1", x"7fe1", x"7fe1", x"7fe2", x"7fe2",
        x"7fe2", x"7fe2", x"7fe3", x"7fe3", x"7fe3", x"7fe3", x"7fe4", x"7fe4",
        x"7fe4", x"7fe4", x"7fe4", x"7fe5", x"7fe5", x"7fe5", x"7fe5", x"7fe5",
        x"7fe6", x"7fe6", x"7fe6", x"7fe6", x"7fe6", x"7fe7", x"7fe7", x"7fe7",
        x"7fe7", x"7fe7", x"7fe8", x"7fe8", x"7fe8", x"7fe8", x"7fe8", x"7fe9",
        x"7fe9", x"7fe9", x"7fe9", x"7fe9", x"7fe9", x"7fea", x"7fea", x"7fea",
        x"7fea", x"7fea", x"7fea", x"7feb", x"7feb", x"7feb", x"7feb", x"7feb",
        x"7feb", x"7fec", x"7fec", x"7fec", x"7fec", x"7fec", x"7fec", x"7fed",
        x"7fed", x"7fed", x"7fed", x"7fed", x"7fed", x"7fed", x"7fee", x"7fee",
        x"7fee", x"7fee", x"7fee", x"7fee", x"7fee", x"7fef", x"7fef", x"7fef",
        x"7fef", x"7fef", x"7fef", x"7fef", x"7ff0", x"7ff0", x"7ff0", x"7ff0",
        x"7ff0", x"7ff0", x"7ff0", x"7ff0", x"7ff1", x"7ff1", x"7ff1", x"7ff1",
        x"7ff1", x"7ff1", x"7ff1", x"7ff1", x"7ff1", x"7ff2", x"7ff2", x"7ff2",
        x"7ff2", x"7ff2", x"7ff2", x"7ff2", x"7ff2", x"7ff2", x"7ff3", x"7ff3",
        x"7ff3", x"7ff3", x"7ff3", x"7ff3", x"7ff3", x"7ff3", x"7ff3", x"7ff3",
        x"7ff4", x"7ff4", x"7ff4", x"7ff4", x"7ff4", x"7ff4", x"7ff4", x"7ff4",
       x"7ff4", x"7ff4", x"7ff4", x"7ff5", x"7ff5", x"7ff5", x"7ff5", x"7ff5");


end package Sigmoid_Values_1024_0_8;
```

B.12 Blocco di pre-processing avanzato *(Pre_Processing_V3.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Pre_Processing_V3 is
    Port (
         clk, reset, valid_in : in  STD_LOGIC;
         data_in : in  STD_LOGIC_VECTOR(15 downto 0);
         valid_out, is_neg_out : out STD_LOGIC;
         addr_out : out UNSIGNED(9 downto 0)
         );
end Pre_Processing_V3;

architecture Behavioral of Pre_Processing_V3 is

    constant CLIP_VAL : signed(16 downto 0) := to_signed(16384,17);
    -- Segnali Pipeline
    signal abs_val : signed(16 downto 0);
    signal neg_reg, valid_reg : std_logic;

begin

    process(clk)
        variable clipped : unsigned(15 downto 0);
    begin
        if rising_edge(clk) then

            if reset = '1' then
                abs_val <= (others => '0');
                neg_reg <= '0';
                valid_reg <= '0';
                addr_out <= (others => '0');
                valid_out <= '0';
                is_neg_out <= '0';

            else

                -- I stadio pipeline: valore assoluto + segno
                if signed(data_in) < 0 then
                    abs_val <= -resize(signed(data_in),17);
                    neg_reg <= '1';
                else
                    abs_val <= resize(signed(data_in),17);
                    neg_reg <= '0';
                end if;

                valid_reg <= valid_in;

                -- II stadio pipeline: clipping e slicing
                if abs_val >= CLIP_VAL then
                    clipped := to_unsigned(16384,16);
                else
                    clipped := unsigned(abs_val(15 downto 0));
                end if;

                addr_out <= clipped(13 downto 4);

                is_neg_out <= neg_reg;
                valid_out <= valid_reg;

            end if;
        end if;
    end process;

end Behavioral;
```

B.13 Blocco di memoria avanzato *(Sigmoid_LUT_V3.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

use work.Sigmoid_Values_V3.all;

entity Sigmoid_LUT_V3 is
    Port (
         clk, reset, valid_in, is_neg_in : in  STD_LOGIC;
         addr_in : in  UNSIGNED(9 downto 0);
         valid_out : out STD_LOGIC;
         data_out : out STD_LOGIC_VECTOR(15 downto 0)
         );
end Sigmoid_LUT_V3;

architecture Behavioral of Sigmoid_LUT_V3 is

    signal rom : lut_type := SIGMOID_TABLE;
    attribute rom_style : string;
    attribute rom_style of rom : signal is "block";

    -- Segnali Pipeline
    signal addr_reg : unsigned(9 downto 0) ;
    signal lut_data : std_logic_vector(15 downto 0);
    signal sign_r1, sign_r2, valid_r1, valid_r2 : std_logic := '0';

    constant ONE : unsigned(15 downto 0) := to_unsigned(32767,16);

begin

    -- I stadio pipeline: registro d'ingresso
    process(clk)
    begin
        if rising_edge(clk) then
            addr_reg <= addr_in;
            sign_r1 <= is_neg_in;
            valid_r1 <= valid_in;
        end if;
    end process;

    -- II stadio pipeline: lettura BRAM
    process(clk)
    begin
        if rising_edge(clk) then
            lut_data <= rom(to_integer(addr_reg));
            sign_r2 <= sign_r1;
            valid_r2 <= valid_r1;
        end if;
    end process;

    -- III stadio pipeline: simmetria + output
    process(clk)
        variable val, outv : unsigned(15 downto 0);

    begin
        if rising_edge(clk) then
            if reset = '1' then
                data_out <= (others => '0');
                valid_out <= '0';
            else
                val := unsigned(lut_data);
                if sign_r2 = '1' then
                    outv := ONE - val;
                else
                    outv := val;
```

```
                end if;

                data_out <= std_logic_vector(outv);
                valid_out <= valid_r2;

            end if;
        end if;
    end process;

end Behavioral;
```

B.14 Top module architettura V3 *(TOP_Design_V3.vhd)*

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Sigmoid_Accelerator_TOP_V3 is
    Port (
         clk, reset, valid_in : in  STD_LOGIC;
         input_data : in  STD_LOGIC_VECTOR(15 downto 0);
         valid_out : out STD_LOGIC;
         output_data : out STD_LOGIC_VECTOR(15 downto 0)
         );
end Sigmoid_Accelerator_TOP_V3;

architecture Structural of Sigmoid_Accelerator_TOP_V3 is

    component Pre_Processing_V3
        Port (
             clk, reset, valid_in : in  STD_LOGIC;
             data_in : in  STD_LOGIC_VECTOR(15 downto 0);
             valid_out, is_neg_out : out STD_LOGIC;
             addr_out : out UNSIGNED(9 downto 0)
             );
    end component;

    component Sigmoid_LUT_V3
        Port (
             clk, reset, valid_in, is_neg_in : in  STD_LOGIC;
             addr_in : in  UNSIGNED(9 downto 0);
             valid_out : out STD_LOGIC;
             data_out : out STD_LOGIC_VECTOR(15 downto 0)
             );
    end component;

    -- Segnali Pipeline
    signal internal_addr : UNSIGNED(9 downto 0);
    signal reg_valid_in, internal_valid, reg_valid_out, internal_is_neg :
STD_LOGIC;
    signal reg_input_data, reg_output_data : STD_LOGIC_VECTOR(15 downto 0);

begin

    -- I stadio pipeline: input buffering
    process(clk)
    begin
        if rising_edge(clk) then

            if reset = '1' then
                reg_input_data <= (others => '0');
                reg_valid_in <= '0';
            else
                reg_input_data <= input_data;
                reg_valid_in <= valid_in;
            end if;
```

```
   end if;
    end process;

    -- II, III stadio pipeline: logica di pre-processing
    PP : Pre_Processing_V3
         port map (clk => clk, reset => reset, valid_in => reg_valid_in,
data_in => reg_input_data, valid_out => internal_valid, is_neg_out =>
internal_is_neg, addr_out => internal_addr);

    -- IV, V, VI stadio pipeline: LUT, simmetria e uscita
    MEM : Sigmoid_LUT_V3
         port map (clk => clk, reset => reset, valid_in => internal_valid,
is_neg_in => internal_is_neg, addr_in => internal_addr, valid_out =>
reg_valid_out, data_out => reg_output_data);

    -- VII stadio pipeline: output buffering
    process(clk)
    begin

        if rising_edge(clk) then
            if reset = '1' then
                output_data <= (others => '0');
                valid_out <= '0';
            else
                if reg_valid_out = '1' then
                    output_data <= reg_output_data;
                else
                    output_data <= (others => '0');
                end if;

                valid_out <= reg_valid_out;
            end if;
        end if;
    end process;

end Structural;
```

B.15 Testbench ad alta risoluzione (*TB_Full_ROM.vhd*)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use STD.TEXTIO.ALL;
use IEEE.STD_LOGIC_TEXTIO.ALL;

entity TB_Sigmoid_V3 is
end TB_Sigmoid_V3;

architecture Behavioral of TB_Sigmoid_V3 is

    component TOP_Design_V3
       Port (
            clk, reset, valid_in : in  STD_LOGIC;
            input_data : in  STD_LOGIC_VECTOR(15 downto 0);
            valid_out : out STD_LOGIC;
            output_data : out STD_LOGIC_VECTOR(15 downto 0)
            );
    end component;

    signal clk, reset, valid_in : STD_LOGIC := '0';
    signal input_data : STD_LOGIC_VECTOR(15 downto 0) := (others => '0');
    signal valid_out : STD_LOGIC;
    signal output_data : STD_LOGIC_VECTOR(15 downto 0);

    constant CLK PERIOD : time := 4 ns;
```

```
    signal stop_sim : boolean := false;

begin

    uut: TOP_Design_V3
        port map (clk=> clk, reset => reset, valid_in => valid_in, input_data
=> input_data, valid_out => valid_out, output_data => output_data);


    -- Generazione clock
    clk_process : process
    begin
        while not stop_sim loop
            clk <= '0';
            wait for CLK_PERIOD/2;
            clk <= '1';
            wait for CLK_PERIOD/2;
        end loop;
        wait;
    end process;


    -- Stimolo
    stim_read : process
        file file_input : text open read_mode is "stimoli_input_1024.mem";
        variable v_line : line;
        variable v_data : std_logic_vector(15 downto 0);
    begin

        reset <= '1';
        valid_in <= '0';
        input_data <= (others => '0');

        wait for 20 * CLK_PERIOD;

        -- release reset su rising edge
        wait until rising_edge(clk);
        reset <= '0';

        for i in 0 to 10 loop
            wait until rising_edge(clk);
        end loop;

        while not endfile(file_input) loop

            readline(file_input, v_line);
            hread(v_line, v_data);

            wait until rising_edge(clk);

            input_data <= v_data;
            valid_in   <= '1';

        end loop;

        wait until rising_edge(clk);
        valid_in <= '0';
        input_data <= (others => '0');

        wait for 50 * CLK_PERIOD;

        stop_sim <= true;

        wait;
    end process;
```

```
    -- Scrittura risultati
    results_writer : process
        file file_output : text open write_mode is "risultati_HW_1024.hex";
        variable v_line : line;

    begin

        wait until reset = '0';
        while not stop_sim loop
            wait until rising_edge(clk);
            if valid_out = '1' then
                hwrite(v_line, output_data);
                writeline(file_output, v_line);
            end if;
        end loop;
        wait;
    end process;

end Behavioral;
```

B.16 Top module architettura parallela V3.1 (*Sigmoid_MultiCore_Top.vhd*)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Sigmoid_MultiCore_Top is
    generic (
        NUM_CORES : integer := 32 --SELEZIONARE QUI NUMERO DI CORES
    );
    Port (
        clk, reset : in  STD_LOGIC;
        valid_in : in  STD_LOGIC_VECTOR(NUM_CORES-1 downto 0);
        input_data : in  STD_LOGIC_VECTOR((NUM_CORES * 16) - 1 downto 0);
        valid_out : out STD_LOGIC_VECTOR(NUM_CORES-1 downto 0);
        output_data : out STD_LOGIC_VECTOR((NUM_CORES * 16) - 1 downto 0)
    );
end Sigmoid_MultiCore_Top;

architecture Structural of Sigmoid_MultiCore_Top is

    constant DATA_WIDTH : integer := 16;

    component Sigmoid_Accelerator_TOP_V3
        Port (
            clk, reset, valid_in : in  STD_LOGIC;
            input_data : in  STD_LOGIC_VECTOR(DATA_WIDTH-1 downto 0);
            valid_out : out STD_LOGIC;
            output_data : out STD_LOGIC_VECTOR(DATA_WIDTH-1 downto 0)
        );
    end component;

begin

    Core_Gen: for i in 0 to NUM_CORES-1 generate
        CORE : Sigmoid_Accelerator_TOP_V3
            port map ( clk => clk, reset => reset, valid_in => valid_in(i),
                       input_data => input_data(((i+1)*DATA_WIDTH)-1 downto
i*DATA_WIDTH),
                       valid_out => valid_out(i),
                       output_data => output_data(((i+1)*DATA_WIDTH)-1 downto
i*DATA_WIDTH)
                       );
    end generate Core_Gen;

end Structural;
```

B.17 Testbench per architettura parallela (*TB_Sigmoid_MultiCore.vhd*)

(Confronto 256 campioni ordinati)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use STD.TEXTIO.ALL;
use IEEE.STD_LOGIC_TEXTIO.ALL;

entity TB_Sigmoid_MultiCore is
end TB_Sigmoid_MultiCore;

architecture Behavioral of TB_Sigmoid_MultiCore is

    constant NUM_CORES : integer := 32;
    constant DATA_WIDTH : integer := 16;
    constant CLK_PERIOD : time := 5 ns;

    signal clk : std_logic := '0';
    signal reset : std_logic := '0';
    signal valid_in : std_logic_vector(NUM_CORES-1 downto 0) := (others =>
'0');
    signal input_data : std_logic_vector(NUM_CORES*DATA_WIDTH-1 downto 0) :=
(others => '0');
    signal valid_out : std_logic_vector(NUM_CORES-1 downto 0);
    signal output_data : std_logic_vector(NUM_CORES*DATA_WIDTH-1 downto 0);

    signal stop_sim : boolean := false;

begin

    uut : entity work.Sigmoid_MultiCore_Top
    port map( clk => clk, reset => reset, valid_in => valid_in,
              input_data => input_data, valid_out => valid_out, output_data =>
output_data);

    -- Processo di generazione del clock
    clk_process : process
    begin
        while not stop_sim loop
            clk <= '0';
            wait for CLK_PERIOD/2;
            clk <= '1';
            wait for CLK_PERIOD/2;
        end loop;
        wait;
    end process;


    -- Processo di lettura:
    stim_read : process
        file file_input : text open read_mode is
"stimoli_input_baseline_p4.mem";
        variable v_line : line;
        variable v_data : std_logic_vector(DATA_WIDTH-1 downto 0);

    begin

        reset <= '1';
        valid_in <= (others => '0');
        input_data <= (others => '0');

        wait for CLK_PERIOD*20;
        wait until falling_edge(clk);
```

```
        reset <= '0';
        wait for CLK_PERIOD*10;

        while not endfile(file_input) loop
            wait until falling_edge(clk);
            valid_in <= (others => '0');
            input_data <= (others => '0');

            for i in 0 to NUM_CORES-1 loop
                if not endfile(file_input) then
                    readline(file_input, v_line);
                    hread(v_line, v_data);
                    input_data((i+1)*DATA_WIDTH-1 downto i*DATA_WIDTH) <=
v_data;
                    valid_in(i) <= '1';
                else
                    valid_in(i) <= '0';
                end if;

            end loop;
        end loop;

        wait until falling_edge(clk);
        valid_in <= (others => '0');
        input_data <= (others => '0');
        wait for CLK_PERIOD * 100;
        stop_sim <= true;
        wait;

    end process;


    -- Processo di scrittura su file:
    results_writer : process

        file file_output : text open write_mode is "risultati_HW.hex";
        variable v_line : line;

    begin

        wait until reset = '0';
        while not stop_sim loop
            wait until falling_edge(clk);
            for i in 0 to NUM_CORES-1 loop
                if valid_out(i) = '1' then
                    hwrite(v_line, output_data((i+1)*DATA_WIDTH-1 downto
i*DATA_WIDTH));
                    writeline(file_output, v_line);

                end if;
            end loop;
        end loop;
        wait;
    end process;

end Behavioral;
```

B.18 Testbench per architettura parallela con dataset reale (*TB_Real.vhd*)

(Confronto 2160 campioni reali)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use STD.TEXTIO.ALL;
use IEEE.STD_LOGIC_TEXTIO.ALL;

entity TB_Real is
end TB_Real;

architecture Behavioral of TB_Real is

    constant NUM_CORES : integer := 32;
    constant DATA_WIDTH : integer := 16;
    constant CLK_PERIOD : time := 5 ns;

    signal clk : std_logic := '0';
    signal reset : std_logic := '0';
    signal valid_in : std_logic_vector(NUM_CORES-1 downto 0) := (others =>
'0');
    signal input_data : std_logic_vector(NUM_CORES*DATA_WIDTH-1 downto 0) :=
(others => '0');
    signal valid_out : std_logic_vector(NUM_CORES-1 downto 0);
    signal output_data : std_logic_vector(NUM_CORES*DATA_WIDTH-1 downto 0);

    signal stop_sim : boolean := false;

begin

    uut : entity work.Sigmoid_MultiCore_Top
    port map( clk => clk, reset => reset, valid_in => valid_in,
              input_data => input_data, valid_out => valid_out, output_data =>
output_data);

    -- Processo di generazione del clock
    clk_process : process
    begin
        while not stop_sim loop
            clk <= '0';
            wait for CLK_PERIOD/2;
            clk <= '1';
            wait for CLK_PERIOD/2;
        end loop;
        wait;
    end process;


    -- Processo di lettura:
    stim_read : process
        file file_input : text open read_mode is "stimoli_real.mem";
        variable v_line : line;
        variable v_data : std_logic_vector(DATA_WIDTH-1 downto 0);

    begin

        reset <= '1';
        valid_in <= (others => '0');
        input_data <= (others => '0');

        wait for CLK_PERIOD*20;
        wait until falling_edge(clk);
        reset <= '0';
```

```
        wait for CLK_PERIOD*10;

        while not endfile(file_input) loop
            wait until falling_edge(clk);
            valid_in <= (others => '0');
            input_data <= (others => '0');

            for i in 0 to NUM_CORES-1 loop
                if not endfile(file_input) then
                    readline(file_input, v_line);
                    hread(v_line, v_data);
                    input_data((i+1)*DATA_WIDTH-1 downto i*DATA_WIDTH) <=
v_data;
                    valid_in(i) <= '1';
                else
                    valid_in(i) <= '0';
                end if;

            end loop;
        end loop;

        wait until falling_edge(clk);
        valid_in <= (others => '0');
        input_data <= (others => '0');
        wait for CLK_PERIOD * 100;
        stop_sim <= true;
        wait;

    end process;


    -- Processo di scrittura su file:
    results_writer : process

        file file_output : text open write_mode is "risultati_real.hex";
        variable v_line : line;

    begin

        wait until reset = '0';
        while not stop_sim loop
            wait until falling_edge(clk);
            for i in 0 to NUM_CORES-1 loop
                if valid_out(i) = '1' then
                    hwrite(v_line, output_data((i+1)*DATA_WIDTH-1 downto
i*DATA_WIDTH));
                    writeline(file_output, v_line);

                end if;
            end loop;
        end loop;
        wait;
    end process;

end Behavioral;
```

Appendice C: Grafici, risultati e metriche di confronto dell'errore (MATLAB)

C.1 Grafico sigmoide *(sigmoide.m)*

```
sigmoid = @(x) 1 ./ (1 + exp(-x));
sigmoid_prime = @(x) sigmoid(x) .* (1 - sigmoid(x));

x = -10:0.1:10;
y = sigmoid(x);
y_prime = sigmoid_prime(x);

figure;
grid on;
hold on;

plot(x, y, 'LineWidth', 2, 'Color', 'b');
plot(x, y_prime, 'LineWidth', 1, 'Color', 'r');

title('Grafico Sigmoide e Derivata (\sigma(x))');
xlabel('Input (x)');
ylabel('Output (y)');
ylim([-1 2]);

plot([-10 10], [1 1], 'k--');
plot([-10 10], [0 0], 'k--');
plot(0, 0.5, 'o', 'MarkerSize', 8, 'MarkerFaceColor', 'g');
plot(0, 0.25, 'd', 'MarkerSize', 7, 'MarkerFaceColor', 'yellow');

legend('\sigma(x)', 'Derivata \sigma''(x)', 'Asintoto Superiore', 'Asintoto
Inferiore','Punto Centrale', 'Max derivata');

yticks([0 0.25 0.5 1]);

hold off;
```

C.2 Calcolo dell'errore e metriche di confronto *(confronto_HW_SW.m)*

```
% Carica il Golden Model
golden_model = load('sigmoid_outputs_sw.txt');

% Carica i valori HEX di Vivado
hex_data = textread('risultati_HW_256.hex', '%s');

% Conversione e Sincronizzazione
n = min(length(golden_model), length(hex_data));
hw_output = zeros(n, 1);
sw_ideal = golden_model(1:n);

for i = 1:n
    val_hw = hex2dec(hex_data{i});

    % Gestione del segno per Q5.11
    if val_hw >= 32768, val_hw = val_hw - 65536; end
    hw_output(i) = val_hw / 2048;
end

% Calcolo Errori
diff_err = sw_ideal - hw_output;
mae = mean(abs(diff_err));
max_err = max(abs(diff_err));

fprintf('MAE: %.10f\n', mae);
fprintf('Errore Massimo: %.10f\n', max_err);

% Grafici
figure('Name', 'Confronto HW - SW', 'Color', 'w');

% SW
subplot(3,1,1);
plot(sw_ideal, 'b', 'LineWidth', 1);
title('Risultati SW (Golden Model Python)','FontSize', 18 );
ylabel('sigma(x)','FontSize', 16);
grid on;

% - HW
subplot(3,1,2);
plot(hw_output, 'r', 'LineWidth', 1);
title('Risultati HW (Vivado)','FontSize', 18);
ylabel('sigma(x)','FontSize', 16);
grid on;

% Errore
subplot(3,1,3);
plot(diff_err, 'm', 'LineWidth', 1);
title('Errore di Quantizzazione','FontSize', 18);
xlabel('Campioni','FontSize', 16);
ylabel('Differenza','FontSize', 16);
grid on;

linkaxes(findall(gcf,'type','axes'),'x');
```

C.3 Calcolo dell'errore e metriche di confronto mod Q1.15 *(confronto_HW_SW_V3.m)*

```
% Carica il Golden Model
golden_model = load('sigmoid_outputs_sw.txt');

% Carica i valori HEX di Vivado
hex_data = textread('risultati_HW.hex', '%s');

% Conversione e Sincronizzazione
n = min(length(golden_model), length(hex_data));
hw_output = zeros(n, 1);
sw_ideal = golden_model(1:n);

for i = 1:n
    val_hw = hex2dec(hex_data{i});

    % Gestione segno per Q1.15
    if val_hw >= 32768, val_hw = val_hw - 65536; end
    hw_output(i) = val_hw / 32768;
end

% Calcolo Errori
diff_err = sw_ideal - hw_output;
mae = mean(abs(diff_err));
max_err = max(abs(diff_err));

fprintf('MAE: %.10f\n', mae);
fprintf('Errore Massimo: %.10f\n', max_err);

% Grafici
figure('Name', 'Confronto HW - SW', 'Color', 'w');

% SW
subplot(3,1,1);
plot(sw_ideal, 'b', 'LineWidth', 1);
title('Risultati SW (Golden Model Python)','FontSize', 18 );
ylabel('sigma(x)','FontSize', 16);
grid on;

% - HW
subplot(3,1,2);
plot(hw_output, 'r', 'LineWidth', 1);
title('Risultati HW (Vivado)','FontSize', 18);
ylabel('sigma(x)','FontSize', 16);
grid on;

% Errore
subplot(3,1,3);
plot(diff_err, 'm', 'LineWidth', 1);
title('Errore di Quantizzazione','FontSize', 18);
xlabel('Campioni','FontSize', 16);
ylabel('Differenza','FontSize', 16);
grid on;

linkaxes(findall(gcf,'type','axes'),'x');
```

C.4 Calcolo dell'errore e metriche di confronto V3.1 *(confronto_HW_SW_V3_1.m)*

(Confronto 256 campioni ordinati)

```
% Carica il Golden Model
golden_model = load('sigmoid_outputs_sw.txt');

% Carica i valori HEX di Vivado
hex_data = textread('risultati_HW_p4.hex', '%s');

% Conversione e Sincronizzazione
n = min(length(golden_model), length(hex_data));
hw_output = zeros(n, 1);
sw_ideal = golden_model(1:n);

for i = 1:n
    val_hw = hex2dec(hex_data{i});

    % Gestione segno per Q5.11
    if val_hw >= 32768, val_hw = val_hw - 65536; end
    hw_output(i) = val_hw / 32768;
end

% Calcolo Errori
diff_err = sw_ideal - hw_output;
mae = mean(abs(diff_err));
max_err = max(abs(diff_err));

fprintf('MAE: %.10f\n', mae);
fprintf('Errore Massimo: %.10f\n', max_err);

% Grafici
figure('Name', 'Confronto HW - SW', 'Color', 'w');

% SW
subplot(3,1,1);
plot(sw_ideal, 'b', 'LineWidth', 1);
title('Risultati SW (Golden Model Python)','FontSize', 18 );
ylabel('sigma(x)','FontSize', 16);
grid on;

% - HW
subplot(3,1,2);
plot(hw_output, 'r', 'LineWidth', 1);
title('Risultati HW (Vivado)','FontSize', 18);
ylabel('sigma(x)','FontSize', 16);
grid on;

% Errore
subplot(3,1,3);
plot(diff_err, 'm', 'LineWidth', 1);
title('Errore di Quantizzazione','FontSize', 18);
xlabel('Campioni','FontSize', 16);
ylabel('Differenza','FontSize', 16);
grid on;

linkaxes(findall(gcf,'type','axes'),'x');
```

C.5 Calcolo dell'errore e metriche di confronto V3.1 *(confronto_HW_SW_V3_1.m)*

(Confronto 2160 campioni reali)

```
% Carica il Golden Model
golden_model = load('2160_outputs_float.txt');

% Carica i valori HEX di Vivado
hex_data = textread('risultati_real.hex', '%s');

% Conversione e Sincronizzazione
n = min(length(golden_model), length(hex_data));
hw_output = zeros(n, 1);
sw_ideal = golden_model(1:n);

for i = 1:n
    val_hw = hex2dec(hex_data{i});

    % Gestione segno per Q5.11
    if val_hw >= 32768, val_hw = val_hw - 65536; end
    hw_output(i) = val_hw / 32768;
end

% Calcolo Errori
diff_err = sw_ideal - hw_output;
mae = mean(abs(diff_err));
max_err = max(abs(diff_err));

fprintf('MAE: %.10f\n', mae);
fprintf('Errore Massimo: %.10f\n', max_err);

% Grafici
figure('Name', 'Confronto HW - SW', 'Color', 'w');

% SW
subplot(3,1,1);
plot(sw_ideal, 'b', 'LineWidth', 1);
title('Risultati SW (Golden Model Python)','FontSize', 18 );
ylabel('sigma(x)','FontSize', 16);
grid on;

% - HW
subplot(3,1,2);
plot(hw_output, 'r', 'LineWidth', 1);
title('Risultati HW (Vivado)','FontSize', 18);
ylabel('sigma(x)','FontSize', 16);
grid on;

% Errore
subplot(3,1,3);
plot(diff_err, 'm', 'LineWidth', 1);
title('Errore di Quantizzazione','FontSize', 18);
xlabel('Campioni','FontSize', 16);
ylabel('Differenza','FontSize', 16);
grid on;

linkaxes(findall(gcf,'type','axes'),'x');
```

C.6 Test elaborazione CPU Single-Thread *(CPU_test_Single_Thread.m)*

```
clc;
clear;

input_data = load('input_valori.txt');
N = length(input_data);
output_loop = zeros(N, 1);


% 1. Misura tempo di elaborazione
tic;
for i = 1:N
    output_loop(i) = 1 / (1 + exp(-input_data(i)));
end
tempo_ms = toc * 1000;

fprintf('TEST 1: SINGLE-THREAD \n');
fprintf('Tempo di elaborazione: %.2f ms\n\n', tempo_ms);

disp('Premere INVIO per avviare lo stress test di 60 secondi...');
pause;


% 2. Stress test di 60 secondi per analisi del consumo
fprintf('\n Stress test in corso...\n');
tic;
while toc < 60
    for i = 1:N
        output_loop(i) = 1 / (1 + exp(-input_data(i)));
    end
end

fprintf('Stress test terminato. \n');
fprintf('=== TEST 1 COMPLETATO ===\n');
```

C.7 Test elaborazione CPU Multi-Thread *(CPU_test_Multi_Thread.m)*

```
clc;
clear;

input_data = load('input_valori.txt');

% 1. Misura tempo di elaborazione
tic;
output_vec = 1 ./ (1 + exp(-input_data));
tempo_ms = toc * 1000;

fprintf('TEST 2: MULTI-THREAD VETTORIZZATO \n');
fprintf('Tempo di elaborazione: %.2f ms\n\n', tempo_ms);

disp('Premere INVIO per avviare lo stress test di 60 secondi...');
pause;

% 2. Stress test di 60 secondi per analisi del consumo
fprintf('\n Stress test in corso...\n');
tic;
while toc < 60
    output_vec = 1 ./ (1 + exp(-input_data));
end
fprintf('Stress test terminato. \n');
fprintf('=== TEST 2 COMPLETATO ===\n');
```

Ringraziamenti

Ringrazio la Prof.ssa Stefania Perri per avermi permesso di realizzare questo lavoro, e per avermi invogliato sin dall'inizio a intraprendere il percorso di laurea. Con la sua gentilezza e disponibilità, è stata per me di grande ispirazione.

Ringrazio mio padre per avermi insegnato a ragionare, ad amare la matematica e la tecnologia, e per avermi continuamente sostenuto in ogni decisione. È merito suo se ho fatto le scelte migliori per il mio futuro.

Ringrazio mia madre per avermi sempre dato amore immenso, la possibilità di vivere la vita che desideravo, e per avermi aiutato a realizzare i miei sogni.

Ringrazio i miei amici per tutti i bei momenti passati insieme durante gli anni universitari.

Ringrazio me stesso per non essermi mai arreso.

Danilo Antonio Tarantino